eazy
bytes

Master Spring, Spring Boot, REST, JPA, Spring Security

Start your journey into Java backend development with Spring Boot.
Build REST APIs, handle data with JPA, secure your applications using Spring Security,
and gain the skills needed to build real-world backend systems.

eazy bytes

Why Learn Backend Development Using Spring Boot?

Backend developers are the brains behind the application. You design APIs, handle data, secure the system, and make sure everything actually works when users start clicking buttons like crazy.

Spring Boot is the Swiss Army knife of Java backend development — REST APIs, databases, security, caching, transactions — all in one powerful stack.

High demand, strong salaries, and never-ending real-world problems to solve. Boredom doesn't stand a chance.

Whether you're building banking systems, e-commerce platforms, job portals or the next startup idea — solid backend skills with Spring Boot are where the real magic happens

eazy bytes

How this course is structured

Core Foundations

- Spring Core fundamentals
- Maven and project setup
- IoC, DI, Beans, and ApplicationContext
- AOP (Aspect Oriented Programming)

Spring Boot & REST APIs

- Spring Boot essentials
- REST API development from scratch
- Request/response anatomy
- API design best practices
- Versioning strategies

Databases & JPA

- H2, MySQL Databases
- Spring Data JPA, Entity mappings
- JPA Relationships, Pagination, sorting
- Advanced queries
- Transactions & performance optimizations

Security

- Security internals & filter chains
- Authentication vs Authorization
- Password encoding and hashing
- JWT-based authentication
- CSRF protection, CORs config

Production-Ready Features

- Logging strategies, Global Exception handling
- Validations, Caching
- Profiles & configuration management
- Actuator, observability using OpenTelemetry

Deployment & Cloud

- Prepare Spring Boot apps for production
- Work with Docker and MySQL
- Deploy applications to AWS

eazy bytes

Pre-requisites

Java

Just Java basics. If you know classes, objects, and basic Java syntax, you're good to go. I'll teach you everything else.

Willingness to Learn and Break Things

A Laptop

Why Core Java Alone Isn’t Enough for Web Development ?

eazy bytes

If you're watching this, you already know **Core Java** - you understand classes, objects, inheritance, collections, and maybe even some multithreading. That's fantastic!

But now you're probably wondering: **'I can build applications with Java... so why do I need Spring?’**

That's exactly what we're going to answer in this section. By the end of these introductory lectures, you'll understand:

- **What Spring and Spring Boot actually are**
- **How they're different from plain Java**
- **Why companies across the globe demand these skills**
- **And most importantly, how they'll make you a more productive developer**

Let's dive in!"

The Problem with Plain Java

eazy bytes

Imagine you're building a web application for an e-commerce site using only Core Java.
What You Need to Do ?

1. Database Connection Management

```
// You'd have to write code like this everywhere:
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, username, password);
    PreparedStatement stmt = conn.prepareStatement("SELECT * FROM
                                                   products");
    ResultSet rs = stmt.executeQuery();
    // Process results...
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    if (conn != null) conn.close(); // Don't forget to close!
}
```

Every single database operation requires this boilerplate code. If you forget to close connections, you'll have memory leaks. You need to manage connection pools manually.

The Problem with Plain Java

eazy bytes

2. Object Creation and Dependencies (pseudo-code)

```
Class EmailService:
    Method sendEmail():
        print "Sending email..."

Class UserService:
    emailService = new EmailService() // Creating dependency manually

    Method registerUser():
        print "User registered successfully."
        emailService.sendEmail() // Uses EmailService directly

Main:
    userService = new UserService()
    userService.registerUser()
```

Problems

- UserService is tightly coupled with EmailService.
- If you want to switch to a new service (like SMSService or WhatsAppService),
- you must edit and recompile UserService.
- Testing is harder because UserService creates its own dependencies.

The Problem with Plain Java

eazy bytes

3. Configuration Management

You'd manually read property files:

```
Properties props = new Properties();
FileInputStream fis = new FileInputStream("config.properties");
props.load(fis);
String dbUrl = props.getProperty("db.url");
```

4. Creating REST APIs

With plain Java, you'd use Servlets:

```
public class ProductServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) {
        // Manually parse parameters
        // Manually convert objects to JSON
        // Manually set response headers
        // Handle exceptions manually
    }
}
```

Then configure everything in web.xml, handle URL mappings, manage servlet lifecycle...

The Problem with Plain Java

eazy bytes

The Core Problems with Java alone:

- Too much boilerplate code - You spend more time on plumbing than business logic
- Manual object management - You're responsible for creating and wiring everything
- Tight coupling - Hard to test, hard to maintain
- Configuration nightmare - XML files everywhere
- Reinventing the wheel - Writing the same patterns over and over

This is where Spring comes in!"

"**Spring Framework is NOT a programming language** - let me be very clear about that. Java is the language; Spring is a framework built on top of Java.

Think of it this way:

- Java gives you the tools: hammer, nails, wood
- Spring gives you the blueprint and power tools to build a house faster and better

Introduction to Spring

eazy bytes

What is Spring ?

Spring is a popular Java framework for building web applications.

It helps developers write clean, modular, and testable code.

Provides built-in features for handling databases, security, messaging, and more.

Why Spring ?

Makes Java development easier. Helps in reducing boilerplate code.

Supports different types of applications – web apps, microservices, cloud apps, etc.

Provides dependency injection (helps in managing object dependencies).

Works well with other technologies (like Hibernate, Kafka, RabbitMQ).

Introduced in 2003, the Spring Framework is a comprehensive solution designed to simplify Java enterprise development. It was created to tackle the challenges of building large-scale applications by offering a unified programming model and seamless integration with various technologies.

Spring Release timeline

eazy bytes

- The first version of Spring was created by Rod Johnson, introduced with his book Expert One-on-One J2EE Design and Development in October 2002.
- Spring emerged in 2003 to simplify the complexities of early J2EE specifications. Instead of competing with Java EE, Spring complements it by integrating only selected features from the Java EE ecosystem.
- Over the years, Spring has continued to grow and evolve. Beyond the main framework, other popular projects like Spring Boot, Spring Security, Spring Data, Spring Cloud, and Spring Batch have been introduced.

What Spring Actually Does ?

eazy bytes

Inversion of Control (IoC) & Dependency Injection

Remember our tightly coupled UserService? Here's the Spring way (pseudo-code):

```
Interface NotificationService:
    Method sendNotification()

Class EmailService implements NotificationService:
    Method sendNotification():
        print " Sending Email Notification..."

Class SmsService implements NotificationService:
    Method sendNotification():
        print " Sending SMS Notification..."

Class UserService:
    notificationService (injected by Spring)    // No "new" keyword!

    Method registerUser():
        print "User registered successfully."
        notificationService.sendNotification()
```

What happened?

- Spring injects EmailService automatically
- Just switch bean or use @Qualifier
- Loosely coupled
- Easy to mock dependencies

Similarly Spring can manage transactions, DB connections, logging, properties management and many others which we will explore in this course.

Over time, Spring grew to include many projects — like Spring MVC (for web apps), Spring Data (for databases), Spring Security (for authentication), Spring Cloud (for microservices) and so on...

How Spring Boot Changed Everything

eazy bytes

Now, let’s talk about Spring Boot — the game changer.

Before Spring Boot, developers had to manually configure everything — XML files, dependency jars, and long setup steps.

Traditional Spring Application:

- Configure dispatcher servlet (50 lines of XML)
- Set up component scanning (XML)
- Configure data source (more XML)
- Set up view resolvers
- Deploy to external Tomcat server
- Configure server settings

Spring Boot simplified all that with:

- Auto-configuration: It detects what you have in your project and configures it automatically.
- Embedded servers: No need to deploy to Tomcat manually — just run your app like any Java program.
- Starter dependencies: Pre-packaged dependency sets like spring-boot-starter-web or spring-boot-starter-data-jpa.
- And many more features

Spring Boot helps you go from idea to working API in minutes. That’s why, most companies use Spring Boot because it includes Spring Framework but makes development 10x faster!"

Spring Boot is NOT a replacement for Spring, but rather an extension that makes Spring easier to use.

Java vs Spring vs Spring Boot

eazy bytes

Analogy: Mail → Landline → Mobile

Stage	Real-world example	Development equivalent	Explanation
Mail	Send a letter → wait for delivery	Core Java	Everything is manual — you write and handle every step yourself. You have full control, but it's slow, repetitive, and prone to errors.
Landline	Easier communication, but still limited to one location	Spring Framework	Automates some work — like dependency management and object creation. Communication (development) becomes easier but setup/configuration still takes effort (like wiring a fixed phone line).
Mobile	Always ready, portable, instant setup	Spring Boot	Everything is built-in — instant startup, auto-configuration, embedded servers, one-click deployment. You can "make a call" (run your app) anywhere, anytime.

Java vs Spring vs Spring Boot

eazy bytes

Car Analogy: Manual → Automatic → Self Driving

Stage	Real-world example	Development equivalent	Explanation
Manual Car	You do everything manually — clutch, gear shifting, starting the car, adjusting mirrors, etc.	Core Java	In Core Java, you manually create and manage objects, handle dependencies, configurations, and setup. Full control, but it's slow and error-prone.
Automatic Car	The car takes care of gear shifting and some mechanical operations. You still need to drive, but it's easier and smoother.	Spring Framework	Spring automates object creation and dependency management through Dependency Injection (IoC). You still configure your project but write less boilerplate code.
Self-Driving Car	You just enter your destination — the car handles everything: acceleration, navigation, and safety.	Spring Boot	Spring Boot auto-configures, embeds servers, manages dependencies, and lets you start apps instantly. You focus purely on the business logic.

eazy bytes

Maven Build Tool

"Imagine you're building a house. You need bricks, cement, windows, doors, and many other materials. Now, what if you had to go to different stores across the city to buy each item separately? That would be exhausting, right? This is exactly what developers faced before Maven came along. Let me show you how Maven changed everything..."

Why do we even need Maven?

eazy bytes

When we write Core Java programs, we usually deal with just a few .java files. However, once we enter the world of enterprise applications — like Spring and Spring Boot — we rely on hundreds of classes provided by:

- Frameworks
- Third-party libraries
- External tools

These are called dependencies.

Example:
- Spring Web library for building web services
- MySQL driver for database access
- JSON libraries like Jackson

We can’t write all of these ourselves. We reuse them.

For a simple web app, you might need 50+ JAR files!
With out Maven, Developers had to manually search and download each JAR file from different websites

The Pain: Before Maven

eazy bytes

Before tools like Maven existed:

- Developers had to manually search the internet for jar files like spring-web.jar, mysql-connector.jar, jackson.jar …
- Download them one by one
- Place them inside a folder like lib/
- Add them manually to the classpath

And if one jar depended on 10 more jars?
Congratulations — more downloading!

Also, every developer in a team had to repeat the same painful steps.

And when you moved the project to another machine…
missing jars → project breaks!

Enter Maven — The Hero

eazy bytes

Maven automates all of this!

You simply tell Maven your project's requirements (dependencies) in one file called:

`pom.xml`

(POM = Project Object Model)

Inside pom.xml, you write something like:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
     <version>X.X.X</version>
</dependency>
```

Every library has a unique address (like a home address):

- GroupId: Company/Organization (like the city)
- ArtifactId: Project name (like the street)
- Version: Specific version (like the house number)

Enter Maven — The Hero

eazy bytes

That’s it!

Maven will:

- Search for the library
- Download it automatically
- Download all its required libraries (called transitive dependencies)
- Keep them in the correct classpath

No manual effort needed.

Where does Maven download from?

Maven downloads dependencies from a central place on the internet called the:

```
Maven Central Repository
```

Think of it like Amazon for Java libraries.

Enter Maven — The Hero

eazy bytes

Local Repository

Once downloaded, these libraries are stored on your machine in a Maven folder called local repository.

Path example (for Mac):

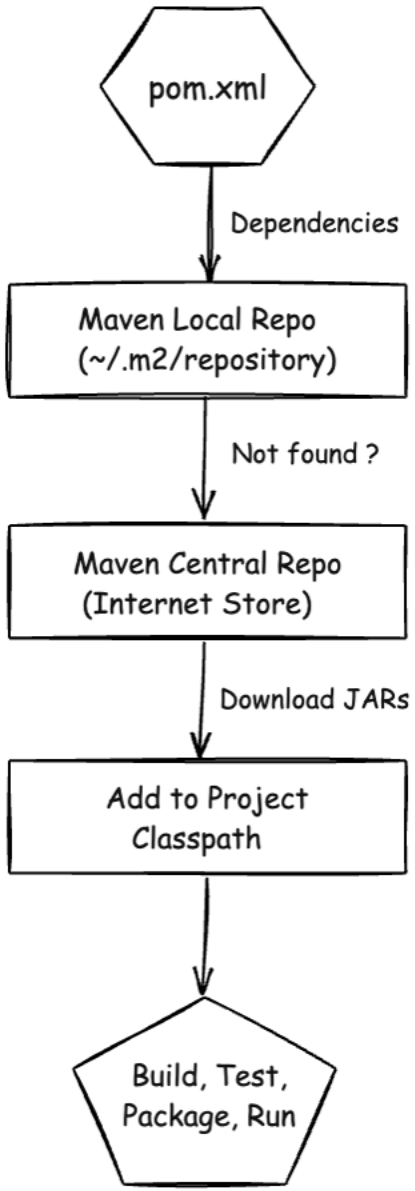
```
~/.m2/repository
```

The cool part? If another project needs the same jar,

- Maven simply reuses it locally
- No need to download again

This saves:

- Time
- Network bandwidth



Enter Maven — The Hero

eazy bytes

Teamwork Advantage

When you share your code with someone else:

- They don’t need to hunt for jars
- pom.xml already has dependency information

They run: `mvn install`

→ Maven downloads everything automatically.

This makes your project:

- Portable
- Easy to share
- Less error-prone

Build Automation

Apart from dependency management, Maven can:

- Compile your Java code
- Run unit tests
- Package applications (JAR/WAR)
- Create reports
- Deploy artifacts

All using simple commands like:

```
mvn clean install
mvn package
```

No complicated manual scripts required.

Maven Project Structure

eazy bytes

Maven loves organization. Every Maven project follows the same structure - like how every McDonald's looks similar worldwide:

```
maven-app/
├── pom.xml                    → Your shopping list
├── src/
│   ├── main/
│   │   ├── java/              → Your Java code goes here
│   │   └── resources/         → Config files, properties
│   └── test/
│       ├── java/              → Your test code
│       └── resources/         → Test configurations
└── target/                    → Compiled code (Maven creates this)
```

This standard structure means:

- Any developer can understand any Maven project instantly
- Tools and IDEs know exactly where to find things
- No more 'Where do I put this file?' confusion"

Common Maven Commands

eazy bytes

Maven is not just a dependency manager. It is also a build automation tool. It can:

- Compile code
- Run tests
- Package jar/war
- Deploy artifacts
- Clean old builds

All of this happens using simple commands. Whenever we run a Maven command, the tool performs a set of predefined tasks called goals.

1. **mvn clean -** Removes old compiled files.

```
mvn clean
```

This deletes the target/ folder. Why we need to run this command ?

- To avoid conflicts with older builds
- To ensure a fresh start

Common Maven Commands

eazy bytes

2. **mvn validate -** checks if our project is correctly configured

```
mvn validate
```

It checks:

- Is pom.xml valid XML?
- Are all required fields present? (groupId, artifactId, version)
- Is the project structure correct?
- Are all necessary plugins available?

3. **mvn compile -** Compiles our Java source code

```
mvn compile
```

This reads files from `src/main/java` and generates .class files into target/

4. **mvn test -** Runs all unit tests

```
mvn test
```

Looks inside `src/test/java`. Only after successful tests can we say the app is stable.

Common Maven Commands

eazy bytes

5. **mvn package -** Packages the project into jar (Java Archives) OR war (Web Archives)

```
mvn package
```

Output is generated in the target folder (`target/myapp-0.0.1-SNAPSHOT.jar`)
This is the file we can run or deploy.

6. **mvn verify -** It ensures the build is ready for final installation or deployment.

```
mvn verify
```

It builds your project, tests it, and verifies everything is working without actually deploying it. If something fails (like a test), it will let you know so you can fix it.

7. **mvn install -** Most important/common command

```
mvn install
```

It compiles -> tests -> packages -> verify and then installs the final .jar into local Maven repository (.m2 folder). This allows other projects on your machine to reuse it.

Common Maven Commands

eazy bytes

8. **mvn deploy -** uploads your project's JAR/WAR file to a remote Maven repository for sharing with other developers/projects.

```
mvn deploy
```

Deploys the artifact into a remote repository (Nexus, Artifactory). Others in the organization can reuse your library.

9. **mvn site -** used to generate a project documentation website for a Maven project — automatically

```
mvn site
```

It takes information from your project (POM, reports, dependencies, plugins, test coverage, etc.) Then creates a HTML documentation site you can open in a browser.

Common Maven Commands

eazy bytes

The Complete Flow

```
validate → compile → test → package → verify  →  install  →  deploy
                              ↓                     ↓           ↓
                          Creates JAR          Saves locally  Uploads to remote
```

Shortcut Tip

You don’t need to run commands separately. For Example: Running `mvn install` automatically runs:

- clean (optional)
- compile
- test
- package
- Install

All in sequence!

Common Maven Commands

eazy bytes

Real-World Analogy

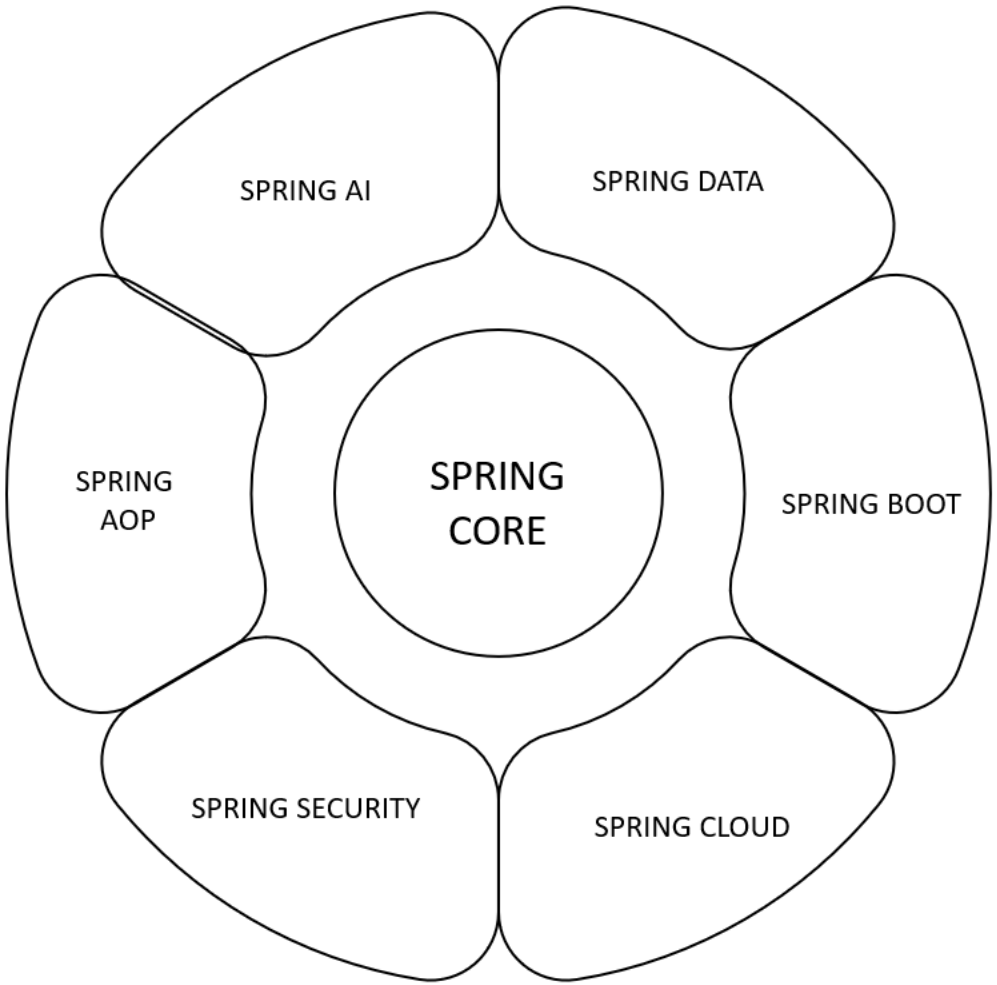
Imagine you want to bake a cake . Steps that you follow,

- `Validate ingredients (validate)`
- `Mix them (compile)`
- `Taste batter (test)`
- `Bake cake (package)`
- `Place in local fridge (install)`
- `Deliver to bakery (deploy)`

These steps must follow order. You can’t bake before mixing!
Maven works exactly the same.

Introduction to Spring Core

eazy bytes



What is Spring Core ?

Spring Core is the foundation of the entire Spring ecosystem. It provides the fundamental principles, base classes, and core mechanisms that power all other Spring modules and projects.

The entire Spring Framework and its related projects are built on top of Spring Core.

Key Components of Spring Core

Spring Core consists of the following essential components:

IoC (Inversion of Control) – Shifts control of object creation to the Spring Container.

DI (Dependency Injection) – Manages dependencies automatically, reducing tight coupling.

Beans – Objects managed by the Spring IoC container.

IoC Container – The core mechanism that creates, manages, and provides dependencies for beans

Context – Provides runtime configuration and manages the lifecycle of beans.

Inversion of Control (IoC)

eazy bytes

Inversion of Control (IoC) is a software design principle that defines how objects are created and managed in a program. It doesn't create the objects itself but outlines a way for their creation and dependency management.

With IoC, the control flow of the program is reversed. Instead of the programmer managing the flow of the application, a framework or service takes over this responsibility, ensuring objects and their dependencies are handled automatically.

- ✓ Normally, in Java, we create objects manually using new keyword.
- ✓ With IoC, Spring manages object creation and dependencies automatically.
- ✓ It reduces manual coding and makes applications more flexible.

Dependency Injection (DI)

eazy bytes

Dependency Injection is a design pattern used to implement Inversion of Control (IoC).

With Dependency Injection, the responsibility of creating and managing objects is transferred from the application to the Spring IoC container. This approach reduces coupling between objects by allowing the framework to dynamically provide the required dependencies at runtime.

- ✓ DI is a way of providing dependencies to a class instead of creating them inside the class.
- ✓ Helps in bringing loose coupling between objects
- ✓ Spring automatically injects required objects.

Advantages of IoC & DI

- Loose coupling between components
- Minimizes the amount of code
- Makes Unit testing easy with different mocks
- Increased system maintainability & module reusability
- Allow concurrent and independent development
- Replacing modules has no side effect on other modules

Loose Coupling

eazy bytes

Loose coupling means that different parts of a system are not tightly dependent on each other. Instead, they are connected in a way that changes in one part have little or no impact on others.

Imagine using a TV remote. The remote can control different TVs, and the TV can work with different remotes. Neither depends on the specific details of the other—they just need to follow a common way of communicating (like the infrared signals).

Why Loose Coupling Is Important

Easier to Modify: You can change one part of the system without breaking others.

Better Reusability: Components can be reused in different contexts since they don’t depend on specific implementations.

Improved Flexibility: You can replace or upgrade parts of the system without affecting others.

Spring Beans

eazy bytes

A Spring Bean is any normal Java class that is instantiated and managed by the Spring IoC (Inversion of Control) container. These beans form the backbone of a Spring application, representing its key components or services.

How Beans Are Created:

Beans are instantiated by the container based on the configuration metadata developer provide, which can be using:

- XML configurations (Legacy approach)
- Annotations (e.g., @Component, @Service)
- Java-based configuration (via @Configuration and @Bean methods)

Bean Lifecycle Management:

The Spring IoC container manages the entire lifecycle of a bean, including its creation, initialization, scope, and destruction.

It also ensures that any required dependencies are injected into the bean as needed.

Spring IoC Container (BeanFactory)

eazy bytes

The IoC (Inversion of Control) Container is the core of the Spring Framework. Its main job is to manage the objects (also known as Spring Beans) in your application. It is responsible for:

- Dependency Injection: Managing object creation and wiring dependencies between objects
- Bean Management: Creating, configuring, and assembling beans based on configuration metadata
- Lifecycle Management: Handling the complete lifecycle of beans from instantiation to destruction

The IoC Container is represented by the BeanFactory interface, which provides the basic functionality for managing beans.

org.springframework.beans.factory.BeanFactory:

- ✓ The simplest IoC container in Spring.
- ✓ It provides the basic functionality for managing Spring Beans.
- ✓ Suitable for lightweight applications where performance is critical and fewer features are needed.

Spring Context (ApplicationContext)

eazy bytes

The Spring Context (org.springframework.context.ApplicationContext) is a more advanced container that:

- Extends BeanFactory: Inherits all IoC Container capabilities
- Adds Enterprise Features:

  - ✓ Internationalization Support (i18n) - Message sources for multiple languages.
  - ✓ Application Events - Publish / listen to events inside Spring.
  - ✓ Resource Loading - Read resources from classpath, filesystem, URL
  - ✓ Environment & Profiles - application.properties, active profiles, etc.
  - ✓ Auto-scan & Annotations - Component Scanning (@Component, @Service)
  - ✓ AOP Integration - Aspect Oriented Programming support.

```
ApplicationContext (Spring Context)
        |
        └── extends BeanFactory (IoC Container)
```

Analogy

IoC Container = Coffee machine engine

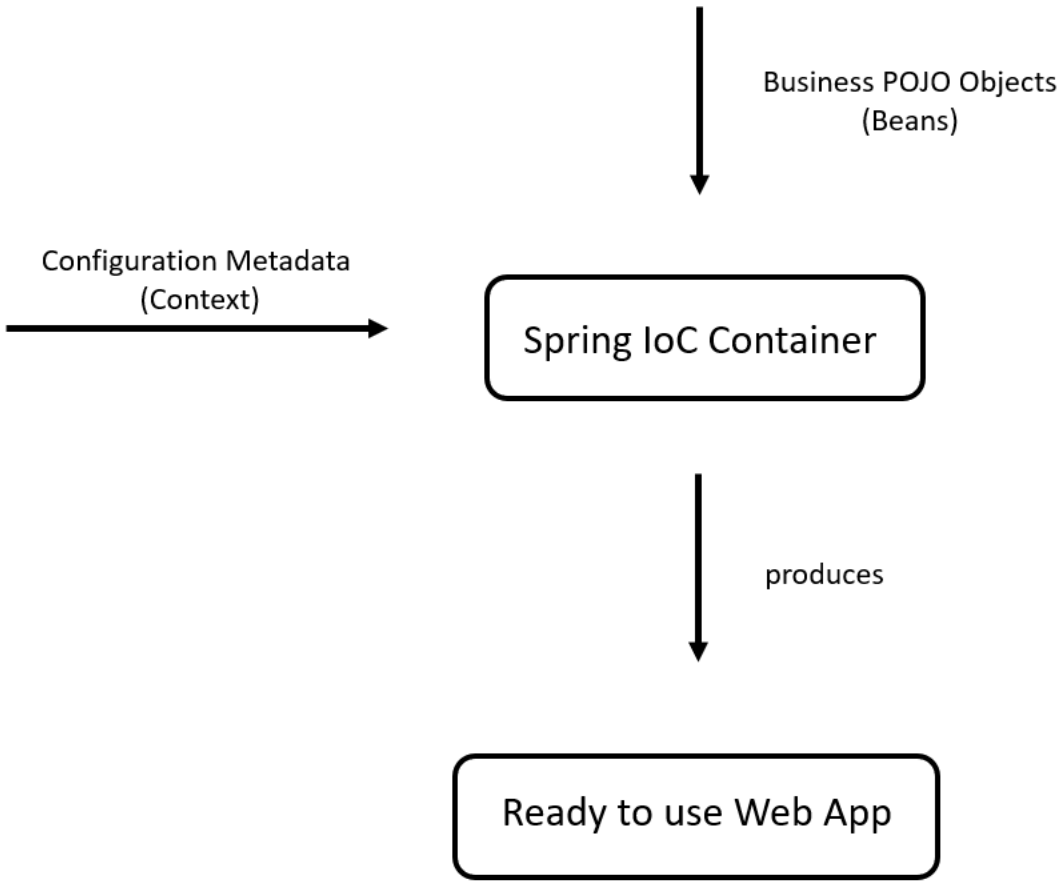
- Makes coffee (creates beans)

Spring Context = Full vending machine

- Makes coffee
- Provides milk, sugar, snacks
- Shows messages
- Accepts coins
- Gives receipts
- More features, still using the engine.

What happens at the core of Spring ?

eazy bytes



The diagram below provides a high-level overview of how Spring works. Your application classes are combined with configuration metadata, and once the ApplicationContext is created and initialized, you get a fully configured, ready-to-run application.

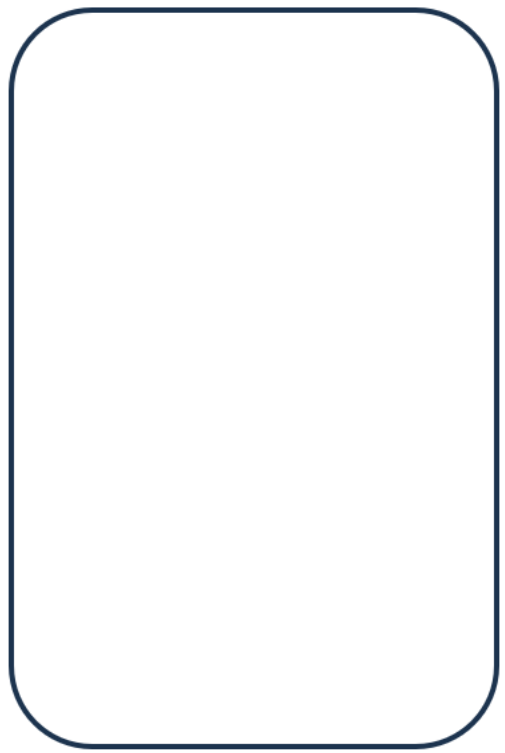
How to add Beans into Spring Context ?

eazy bytes

When we create an java object with new () operator directly as shown below, then your Spring Context/Spring IoC Container will not have any clue of the object.

```
Vehicle veh = new Vehicle();
```

Spring Context

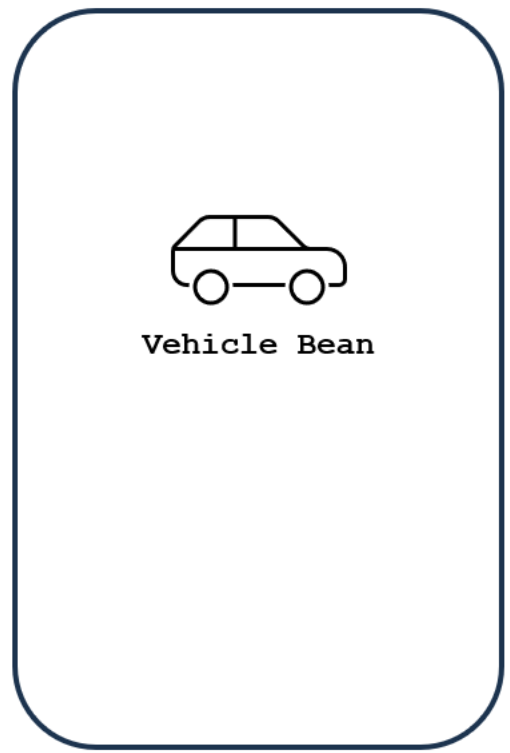


@Configuration is a Spring annotation used to mark a class as a source of bean definitions

@Bean annotation lets Spring know that it needs to call this method when it initializes its context and adds the returned object/value to the Spring context/Spring IoC Container.

```
@Configuration
public class ProjectConfig {

@Bean
Vehicle vehicle() {
    var veh = new Vehicle();
    veh.setName("Tesla");
    return veh;
}

}
```

Spring Context

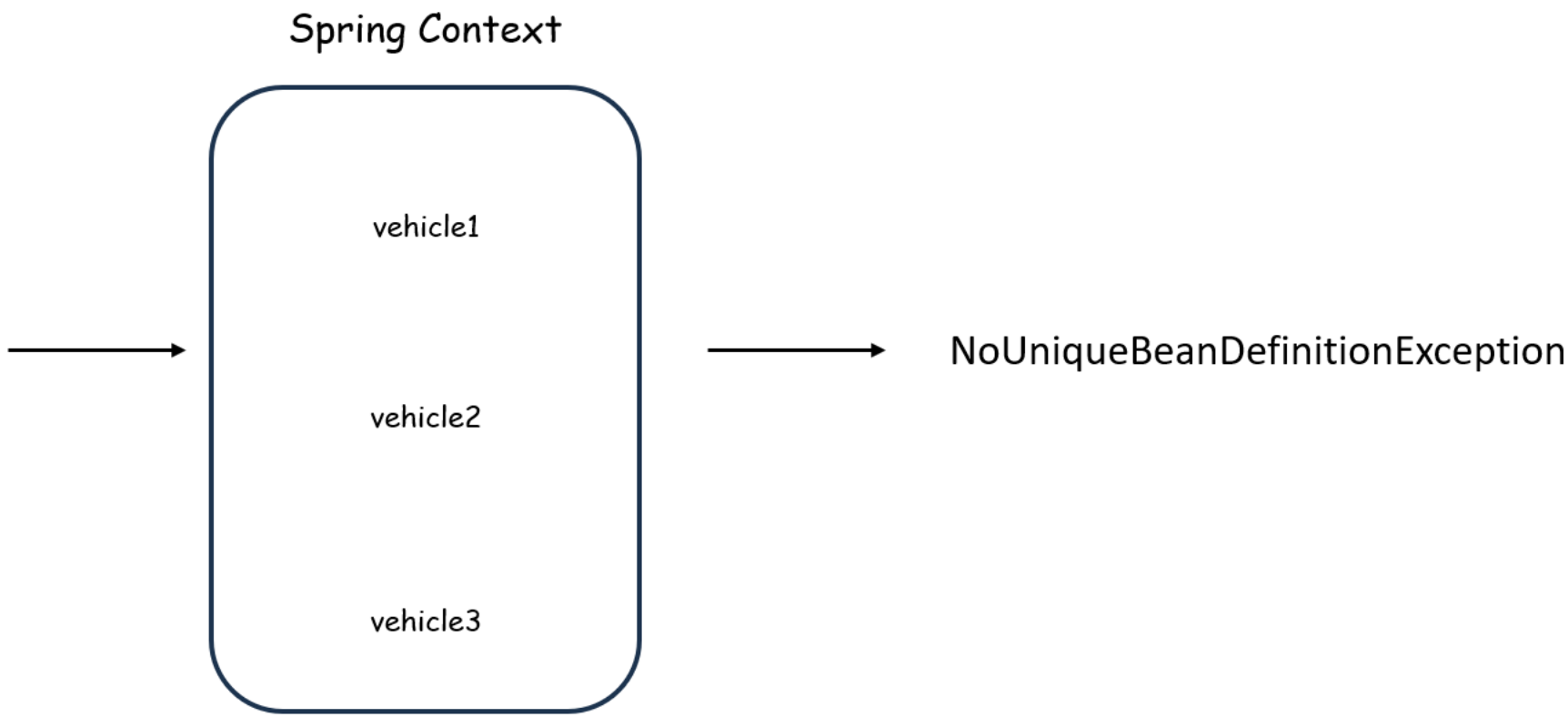


What is NoUniqueBeanDefinitionException ?

eazy bytes

When multiple beans of the same type are defined in the Spring context, and we attempt to retrieve a bean by type, Spring cannot determine which instance to provide. This ambiguity results in a NoUniqueBeanDefinitionException, as shown below.

```
@Bean
Vehicle vehicle1() {
    var veh = new Vehicle();
    veh.setName("Audi");
    return veh;
}

@Bean
Vehicle vehicle2() {
    var veh = new Vehicle();
    veh.setName("Honda");
    return veh;
}

@Bean
Vehicle vehicle3() {
    var veh = new Vehicle();
    veh.setName("Ferrari");
    return veh;
}
```

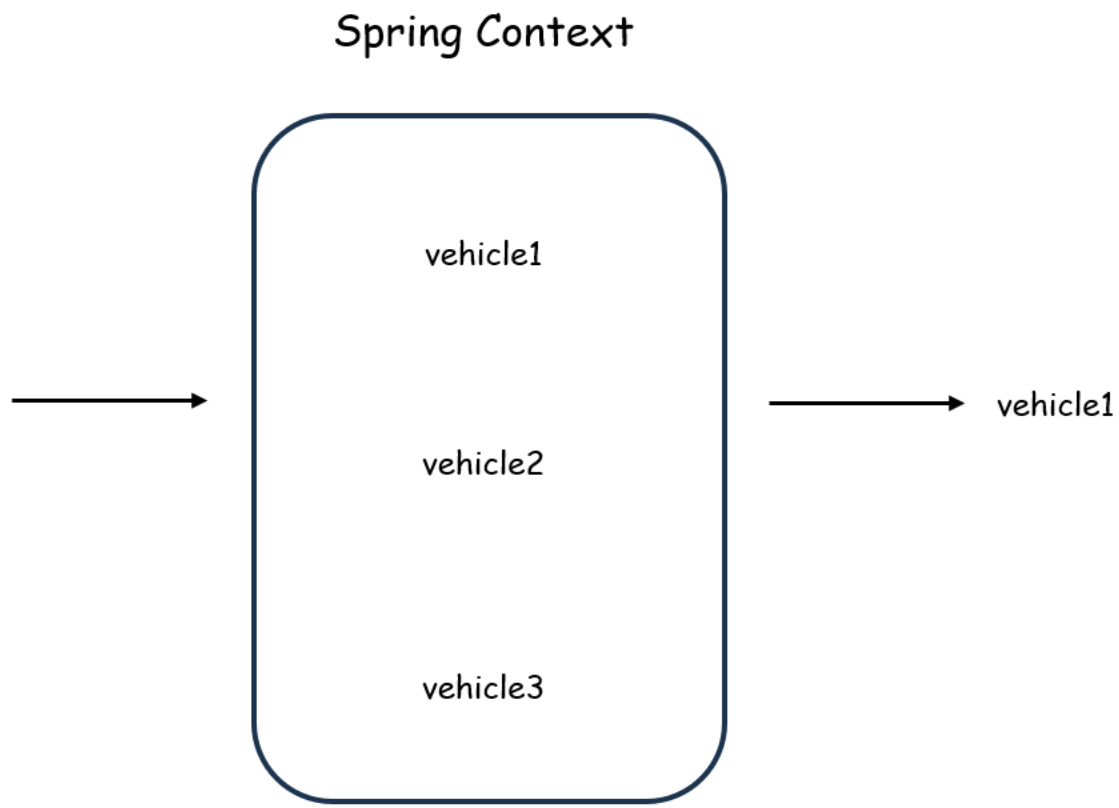


How to solve NoUniqueBeanDefinitionException ?

eazy bytes

To avoid NoUniqueBeanDefinitionException in these kind of scenarios, we can fetch the bean from the context by mentioning it's name like shown below,

```
@Bean
Vehicle vehicle1() {
    var veh = new Vehicle();
    veh.setName("Audi");
    return veh;
}

@Bean
Vehicle vehicle2() {
    var veh = new Vehicle();
    veh.setName("Honda");
    return veh;
}

@Bean
Vehicle vehicle3() {
    var veh = new Vehicle();
    veh.setName("Ferrari");
    return veh;
}
```

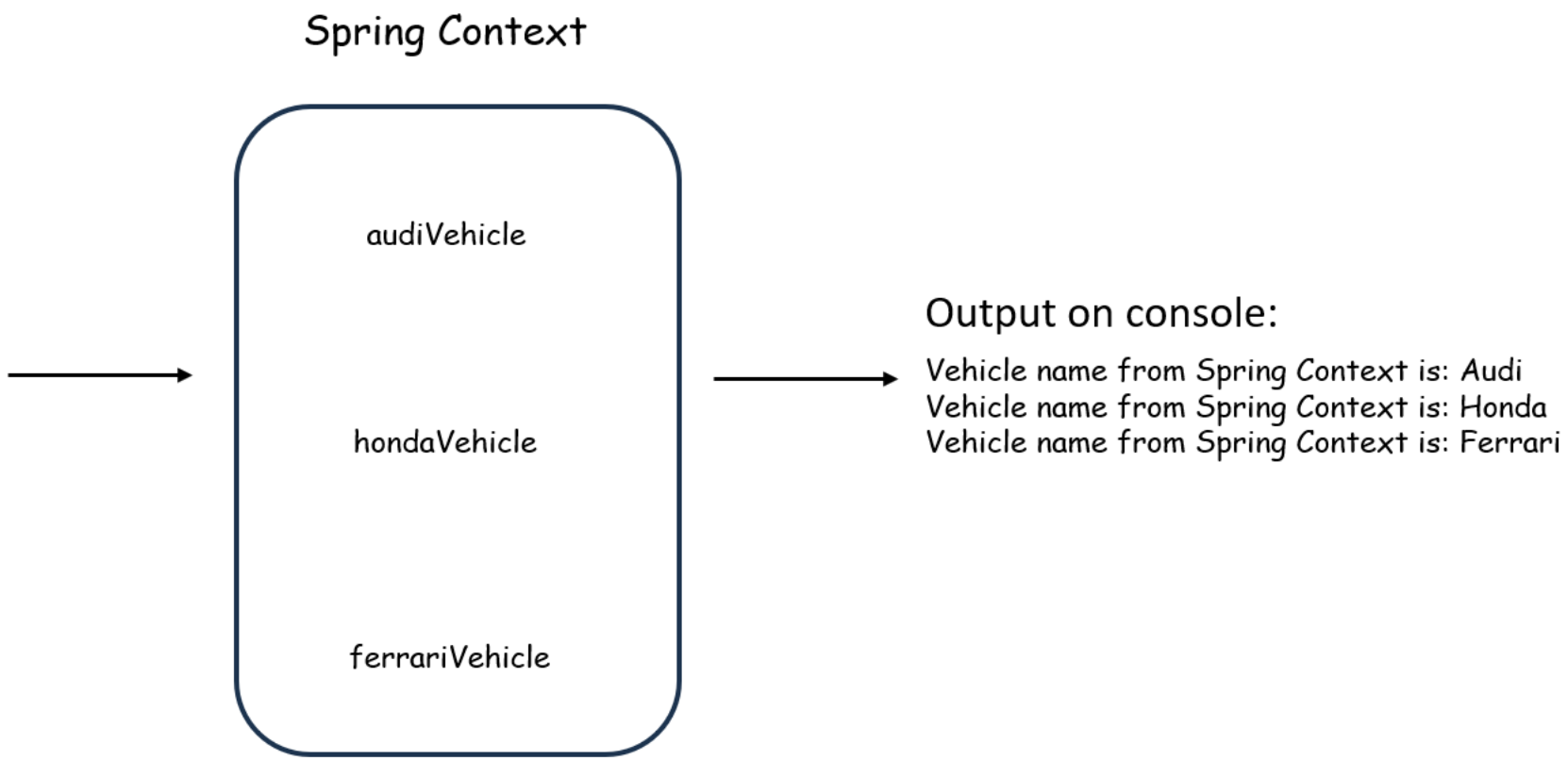


Different ways to name a Bean

eazy bytes

By default, Spring will consider the method name as the bean name. But if we have a custom requirement to define a separate bean name, then we can use any of the below approach with the help of @Bean annotation,

```
@Bean(name="audiVehicle")
Vehicle vehicle1() {
  var veh = new Vehicle();
  veh.setName("Audi");
  return veh;
}

@Bean(value="hondaVehicle")
Vehicle vehicle2() {
  var veh = new Vehicle();
  veh.setName("Honda");
  return veh;
}

@Bean("ferrariVehicle")
Vehicle vehicle3() {
  var veh = new Vehicle();
  veh.setName("Ferrari");
  return veh;
}
```



Different ways to name a Bean

eazy bytes

Giving Multiple Names to a Bean (Bean Aliasing)

Sometimes, you may want a single bean to be known by multiple names. This is called bean aliasing. You can define aliases by passing multiple names in the @Bean annotation using a String array. Now the same bean can be accessed by any of these names — audiVehicle, audi, myFavVehicle.

```
@Bean({”audiVehicle", ”audi", ”myFavVehicle"})
Vehicle vehicle() {
  var veh = new Vehicle();
  veh.setName("Audi");
  return veh;
}
```

Adding a Description to a Bean

Sometimes, it’s useful to add a short text description for a bean — especially if it’s exposed for monitoring or documentation. You can use the @Description annotation for that. The @Description text helps describe what the bean does — for clarity in tools or during debugging.

```
@Bean
@Description(”Represents a Vehicle of type Audi")
Vehicle vehicle() {
  var veh = new Vehicle();
  veh.setName("Audi");
  return veh;
}
```

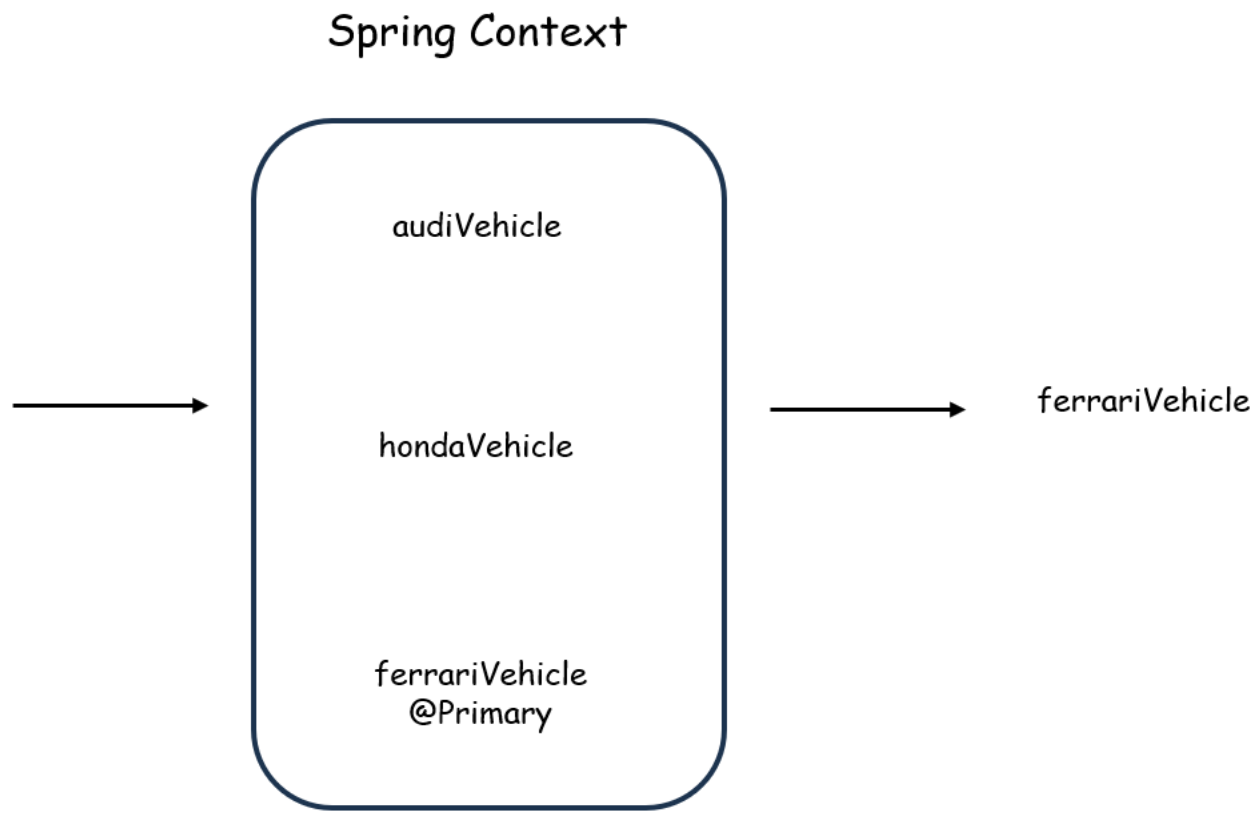
@Primary Annotation

eazy bytes

When multiple beans of the same type exist in the Spring context, you can designate one as the default by using the @Primary annotation.

The primary bean is the one Spring will automatically select when multiple candidates are available, and no specific bean name is provided. In other words, @Primary helps resolve ambiguity by instructing Spring which bean to prioritize when multiple options exist.

```
@Bean(name="audiVehicle")
Vehicle vehicle1() {
    var veh = new Vehicle();
    veh.setName("Audi");
    return veh;
}

@Bean(value="hondaVehicle")
Vehicle vehicle2() {
    var veh = new Vehicle();
    veh.setName("Honda");
    return veh;
}

@Primary
@Bean("ferrariVehicle")
Vehicle vehicle3() {
    var veh = new Vehicle();
    veh.setName("Ferrari");
    return veh;
}
```



Using the @Import Annotation

eazy bytes

@Import allows one @Configuration class to include bean definitions from another configuration class.

It helps organize and reuse configurations easily. ConfigB automatically imports all beans from ConfigA.

```
@Configuration
public class ConfigA {

    @Bean
    public A a() {
        return new A();
    }

}

@Configuration
@Import(ConfigA.class)
public class ConfigB {

    @Bean
    public B b() {
        return new B();
    }

}
```

It is also possible to import multiple config classes at a time.
Ex: `@Import({ConfigA.class, ConfigB.class})`

Using It in Application

- Simplifies application context creation
- No need to list multiple configuration classes manually
- Promotes modular and reusable configuration design

```
public static void main(String[] args) {
    var ctx = new
AnnotationConfigApplicationContext(ConfigB.class);

    // Both A and B beans are available
    A a = ctx.getBean(A.class);
    B b = ctx.getBean(B.class);
}
```

@Component Annotation

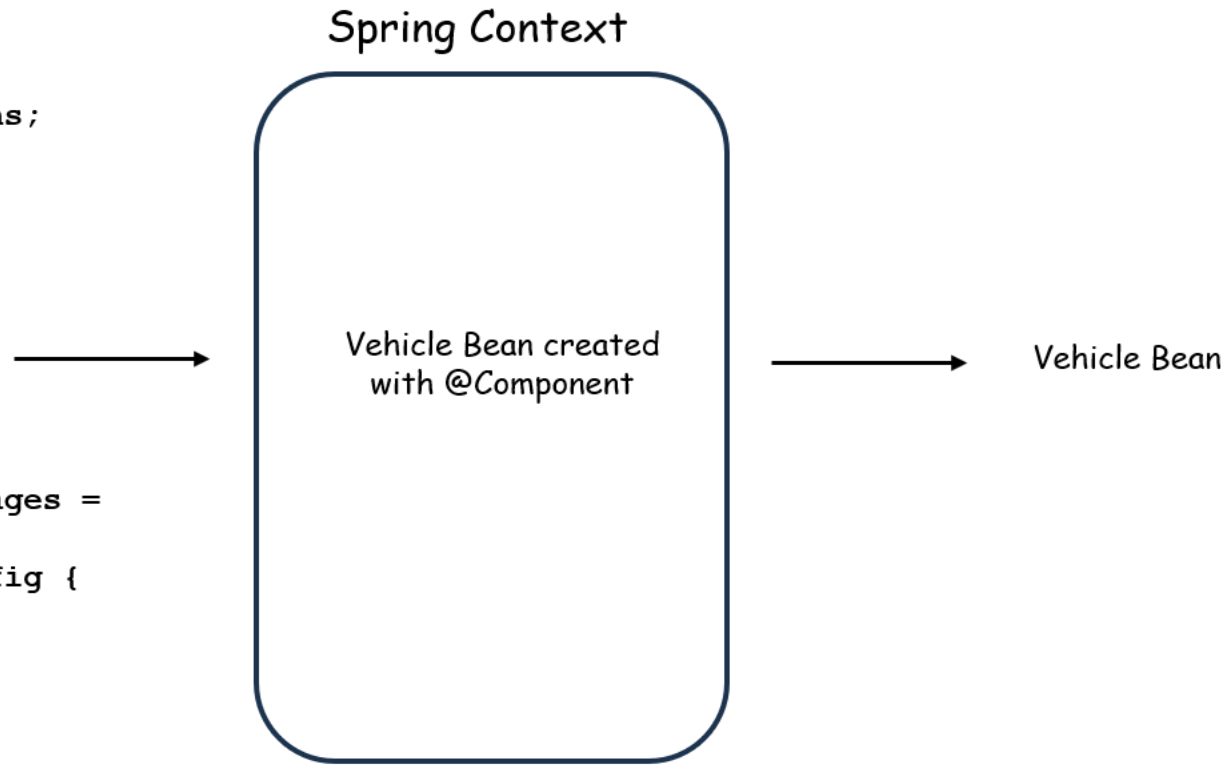
eazy bytes

@Component is one of the most commonly used stereotype annotations in Spring. It simplifies the process of creating and registering a bean in the Spring context with minimal code, compared to the @Bean method. By applying @Component to a class, Spring automatically detects and manages it as a Spring Bean.

To ensure Spring scans and registers these annotated classes, you can use the @ComponentScan annotation over a configuration class.

```
package com.example.beans;

@Component
public class Vehicle {
    // Other code
}
```

```
@Configuration
@ComponentScan(basePackages =
"com.example.beans")
public class ProjectConfig {
    // Other code
}
```



@PostConstruct Annotation

eazy bytes

@PostConstruct is a Spring annotation used to run a method automatically after a Spring bean is created and its dependencies are injected.

When is @PostConstruct used?

- To Perform setup logic when a bean is created.
- To Load data, set default values, or execute startup logic.
- To Initialize resources after dependency injection.

When Does @PostConstruct Run?

- It runs after the constructor and after all dependencies are injected.
- It runs only once in the bean's lifecycle.

Example

```
@Component
public class Vehicle {

    private String name;

    @PostConstruct
    public void initialize() {
        this.name = "Honda";
    }
}
```

- Spring creates Vehicle bean when the application starts.
- Before using the bean, it calls the initialize() method automatically.
- This is useful for setting up initial data or configurations.

@PostConstruct Annotation

eazy bytes

Alternative Approaches

If you prefer to avoid annotations like @PostConstruct, you can achieve similar behavior using:

- The InitializingBean interface, by overriding afterPropertiesSet()
- A custom init method defined in @Bean(initMethod="...")

Examples

```
@Bean(initMethod = "initialize")
public Vehicle vehicle() {
    return new Vehicle();
}

@Component
public class Vehicle implements InitializingBean {

    private String name;

    @Override
    public void afterPropertiesSet() throws Exception {
        this.name = "Honda";
        System.out.println("Vehicle initialized using InitializingBean: " + name);
    }
}
```

Use @PostConstruct for modern Spring apps, as it’s cleaner and framework-agnostic. Use InitializingBean only when you prefer interface-based initialization (e.g., for older codebases or when annotations aren’t suitable).

@PreDestroy Annotation

eazy bytes

@PreDestroy is a annotation used to run a method automatically before a Spring bean is destroyed — that is, right before the Spring container shuts down or removes the bean from its context.

When is @PreDestroy Used?

- To perform cleanup logic before a bean is destroyed.
- To close resources such as database connections, file handles, or threads.
- To release memory or stop background tasks.
- To log or notify that the bean is being shut down.

Example

```
@Component
public class Vehicle {

    @PreDestroy
    public void destroy() {
        System.out.println( "Vehicle Destroyed");
    }

}
```

- When the Spring context is about to close/shutdown, Spring automatically calls the destroy() method.
- This ensures any open resources are safely released before the application terminates.

When Does @PreDestroy Run?

- It runs just before the Spring container destroys the bean.
- It is called after the application context is closed.
- It runs only once in the bean's lifecycle.

@PreDestroy Annotation

eazy bytes

Alternative Approaches

If you prefer to avoid annotations like @PreDestroy, you can achieve similar behavior using:

- The DisposableBean interface, by overriding destroy()
- A custom init method defined in @Bean(destroyMethod="...")

Examples

```
@Bean(destroyMethod = "cleanup")
public Vehicle vehicle() {
    return new Vehicle();
}

@Component
public class Vehicle implements DisposableBean {

    private String name = "Honda";

    @Override
    public void destroy() throws Exception {
        System.out.println("Cleaning up Vehicle using DisposableBean: " + name);
    }
}
```

Use @PreDestroy for modern Spring apps, as it's cleaner and framework-agnostic. Use DisposableBean only when you prefer interface-based cleanup (e.g., for older codebases or when annotations aren't suitable).

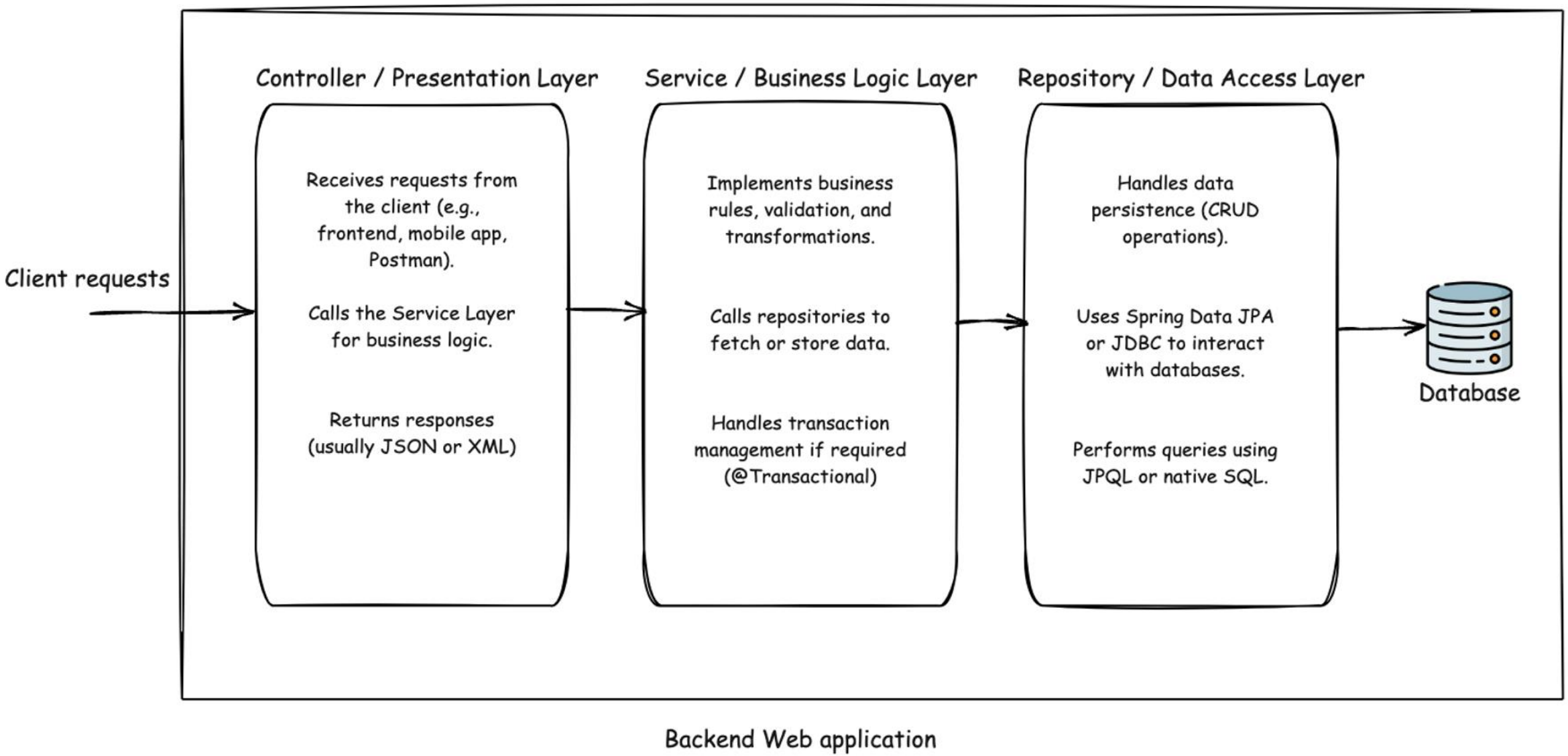
@Bean Vs @Component

eazy bytes

@Bean	@Component
One or more instances of the class can be added to the Spring Context	Only one instance of the class can be added to the Spring context
We can create an object instance of any type of class including present inside libraries like String etc.	We can create an object instance for application class only which are created by Dev team
Usually we need to write more code like separate methods to create beans instances	Bean instances can be created with very less code like using @Component on top of the class
Developer will have full control in creating and configuring the bean	Developer will not have any control in creating and configuring the bean
Spring framework creates the bean based on the instructions and values provided by Developer	Spring framework takes charge of creating the bean and post that Developer will have access to it

Layers in a Backend application

eazy bytes

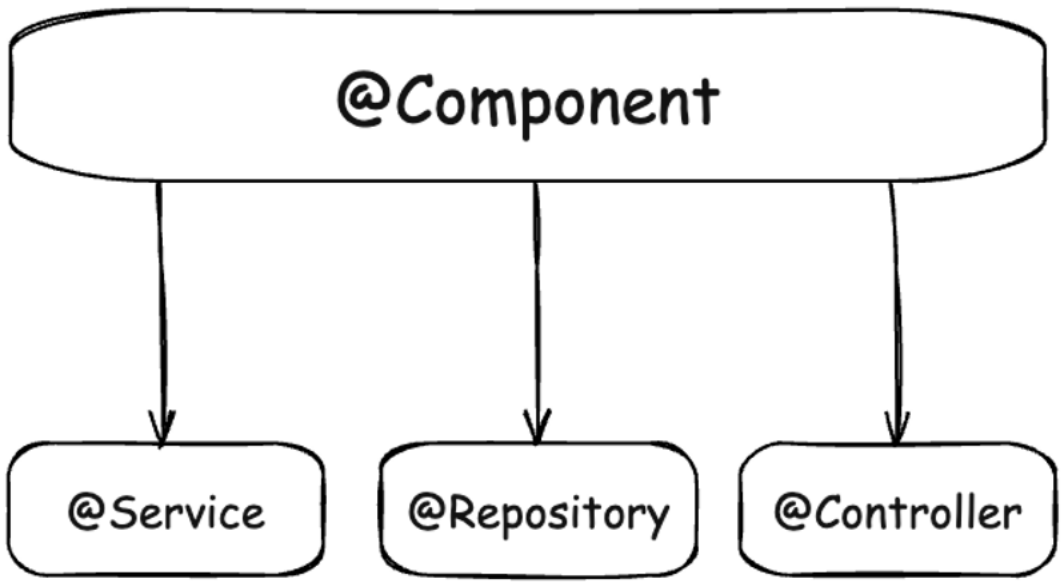


Spring Stereotype Annotations

eazy bytes

Spring provides stereotype annotations, which are special annotations that automatically register Spring beans in the application context. These annotations help simplify bean creation and management.

The key stereotype annotations in Spring include: @Component, @Service, @Repository, @Controller



@Component → Generic bean (used for any class).

@Service → Business logic layer.

@Repository → Data access layer

@Controller → Handles incoming web requests in Spring MVC

@RestController → Simplified @Controller, returns JSON responses. Used to build REST APIs

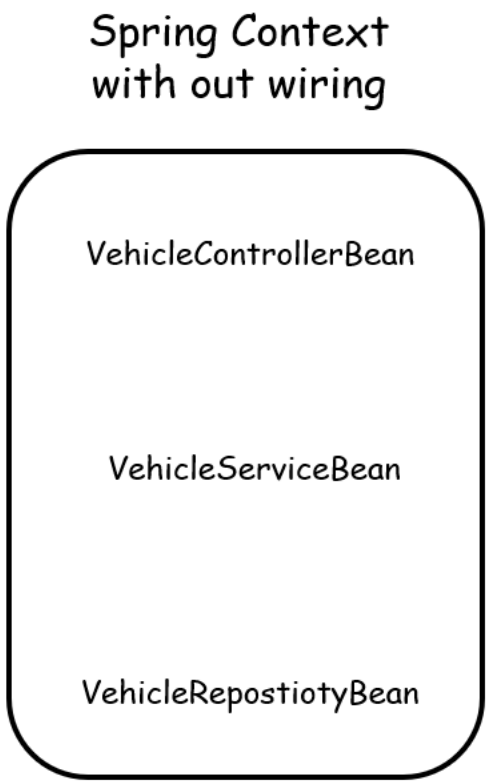
Use @ComponentScan to enable automatic scanning of these annotations.

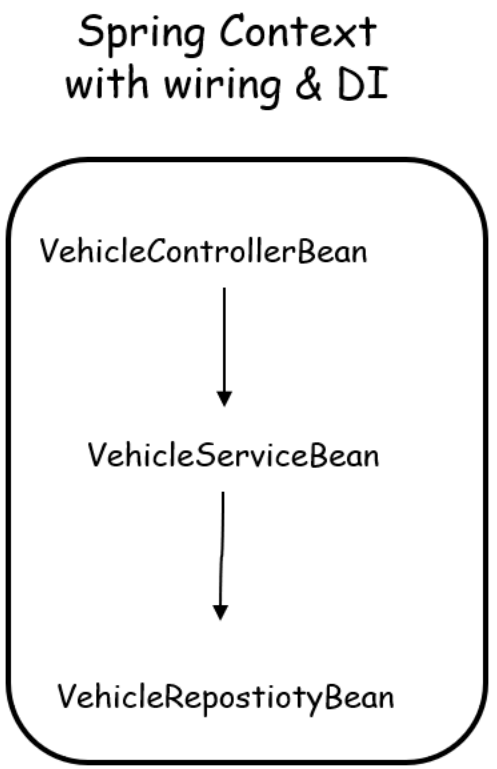
Introduction to Beans Wiring inside Spring

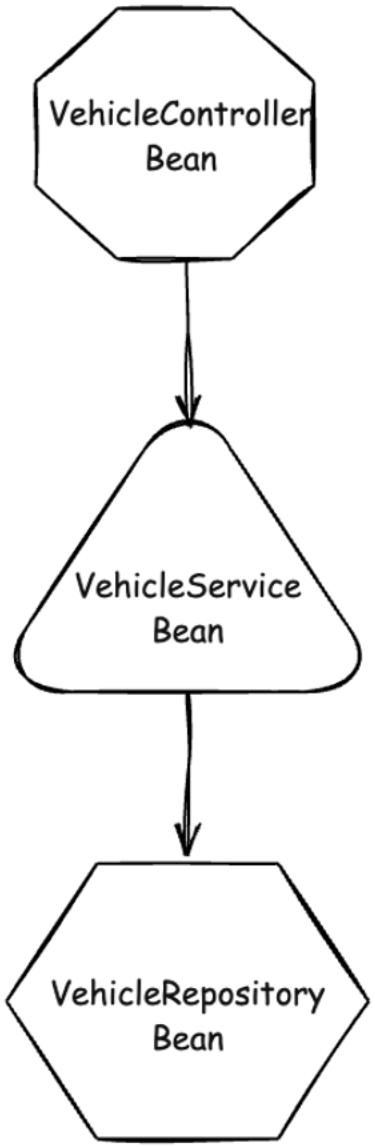
eazy bytes

In Java web applications, objects often delegate tasks to other objects, creating dependencies between them. Similarly, in Spring, when we define multiple beans, some beans may rely on others to function correctly. Bean wiring is the process of connecting these dependent beans within the Spring IoC container.

Manually managing these dependencies can become complex, so Spring provides automatic wiring (Autowiring) to simplify the process.







No wiring scenario inside spring

eazy bytes

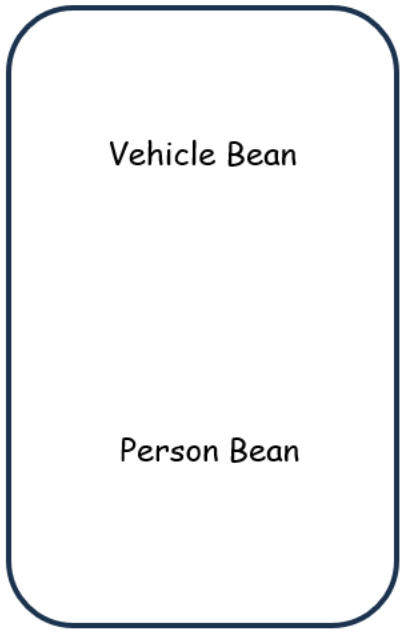
Consider a scenario where we have two java classes Person and Vehicle. The Person class has a dependency on the Vehicle. Based on the below code,

we are only creating the beans inside the Spring Context and no wiring will be done. Due to this both this beans present inside the Spring context with out knowing about each other.

```
public class Person {

  private String name;
  private Vehicle vehicle;

}
```

```
public class Vehicle {

  private String name;

}
```

```
@Bean
public Vehicle vehicle() {
    Vehicle vehicle = new Vehicle();
    vehicle.setName("Toyota");
    return vehicle;
}


@Bean
public Person person() {
    Person person = new Person();
    person.setName("Lucy");
    return person;
}
```



Vehicle doesn't belong to any person. The Person and Vehicle beans present in context but no relation established

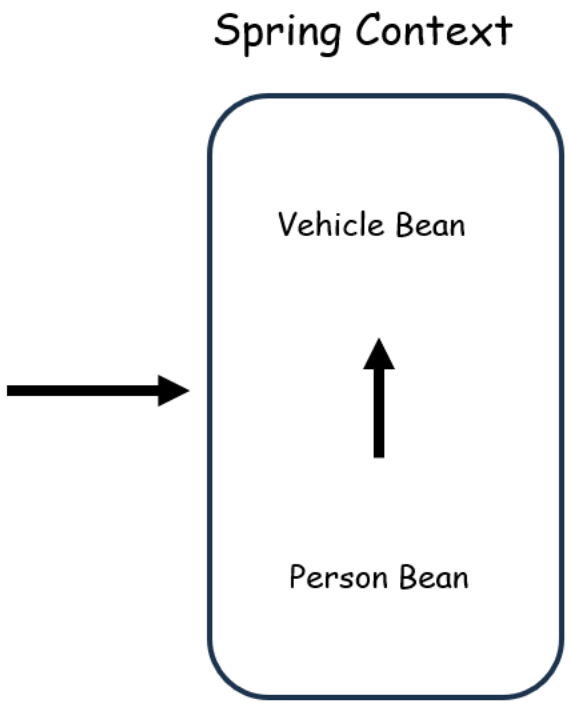
Wiring Beans using method call

eazy bytes

In the code below, we establish a relationship between the Person and Vehicle beans by calling the vehicle() bean method from within the person() bean method. This ensures that the Person bean owns a Vehicle within the Spring context.

Spring automatically ensures that only one instance of the Vehicle bean is created and manages its lifecycle efficiently. Since the Person bean depends on the Vehicle bean, Spring will always instantiate the Vehicle bean first before creating the Person bean.

```
@Bean
public Vehicle vehicle() {
    Vehicle vehicle = new Vehicle();
    vehicle.setName("Toyota");
    return vehicle;
}


@Bean
public Person person() {
    Person person = new Person();
    person.setName("Lucy");
    person.setVehicle(vehicle());
    return person;
}
```




```
var context = new
AnnotationConfigApplicationContext(ProjectConfig.c
lass);
Person person = context.getBean(Person.class);
Vehicle vehicle = context.getBean(Vehicle.class);

System.out.println("Person name from Spring
Context is: " + person.getName());
System.out.println("Vehicle name from Spring
Context is: " + vehicle.getName());
System.out.println("Vehicle that Person own is: "
+ person.getVehicle());
```

Output on console:

```
Vehicle bean created by Spring
Person bean created by Spring
Person name from Spring Context is: Lucy
Vehicle name from Spring Context is: Toyota
Vehicle that Person own is: Vehicle name is - Toyota
```

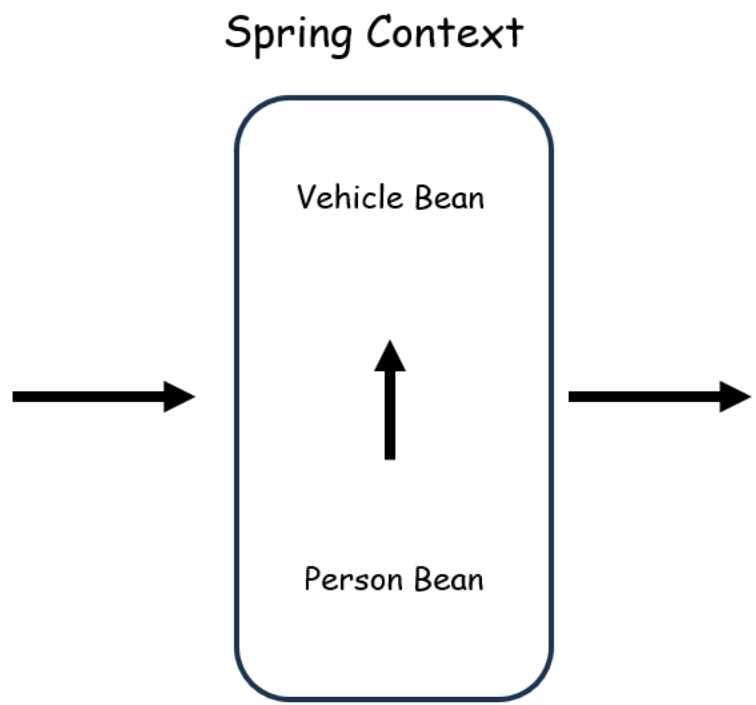
Wiring Beans using method parameters

eazy bytes

In the code below, we establish a relationship between the Person and Vehicle beans by passing the Vehicle bean as a method parameter to the person() bean method. This ensures that within the Spring context, the Person bean owns a Vehicle.

Spring injects the Vehicle bean into the Person bean using Dependency Injection, ensuring seamless management of dependencies. Additionally, Spring guarantees that only one instance of the Vehicle bean is created and that it is instantiated first, since the Person bean depends on it.

```
@Bean
public Vehicle vehicle() {
    Vehicle vehicle = new Vehicle();
    vehicle.setName("Toyota");
    return vehicle;
}


@Bean
public Person person(Vehicle veh) {
    Person person = new Person();
    person.setName("Lucy");
    person.setVehicle(veh);
    return person;
}
```




```
var context = new
AnnotationConfigApplicationContext(ProjectConfig.c
lass);

Person person = context.getBean(Person.class);
Vehicle vehicle = context.getBean(Vehicle.class);

System.out.println("Person name from Spring
Context is: " + person.getName());
System.out.println("Vehicle name from Spring
Context is: " + vehicle.getName());
System.out.println("Vehicle that Person own is: "
+ person.getVehicle());
```

Output on console:

```
Vehicle bean created by Spring
Person bean created by Spring
Person name from Spring Context is: Lucy
Vehicle name from Spring Context is: Toyota
Vehicle that Person own is: Vehicle name is - Toyota
```

What is Autowiring in Spring?

eazy bytes

Autowiring is a feature in Spring that automatically injects dependencies.

- ✓ It eliminates the need for manual object creation using new keyword.
- ✓ Spring finds the right dependency and injects it automatically.
- ✓ Makes applications loosely coupled and easier to maintain.

Why Use Autowiring?

- ✓ Reduces boilerplate code – No need for manual new object creation.
- ✓ Improves code readability – Dependencies are automatically injected.
- ✓ Supports loose coupling – Easy to modify and test.
- ✓ Spring manages dependency resolution efficiently.

Ways to Autowire in Spring

Spring provides three main ways to perform autowiring:

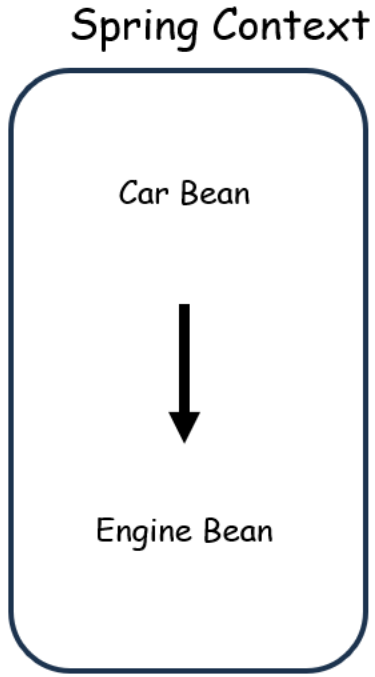
1. By Field Injection (@Autowired on field)
2. By Setter Injection (@Autowired on Setter Method)
3. By Constructor Injection (@Autowired on Constructor)

Autowiring using Field Injection (@Autowired on Field)

eazy bytes

Spring looks for a matching bean type and injects it automatically. Simple and commonly used method.

```
@Component
class Engine { }

@Component
class Car {

    @Autowired
    private Engine engine;

}
```



While @Autowired by type (field injection) is convenient, it comes with drawbacks as mentioned below,

1. Hard to unit test
    - You cannot easily create an object and set its dependencies yourself.
    - Because the field is private and has no setter, you must rely on Spring.
    - So in tests, it becomes difficult to mock/replace the dependency.

2. No way to make the field final

    If Spring injects via field, you cannot write:

    ```
    private final AccountService accountService;
    ```

    So, you cannot guarantee the dependency is always available. It may be accidentally changed later.

3. Hidden dependencies

    The class doesn't show what it needs from outside.

4. Can create NullPointerExceptions
    - If Spring fails to inject, the object can still be created, and you might get NullPointerException at runtime.
    - Constructor injection avoids this, because the app won't even start if dependencies are missing.

@Autowired(required = false) will help to avoid the NoSuchBeanDefinitionException if the bean is not available during Autowiring process.

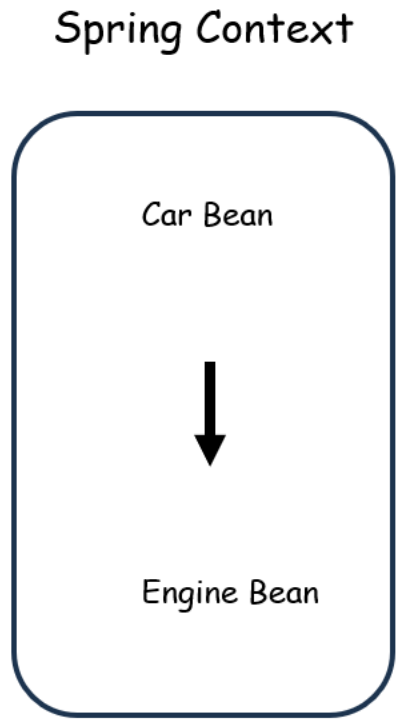
Autowiring using Setter Injection (@Autowired on Setter)

eazy bytes

Spring injects the dependency using a setter method. Useful when the dependency is optional.

```
@Component
class Engine { }

@Component
class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

- ✓ Has same drawbacks as Field Injection
- ✓ Less commonly used compared to constructor injection.



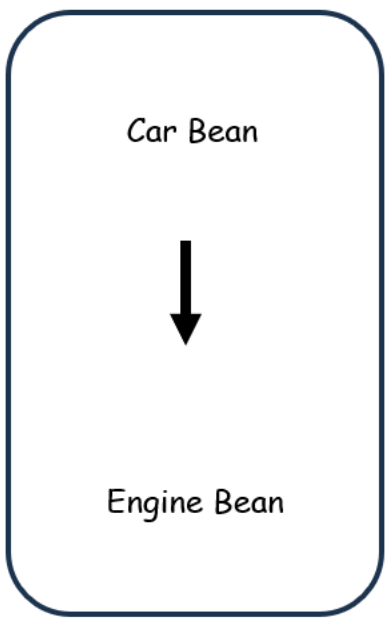
Autowiring by Constructor Injection (@Autowired on Constructor)

eazy bytes

Spring injects dependencies through the constructor. Recommended for mandatory dependencies.

```
@Component
class Engine { }

@Component
class Car {
    private final Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

- ✓ Preferred approach in modern Spring applications.
- ✓ Works well with immutability and unit testing.



Understanding & Avoiding Circular dependencies

eazy bytes

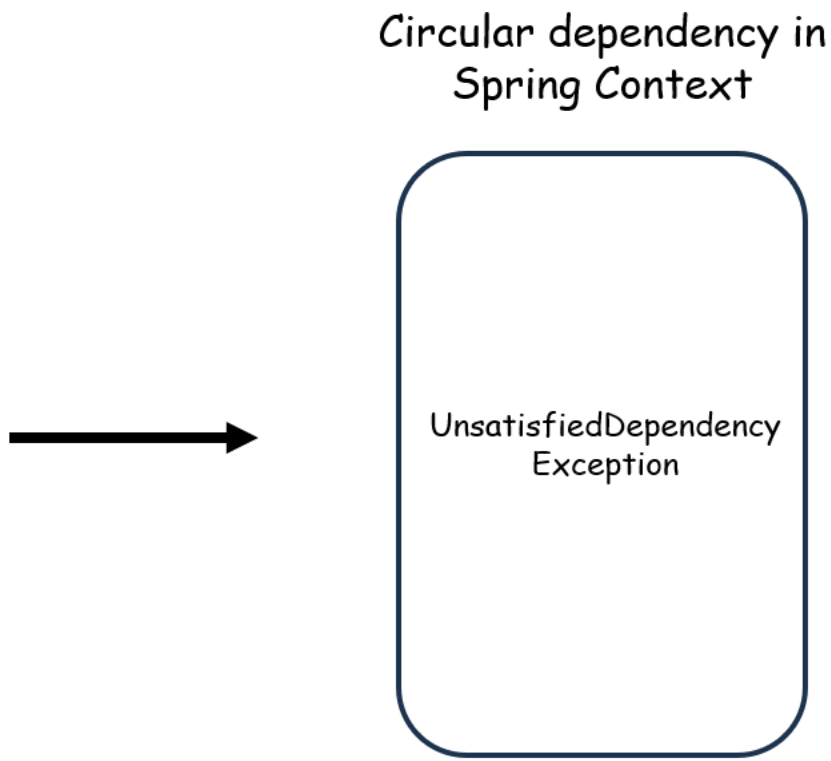
A circular dependency occurs when two or more beans depend on each other directly or indirectly. Spring cannot resolve dependencies and throws an UnsatisfiedDependencyException. This usually happens in constructor-based injection.

Example of Circular Dependency

Scenario:

- Class Car depends on Class Engine.
- Class Engine depends on Class Car.
- Spring fails to resolve this dependency loop.

```
@Component
class Car {
    private final Engine eng;

    @Autowired
    public Car(Engine eng) {
        this.eng = eng;
    }
}

@Component
class Engine {
    private final Car car;

    @Autowired
    public Engine(Car car) {
        this.car = car;
    }
}
```



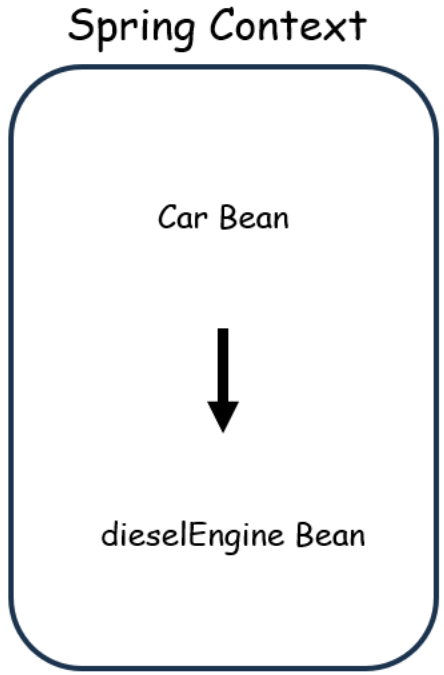
@Primary Annotation

eazy bytes

When multiple beans match for autowiring by type, @Primary gives one bean higher preference so it’s chosen automatically as the default.

```
@Component("petrolEngine")
class PetrolEngine implements Engine { }

@Component("dieselEngine")
@Primary
class DieselEngine implements Engine { }

@Component
class Car {

    @Autowired
    private Engine engine;

}
```

- ✓ @Primary is not limited to field injection; it works with constructor, setter, and method injection.
- ✓ Best practice: Use @Primary with constructor injection for required dependencies.



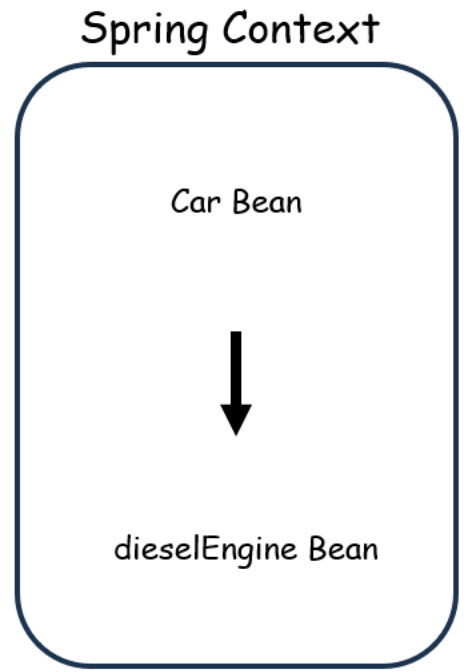
@Qualifier Annotation

eazy bytes

If multiple beans of the same type exist, we use @Qualifier to specify the exact bean. Helps when multiple beans of the same type exist.
Avoids ambiguity in autowiring.

```
@Component("petrolEngine")
class PetrolEngine implements Engine { }

@Component("dieselEngine")
class DieselEngine implements Engine { }

@Component
class Car {
    @Autowired
    @Qualifier("dieselEngine")
    private Engine engine;
}
```

- ✓ @Qualifier is not limited to field injection; it works with constructor, setter, and method injection.
- ✓ Best practice: Use @Qualifier with constructor injection for required dependencies.



@Qualifier Vs @Primary

eazy bytes

If @Primary is used, you do not need to use @Qualifier explicitly. Spring will automatically select the @Primary bean when multiple beans of the same type exist. However, there are cases where @Qualifier is still needed, even when @Primary is defined

Annotation	When to Use?	How It Works
@Primary	When you have a default bean that should be used most of the time.	Spring automatically selects the @Primary bean if no specific qualifier is provided.
@Qualifier	When you need to explicitly specify which bean to use.	Overrides @Primary and allows selecting a specific bean when multiple exist.

@Qualifier Vs @Primary

eazy bytes

Imagine you walk into a coffee shop that serves different types of coffee

Example 1: @Primary (Default Choice)

The shop has multiple coffee types, but the barista always serves Espresso by default unless you ask for something specific.
Here, Espresso is marked as @Primary, so whenever a customer asks for coffee without specifying a type, they get an Espresso.

```
@Component
@Primary
public class Espresso implements Coffee {}

@Component
public class Cappuccino implements Coffee {}

@Component
public class CoffeeShop {

    @Autowired
    private final Coffee coffee;
}
```

@Qualifier Vs @Primary

eazy bytes

Example 2: @Qualifier (Explicit Selection)

Now, a customer wants a Cappuccino instead of the default Espresso. They specify their choice by saying: "I want a Cappuccino!"

Here, @Qualifier acts like the customer specifying their order, overriding the default choice.

```
@Component
@Primary
public class Espresso implements Coffee {}

@Component
public class Cappuccino implements Coffee {}

@Component
public class CoffeeShop {
    private final Coffee coffee;

    @Autowired
    public CoffeeShop(@Qualifier("cappuccino") Coffee coffee)
        this.coffee = coffee;
    }
}
```

Programmatic Bean Registration in Spring

eazy bytes

Instead of defining beans with @Bean methods or @Component, Spring 7 introduced programmatic bean registration using the BeanRegistrar interface. This gives developers fine-grained control to register beans at runtime, using if conditions, loops, or environment checks — all in Java code.

```
class MyBeanRegistrar implements BeanRegistrar {

    @Override
    public void register(BeanRegistry registry, Environment env) {

         // Register Engine bean
        registry.registerBean(Engine.class);

        // Register Vehicle bean (depends on Engine)
        registry.registerBean(Vehicle.class,
            spec -> spec.supplier(context ->
                new Vehicle(context.bean(Engine.class))));
    }
}

@Configuration
@Import(MyBeanRegistrar.class)
public class AppConfig {
}
```

Bean scopes in Spring

eazy bytes

Bean scope defines how Spring creates and manages bean instances. It answers two key questions:

- How many instances of a bean will Spring create?
- When will Spring create these instances?

Spring provides six bean scopes:

01 **Singleton (Default)**

02 **Prototype**

03 **Request** (inside web apps)

04 **Session** (inside web apps)

05 **Application** (inside web apps)

06 **Websocket** (inside web apps)

Singleton scope (Default)

eazy bytes

Singleton is the default scope of a bean in Spring. This means that whenever a bean is referenced or autowired within the application, the same instance is returned each time.

Unlike the Singleton Design Pattern, which ensures only one instance per application, Spring's Singleton scope maintains one instance per bean definition within the Spring context. If multiple beans of the same type are declared, Spring will manage a separate singleton instance for each unique bean definition, rather than enforcing a single instance across the entire application.

```
@Component
@Scope("singleton") // Optional
public class MyService {
    public MyService() {
        System.out.println("MyService
instance created!");
    }
}
```

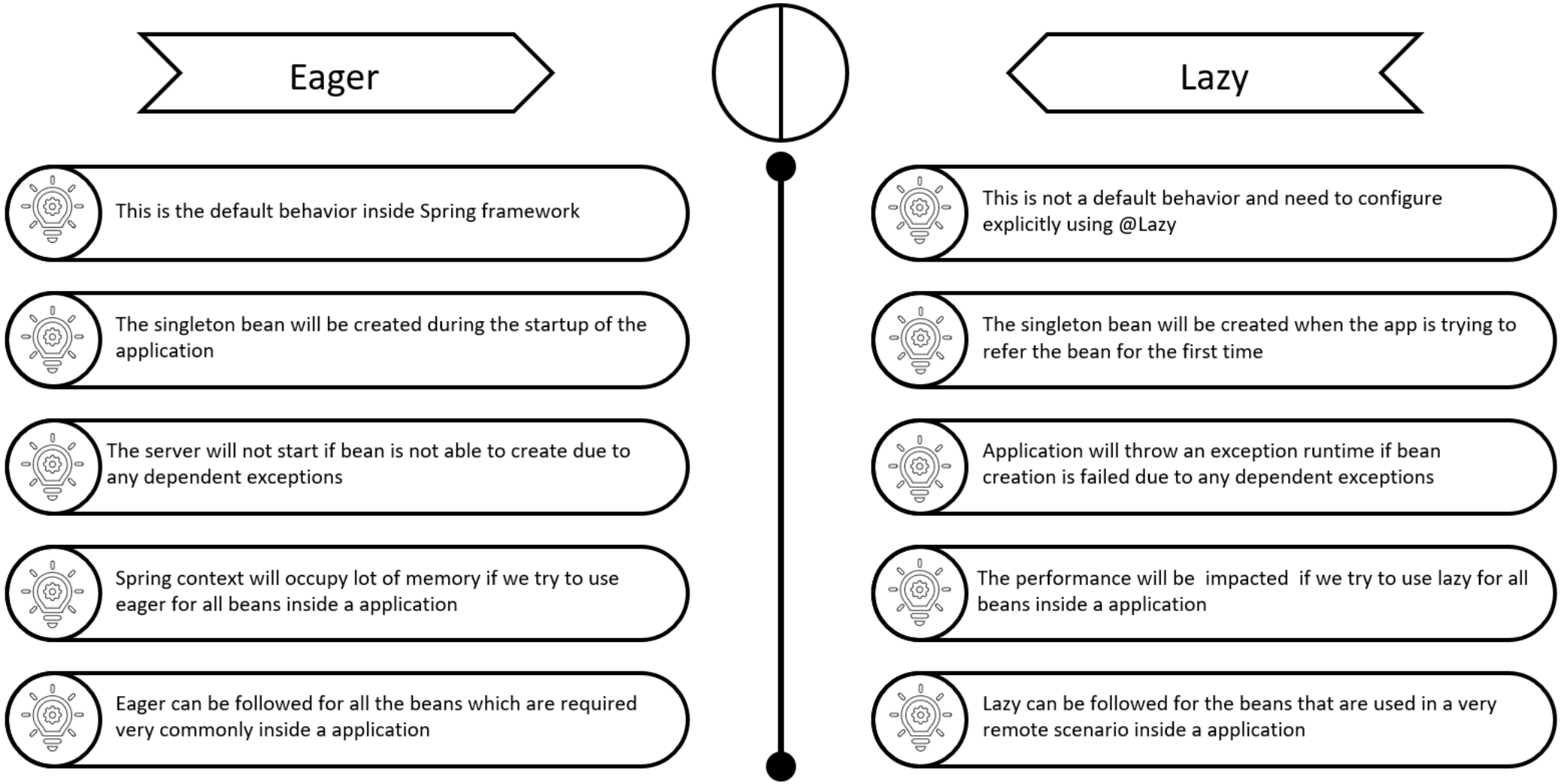
Use Singleton when you need a single shared instance across the application, such as a service managing a cache.

```
var context = new AnnotationConfigApplicationContext(AppConfig.class);
MyService service1 = context.getBean(MyService.class);
MyService service2 = context.getBean(MyService.class);

System.out.println(service1 == service2); // true (Both are the same instance)
```

Eager & Lazy instantiation

eazy bytes

By default Spring will create all the singleton beans eagerly during the startup of the application itself. This is called Eager instantiation. We can change the default behavior to initialize the singleton beans lazily only when the application is trying to refer to the bean. This approach is called Lazy instantiation.



Prototype scope

eazy bytes

With prototype scope, every time we request a reference of a bean, Spring will create a new object instance and provide the same.

Prototype scope is rarely used inside the applications and we can use this scope only in the scenarios where you need a new instance each time, such as an object representing user input.

```
@Component
@Scope("prototype")
public class UserSession {
    private String sessionId;

    public UserSession() {
        this.sessionId = UUID.randomUUID().toString(); // Generate a unique ID
    }

    public String getSessionId() {
        return sessionId;
    }
}
```

```
UserSession user1 = context.getBean(UserSession.class);
UserSession user2 = context.getBean(UserSession.class);

System.out.println(user1 == user2); // false (Different instances)
```

Singleton Vs Prototype

eazy bytes

Singleton scope

This is the default scope inside Spring framework

The same object instance will be returned every time we refer a bean inside the code

We can configure to create the beans during the start up or when the first time referred

Immutable objects can be ideal for Singleton scope

Most commonly used scope

Prototype scope

Need to explicitly configure using @Scope(BeanDefinition.SCOPE_PROTOTYPE)

New object instance will be returned every time we refer a bean inside the code

Spring always creates the new object when we try to refer the bean. No eager instantiation is possible

Mutable objects can be ideal for prototype scope

Very rarely used scope

Spring Boot : The HERO of Spring framework

eazy bytes

1. Spring Boot was introduced in April 2014 to reduce some of the burdens while developing a Java web application using Spring.

2. With SpringBoot, we can create Web Apps skeletons with in seconds or at least in 1-2 mins ☺. It helps eliminating all the configurations we need to do.

3. Spring Boot is now one of the most appreciated projects in the Spring ecosystem. It helps us to create Spring apps more efficiently and focus on the business code.

4. SpringBoot is a mandatory skill now due to the latest trends like Full Stack Development, Microservices, Serverless, Containers, Docker etc.

Spring Boot applications can be created using **Spring Initializr** which is a web-based tool that helps generate a Spring Boot project with required dependencies.

Steps:

1) Visit https://start.spring.io/
2) Choose build tool for the project either as Maven or Gradle
3) Choose Language: `Java`
4) Select the desired Spring Boot Version
5) Fill Project Metadata details
6) Add required Dependencies (e.g., Spring Web, Spring Data JPA, etc.)
7) Click Generate and download the project
8) Extract the ZIP file and open it in IntelliJ IDEA

What are Spring Boot Starters ?

eazy bytes

Spring Boot Starters are ready-made dependency bundles that bring in all required libraries, configurations, and auto-settings for specific features.

Why Use Starters?

- They reduce manual setup of dependencies.
- They avoid version conflicts between libraries.
- They help you focus on coding instead of configuration

How Do They Work?

Instead of adding multiple dependencies manually, you just add one Starter dependency in your pom.xml (Maven) or build.gradle (Gradle).

What are Spring Boot Starters ?

eazy bytes

Example Without Starter (Manual Approach)

```
<dependencies>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-core</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-web</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-mvc</artifactId>
   </dependency>
   <dependency>
      <groupId>org.apache.tomcat</groupId>
      <artifactId>tomcat-embed-core</artifactId>
   </dependency>
</dependencies>
```

Example With Starter

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

With out starter projects, it will be too much work and can cause version conflicts!

With starter projects, all the necessary libraries (Spring Web, MVC, Tomcat, etc.) will be automatically pulled

Developing Web Apps using Spring, Spring Boot

eazy bytes

Spring and Spring Boot allow developers to build powerful web applications. These applications can be classified into two major types. Spring MVC is the key differentiator between these two approaches.

Spring MVC-Based Web Apps

- Web Apps which holds UI elements like HTML, CSS, JS and backend logic
- Here the App is responsible to fully prepare the view along with data in response to a client request
- Spring MVC Web Apps serves HTML pages (for humans) via browser

REST API-Based Web Applications

- Web Apps which holds only backend logic. These Apps send data like JSON to separate mobile apps, backend apps, UI Apps built based on Angular, React etc.
- Here the App is responsible to only process the request and respond with only data ignoring view.
- Spring REST API Apps serves JSON data (for other systems)

Feature	Spring MVC Web App	Rest API-Based App
Controller Type	@Controller	@RestController = @Controller + @ResponseBody
Response Type	HTML / View	JSON / XML
Rendering	Server-side	Client-side
View Engine	JTE / Thymeleaf / JSP	Separate Frontend(React, Angular)
Coupling	Tightly coupled	Loosely coupled
Client	Browser	Mobile Apps, UI Apps, Other Services etc.
Typical Use	UI-based monolithic apps	Backend APIs or microservices

Understanding @SpringBootApplication

eazy bytes

@SpringBootApplication is a shortcut annotation that combines three annotations:

1. @SpringBootConfiguration → Marks this class as a configuration class.
2. @EnableAutoConfiguration → Automatically configures Spring Boot based on the dependencies.
3. @ComponentScan → Scans and registers Spring components (like @Component, @Service, etc.).

Why Use @SpringBootApplication ?

- Saves time – No need for long XML configuration.
- Auto-configures – Detects dependencies and configures them automatically.
- Simplifies Spring Boot setup – Everything works with minimal code.

Understanding @SpringBootApplication

eazy bytes

The “main” Method

The main method serves as the application’s entry point, following standard Java conventions. It calls SpringApplication.run(), which bootstraps the Spring Boot application—starting Spring itself and the embedded Tomcat server. We pass BackendApplication.class to identify the main Spring component and include args to forward any command-line arguments.

```
@SpringBootApplication
public class BackendApplication {

    static void main(String[] args) {
            SpringApplication.run(BackendApplication.class, args);
    }

}
```

Running the SpringBoot app

- Using IntelliJ IDEA
- Maven (mvn spring-boot:run)
- Gradle (gradle bootRun)

Spring Boot Principles

eazy bytes

Spring Boot primarily follows the principle of **“Convention over Configuration”.** This means, Spring Boot gives you sensible defaults so you don’t need to manually configure everything.

Instead of writing long XML files or heavy setup, Spring Boot auto-configures components based on:

- What dependencies are present
- Common use-case defaults

So developers can focus on business logic, not boilerplate.

Supporting Principles

1) Opinionated Defaults

Spring Boot provides pre-configured defaults (web server, JSON mapper, logging, DB) so new apps can start quickly.

`Example: If spring-boot-starter-web is on classpath → auto-configures Tomcat, Spring MVC, Jackson, etc.`

2) Auto-Configuration

Spring Boot automatically configures beans based on classpath and environment without needing explicit configuration.

Example:

- `If H2 dependency is present → auto-configures H2 database`
- `If spring-data-jpa is present → configures EntityManager`

Spring Boot Auto-Configuration

eazy bytes

Auto-Configuration in Spring Boot automatically configures your application based on the dependencies you add. In simple words, you don't have to manually set up configurations. Spring Boot does it for you!

How Does Auto-Configuration Work?

1. You add a Spring Boot Starter dependency (e.g., spring-boot-starter-web).
2. Spring Boot automatically detects what you need and configures it for you.
3. You can override the default settings if needed.

For example, If spring-boot-starter-web is present, Spring Boot assumes:

- The app is a web application.
- It starts an embedded Tomcat server on port 8080.
- It creates a default DispatcherServlet for handling requests.

Few of the default values assumed during Auto Configuration:

server.port=8080
server.servlet.context-path=/
logging.level.root=INFO

Spring Boot Auto-Configuration

eazy bytes

If you were using **traditional Spring**, you would have to manually configure **a web server**, **database connections**, and other settings.

Auto-Configuration is enabled by default in Spring Boot. It is controlled by the @SpringBootApplication annotation. This annotation includes @EnableAutoConfiguration, which tells Spring Boot to configure everything automatically.

Mentioning debug=true will print the Autoconfiguration report on the console.

If you want to disable auto-configuration for specific components, you can use:

```
@SpringBootApplication(exclude = SslAutoConfiguration.class)
```

Building REST Services with Spring Boot

eazy bytes

What is a REST API?

REST (Representational State Transfer) is a way for applications to communicate over HTTP. A REST API:

- Allows clients (e.g., web, mobile apps) to interact with your application.
- Returns JSON or XML responses.

REST APIs use HTTP methods for different actions:

HTTP Method	Action
GET	Fetch data
POST	Create new data
PUT	Full update (replaces the entire resource)
PATCH	Partial update (modifies only specified fields)
DELETE	Remove data

Building REST Services with Spring Boot

eazy bytes

In Spring Boot, a REST API is created using @RestController. The @RestController annotation in Spring Boot is a specialized version of @Controller that simplifies the development of RESTful web services.

@RestController = @Controller + @ResponseBody

This means that:

1. The class is treated as a controller that handles HTTP requests.
2. Each method automatically serializes the return value into JSON (or XML) and sends it in the HTTP response.

Example:

```
@RestController
public class HelloController {

    @GetMapping(path = {"/", "/hello"})
    public String hello() {
        return "Hi there, this is Job Portal REST API";
    }
}
```

Use nouns, not verbs while defining REST API paths,

Good Examples:

```
GET api/users          → Get all users
GET api/users/{id}     → Get a specific user
POST api/users         → Create a new user
PUT api/users/{id}     → Update user details
DELETE api/users/{id} → Delete a user
```

Avoid bad practices

```
GET api/getUsers
POST api/createUser
DELETE api/removeUser
```

Return proper HTTP status codes. Avoid returning 200 OK for everything

HTTP Method	Purpose	Status code
GET	Fetch data	200 OK
POST	Create new data	201 Created or 202 Accepted
PUT	Full update	200 OK or 204 No Content
PATCH	Partial update	200 OK or 204 No Content
DELETE	Remove data	200 OK or 204 No Content

Additional exception status codes

- 400 Bad Request → If the request is invalid
- 401 Unauthorized → When authentication is required but missing or incorrect.
- 403 Forbidden → When access is denied despite authentication.
- 404 Not Found → If the resource does not exist.
- 405 Method Not Allowed → If the HTTP method is not allowed on the resource.
- 500 Internal Server Error → For unexpected server errors.
- 503 Service Unavailable → If the server is temporarily overloaded or down.

Anatomy of an HTTP Request (in REST APIs)

eazy bytes

HTTP Request – What a Client Sends

Every REST API call starts with an HTTP Request from a client (browser, app, or another service).

Example:

```
GET /api/customers/101?active=true HTTP/1.1
Host: example.com
Authorization: Bearer abc123
Content-Type: application/json
```

HTTP Request = HTTP Method + Path + Path Variables* + Req Params* + Headers* + Body*

```
* - Optional
```

Part	Meaning	Example
HTTP Method	Type of action	`GET`, `POST`, `PUT`, `DELETE`
Path / Endpoint	Identifies resource	`/api/customers/`
Path Variable	Dynamic value in Path	{id} → /customers/101
Query Params	Filters/modifiers	?active=true&sortBy=name
Headers	Metadata (auth, content type, language, etc.)	Authorization: Bearer token
Body (Payload)	Data sent to server (for POST/PUT)	{ "name": "Madan" }

Anatomy of an HTTP Response (from REST APIs)

eazy bytes

HTTP Response – What the Server Sends Back

After processing, the server returns an HTTP Response containing the result

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 101,
  "name": "Madan Reddy",
  "email": "madan@example.com"
}
```

HTTP Response = Status code + Headers* + Body*

```
* - Optional
```

Part	Meaning	Example
Status code	Indicates result	`200 OK` , `201 Created` , `400 Bad Request` , `404 Not Found`
Headers	Metadata or Extra info	`Content-Type: application/json`
Body	Actual data returned by server	JSON or XML (usually JSON)

@PathVariable

eazy bytes

Using @PathVariable to Accept Path Parameters

@PathVariable is used when parameters are part of the URL path. Example Request: GET api/dummy/users/123/posts/456

```
@GetMapping({"/api/dummy/users/{userId}/posts/{postId}", "/api/dummy/users/{userId}"})
public String searchUserPostWithMultiPathVariables(@PathVariable Long userId,
                  @PathVariable(required = false) Long postId)
{

}
```

Making @PathVariable Optional

By default, @PathVariable is required. To make it optional, set required = false:

```
@GetMapping({"/api/dummy/users/{userId}/posts/{postId}", "/api/dummy/users/{userId}"})
public String searchUserPostWithMultiPathVariables(@PathVariable Long userId,
                  @PathVariable(required = false) Long postId)
{

}
```

@PathVariable

eazy bytes

Custom Path Variable Names

By default, the method parameter name should be same as Path Variable placeholder name. But you can customize it.

```
@GetMapping({"/api/dummy/users/{userId}/orders/{orderId}"})
public String searchUserOrderWithMultiPathVariables(@PathVariable(name = "userId")
        Long customerId, @PathVariable Long orderId) {
}
```

Accepting multiple path variables using Map

You can retrieve all path variables dynamically using a Map<String, String>.

```
@GetMapping({"/api/dummy/users/{userId}/address/{addressId}"})
public String searchUserAddressWithMultiPathVariables(@PathVariable Map<String, String>
        pathVariablesMap)
{
    return "Fetched user with id: " + pathVariablesMap.get("userId") + " and address id: "
                + pathVariablesMap.get("addressId");
}
```

@RequestMapping

eazy bytes

What is @RequestMapping?

@RequestMapping is an annotation used in Spring Boot to map an HTTP request URL to a controller method or class. It tells Spring which URL should call which code.

Where can it be used?

- Class level
- Method level

@RequestMapping supports:

- URL path/Values
- HTTP method
- Params
- Headers
- Consumes (input format)
- Produces (output format)

Basic Example

```
@RestController
public class HelloController {

    @RequestMapping("/hello")
    public String sayHello() {
        return "Hello World!";
    }
}
```

When user goes to: /hello → this method executes.

@RequestMapping

eazy bytes

HTTP Method Mapping

By default, @RequestMapping allows all HTTP methods (GET/POST/PUT etc.) But you can specify method explicitly:

```
@RequestMapping(value="/hello", method=RequestMethod.GET)
```

Equivalent shorter annotations

HTTP Method	Shortcut
GET	@GetMapping
POST	@PostMapping
PUT	@PutMapping
PATCH	@PatchMapping
DELETE	@DeleteMapping

@RequestMapping

eazy bytes

Class-level + Method-level usage

```
@RestController
@RequestMapping("/api")
public class UserController {

    @RequestMapping("/users")
    public String getUsers() {
        return "List of Users";
    }
}
```

URL becomes -> /api/users

Produces (Response type)

```
@RequestMapping(value="/hello",produces="application/json")
public String hello() {
    return "{\"msg\":\"Hello\"}";
}
```

Consumes (Request type)

```
@RequestMapping(value="/save", consumes="application/json")
public String save(@RequestBody User user) {
    return "Saved!";
}
```

@RequestMapping is more flexible. It is best when,

- specifying multiple HTTP methods.
- specifying a parent path for all the REST API methods

@RequestParam

eazy bytes

Using @RequestParam to Accept Query Parameters

Use @RequestParam when parameters are sent as part of the URL.
Example Request: `GET /api/dummy/users/search?name=John&gender=Male`

```
@GetMapping("/search")
public String searchUserWithQueryParams(@RequestParam String name,
                                        @RequestParam String sex)
{
   return "Fetched user with query params : " + name + " and gender: " + sex;
}
```

Making @RequestParam Optional

By default, @RequestParam is required. If you do not pass the parameter, the server will return a 400 Bad Request error. To make it optional, set required = false:

```
@GetMapping("/search")
public String searchUserWithQueryParams(@RequestParam(required = false) String name,
                                        @RequestParam String sex)
{
   return "Fetched user with query params : " + name + " and gender: " + sex;
}
```

@RequestParam

eazy bytes

Setting Default Values

You can provide a default value to avoid null when the parameter is missing.

```
@GetMapping("/search")
public String searchUserWithQueryParams(@RequestParam(required = false,defaultValue =
        "Guest") String name, @RequestParam String sex)
{
   return "Fetched user with query params : " + name + " and gender: " + sex;
}
```

Custom Parameter Names

By default, the method argument name is used as the parameter name, but you can customize it.

```
@GetMapping("/search")
public String searchUserWithQueryParams(@RequestParam(required = false,defaultValue =
        "Guest") String name, @RequestParam(name = "gender") String sex)
{
   return "Fetched user with query params : " + name + " and gender: " + sex;
}
```

@RequestParam

eazy bytes

Using @RequestParam with Map

You can retrieve all request parameters dynamically using a Map<String, String>.

```
@GetMapping("/search/map")
public String searchUserWithMapQueryParams(@RequestParam Map<String, String> requestParams) {
    return "Fetched user with query params : " + requestParams.get("name") + " and gender: " +
                                    requestParams.get("gender");
}
```

@RequestHeader

eazy bytes

Using @RequestHeader to Accept Headers

Sometimes, we need to extract custom headers from the request.

```
@GetMapping("/headers")
public String readRequestHeaders(@RequestHeader("User-Agent") String userAgent,
  @RequestHeader(name = "User-Location",required = false,defaultValue = "Hyderabad")  String userLocation)
{
  return "Received: " + userAgent + " " + userLocation;
}
```

Using Map<String, String> to accept all headers

```
@GetMapping("/headers/map")
public String readRequestHeadersWithMap(@RequestHeader Map<String, String> requestHeaders) {
   return "Received: " + requestHeaders.get("User-Agent") + " " + requestHeaders.get("User-Location");
}
```

Using HttpHeaders for additional flexibility

```
@GetMapping("/headers/http-headers")
public String readRequestHeadersWithHttpHeaders(@RequestHeader HttpHeaders requestHeaders) {
  return "Received: " + requestHeaders.get("User-Agent") + " " + requestHeaders.get("User-Location");
}
```

@RequestBody

eazy bytes

Using @RequestBody to Accept JSON Data

When a client sends JSON data in the request body, we use @RequestBody to bind it to a Java object. Spring Boot automatically converts JSON to Java objects using Jackson (default library for JSON processing).

```
@PostMapping
public String createUser(@RequestBody UserDto userDto) {
  return "Created User with the data: " + userDto.toString();
}

public record UserDto(String name, String email, String gender) {

}
```

RequestEntity & ResponseEntity

eazy bytes

Using RequestEntity to Access Full HTTP Request

RequestEntity allows accessing the complete request, including headers and body

```
@PostMapping("request-entity")
public String createUserWithRequestEntity(RequestEntity<UserDto> requestEntity) {
    HttpHeaders httpHeaders = requestEntity.getHeaders();
    UserDto userDto = requestEntity.getBody();
    String queryParam = requestEntity.getUrl().getQuery();
    String pathVariables = requestEntity.getUrl().getPath();
    return "Created User with the data: " + userDto.toString();
}
```

Using ResponseEntity for Custom Responses

ResponseEntity allows customizing the HTTP status, headers, and response body.

```
@PostMapping("request-entity")
public ResponseEntity<String> createUserWithRequestEntity(RequestEntity<UserDto> requestEntity) {
    return ResponseEntity.status(HttpStatus.CREATED)
            .header("Custom-Header", "ExampleValue")
            .body("Created User with the data: " + userDto.toString());
}
```

API Versioning

eazy bytes

When you build REST APIs, over time, you might need to make changes like:

- Adding new fields.
- Changing request/response formats.
- Modifying business logic.
- Deprecating old features.

To avoid breaking existing clients, we use **API Versioning**

There are multiple ways to version REST APIs. The most popular ones are:

1. URI Versioning (Path Versioning)
2. Request Parameter Versioning
3. Header Versioning
4. Media Type Versioning (Content Negotiation)

API Versioning: A Real-World Scenario

eazy bytes

Let's follow the journey of FoodRush, a food delivery company, and see what happens when they DON'T use API versioning... and then how versioning saves the day!

The Setup - FoodRush provides an API that powers:

- Mobile App (iOS & Android) - 1 million users, Website - 500K daily visitors
- Restaurant Partner Apps - 5,000 restaurants, 🚴 Delivery Partner Apps - 10,000 delivery drivers
- Third-party Integrations - 50 business partners

Original API Endpoint (No Versioning):

GET https://api.foodrush.com/restaurants/menu/{restaurant_id}

Original Response:

```
{
  "restaurant": "Pizza Palace",
  "items": [
    {
      "id": 101,
      "name": "Margherita Pizza",
      "price": 12.99,
      "available": true
    }
  ]
}
```

API Versioning: A Real-World Scenario

eazy bytes

The Disaster Day

The development team decides to "improve" the API by making the response more detailed:

New Response (Breaking Change):

```
{
  "restaurant": {
    "name": "Pizza Palace",
    "id": "rest_001",
    "rating": 4.5
  },
  "menu_items": [  // Changed from "items" to "menu_items"
    {
      "id": 101,
      "name": "Margherita Pizza",
      "pricing": {  // Changed from flat "price" to nested object
        "amount": 12.99,
        "currency": "USD"
      },
      "availability": {  // Changed from "available" boolean to object
        "in_stock": true,
        "prep_time": 20
      }
    }
  ]
}
```

What They Changed:

- restaurant changed from string to object
- items renamed to menu_items
- price changed to nested pricing object
- available changed to nested availability object

API Versioning: A Real-World Scenario

eazy bytes

The Solution: API Versioning to the Rescue!

How It Should Have Been Done:

Version 1 (Keep it Stable):

```
GET https://api.foodrush.com/v1/restaurants/menu/{restaurant_id}
```

Returns the ORIGINAL format - all existing apps keep working!

Version 2 (New Features):

```
GET https://api.foodrush.com/v2/restaurants/menu/{restaurant_id}
```

Returns the NEW improved format

API Versioning: A Real-World Scenario

eazy bytes

The Happy Outcome with Versioning

Gradual Migration Timeline:

Week 1: Launch v2

- v1 still works - zero disruption
- New features available in v2
- Email sent to all developers about v2

Month 1: Mobile App Update

- New app version uses v2
- Old app versions still work with v1
- Users update at their own pace

Month 2: Partner Migration

- Partners test v2 in staging
- Switch to v2 when ready
- No forced deadlines

Month 6: Deprecation Notice

- "v1 will be deprecated in 6 months"
- 99% of traffic now on v2
- Help stragglers migrate

Year 1: v1 Sunset

- Turn off v1 with confidence
- All clients successfully migrated
- Zero business disruption

Various approaches of API Versioning

eazy bytes

URI Versioning (Path Versioning)

```
GET /api/v1/users
GET /api/v2/users
```

Request Parameter Versioning

```
GET /api/users?version=1
GET /api/users?version=2
```

Header Versioning

```
GET /api/users
Headers:
X-API-VERSION: 1
```

Media Type Versioning (Content Negotiation)

```
GET /api/users
Accept: application/vnd.myapp.v1+json
```

This tells the server: “Return version 1 of this API’s response, formatted as JSON.”

Various approaches of API Versioning

eazy bytes

Breakdown of the MIME Type

`MIME = Multipurpose Internet Mail Extensions`

It is a standard way of telling the computer what type of data is being sent so that it can be handled correctly.
In simple words: MIME describes the format of the content.

Part	Meaning
`application`	Top-level MIME type (general category of data)
`vnd`	Vendor-specific (custom type)
`myapp`	Your application / company name
`v1`	API version (v1, v2, v3…)
`+json`	Format of data → JSON

Various approaches of API Versioning

eazy bytes

Why vnd? -> vnd stands for vendor-specific. It means this media type is specific to YOU (your app/organization), not a global standard. Ex: `application/vnd.github.v3+json` is a custom JSON format defined by GitHub.

Why use media type versioning?

Because:

- You don’t pollute URLs with /v1, /v2
- API versioning is done using headers rather than paths
- URL remains stable, only representation changes
- More REST-compliant — same resource, different representations

Legacy approaches of API Versioning

eazy bytes

URI Versioning (Path Versioning)

```
@GetMapping({"","/","/v1"})
public String defaultPathVersion() {
    return "Response from defaultPathVersion API 1.0.0";
}

@GetMapping("/v2")
public String pathV2Version() {
    return "Response from pathV2Version API 2.0.0";
}
```

Request Parameter Versioning

```
@GetMapping(params = "version=1" )
public String defaultReqParamVersion() {
    return "Response from defaultReqParamVersion API 1.0.0";
}

@GetMapping(params = "version=2" )
public String v2ReqParamVersion() {
    return "Response from v2ReqParamVersion API 2.0.0";
}
```

Legacy approaches of API Versioning

eazy bytes

Request Header Versioning

```
@GetMapping(headers = "X-API-VERSION=1")
public String defaultReqHeaderVersion() {
    return "Response from defaultReqHeaderVersion API 1.0.0";
}

@GetMapping(headers = "X-API-VERSION=2" )
public String v2ReqHeaderVersion() {
    return "Response from v2ReqHeaderVersion API 2.0.0";
}
```

Media Type Versioning (Content Negotiation)

```
@GetMapping(produces = "application/vnd.eazyapp.v1+json")
public String defaultMediaTypeVersion() {
    return "Response from defaultMediaTypeVersion API 1.0.0";
}

@GetMapping(produces = "application/vnd.eazyapp.v2+json")
public String v2MediaTypeVersion() {
    return "Response from v2MediaTypeVersion API 2.0.0";
}
```

API Versioning using Spring

eazy bytes

URI Versioning (Path Versioning)

```
@RestController
@RequestMapping("/api/versions/{v}")
public class VersionController {

    @GetMapping(version = "1.0")
    public String defaultVersion() {
        return "Response from default API 1.0 Version";
    }

    @GetMapping(version = "2.0+")
    public String v2Version() {
        return "Response from API 2.0 Version";
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.usePathSegment(2).addSupportedVersions("1.0", "2.0", "3.0");
    }

}
```

API Versioning using Spring

eazy bytes

Request Parameter Versioning

```
@RestController
@RequestMapping("/api/versions")
public class VersionController {

    @GetMapping(version = "1.0")
    public String defaultVersion() {
        return "Response from default API 1.0 Version";
    }

    @GetMapping(version = "2.0+")
    public String v2Version() {
        return "Response from API 2.0 Version";
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.useQueryParam("version").addSupportedVersions("1.0", "2.0",
"3.0").setDefaultVersion("1.0");
    }

}
```

API Versioning using Spring

eazy bytes

Request Header Versioning

```
@RestController
@RequestMapping("/api/versions")
public class VersionController {

    @GetMapping(version = "1.0")
    public String defaultVersion() {
        return "Response from default API 1.0 Version";
    }

    @GetMapping(version = "2.0+")
    public String v2Version() {
        return "Response from API 2.0 Version";
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.useRequestHeader("X-API-VERSION").addSupportedVersions("1.0", "2.0",

          "3.0").setDefaultVersion("1.0");
    }

}
```

API Versioning using Spring

eazy bytes

Media Type Versioning (Content Negotiation)

```
@RestController
@RequestMapping("/api/versions")
public class VersionController {

    @GetMapping(version = "1.0")
    public String defaultVersion() {
        return "Response from default API 1.0 Version";
    }

    @GetMapping(version = "2.0+")
    public String v2Version() {
        return "Response from API 2.0 Version";
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {

configurer.useMediaTypeParameter(MediaType.parseMediaType("application/vnd.eazyapp+json"),
                    ”v").addSupportedVersions("1.0", "2.0",
"3.0").setDefaultVersion("1.0");    }

}
```

Package Structure Approaches in Spring Boot

eazy bytes

Spring Boot doesn't force a package structure — but following clean architecture practices makes your project scalable, readable, and easy to maintain.

Basic Rule

Keep your main application class at the root package . So Spring Boot can scan all components automatically.

Example:

```
com.example.myapp
  └── MyAppApplication.java
```

There are two popular approaches:

1) By Layer (Traditional Layered Architecture)
2) By Feature / Domain (Recommended for microservices & large apps)

Package Structure Approaches in Spring Boot

eazy bytes

By Layer (Traditional Layered Architecture)

Organize code by technical roles.

```
com.example.myapp
 ├── controller
 ├── service
 ├── repository
 ├── entity
 ├── exception
 └── config
```

Pros

- Easy to understand
- Good for small-to-medium projects
- Common & beginner friendly

Cons

- Hard to scale for large domains
- Business logic spreads across layers

By Feature / Domain

Group files by business capability.

```
com.example.myapp
 └── user
       ├── controller
       ├── service
       ├── repository
       ├── entity
       └── dto
 └── job
       ├── controller
       ├── service
       ├── repository
       ├── entity
       └── dto
```

Pros

- High modularity
- Easy to maintain and scale
- Easier team collaboration
- Improves domain-driven design

Cons

- Slightly more initial planning required

Package Structure Approaches in Spring Boot

eazy bytes

Additional Supporting Packages

Example:

```
com.example.myapp
  ├── common          # Reusable helpers
  ├── config          # Configurations
  ├── security        # Security config, filters
  ├── exception       # Error handling
  └── util            # Utilities
```

Naming Conventions

- Lowercase package names
- Avoid plural for domain name (job, not jobs)
- Consistent naming

Package Structure Approaches in Spring Boot

eazy bytes

```
com.example.jobportal
 ├── JobPortalApplication.java
 |
 ├── common
 |    ├── dto
 |    ├── exception
 |    ├── util
 |    └── mapper
 |
 ├── config
 |    ├── security
 |    └── web
 |
 ├── auth
 |    ├── controller
 |    ├── service
 |    ├── dto
 |    ├── entity
 |    ├── repository
 |    ├── mapper
 |    ├── jwt
 |    └── model
 |
 ├── user
 |    ├── controller
 |    ├── service
 |    ├── dto
 |    ├── entity
 |    ├── repository
 |    ├── mapper
 |    └── enums
 |
```

Authentication & Authorization Package

```
auth
 ├── AuthController
 ├── AuthService
 ├── entity (UserAuth / LoginHistory)
 ├── jwt
 |    ├── JwtFilter
 |    ├── JwtUtils
 ├── dto
 ├── repository
 └── mapper
```

config Package

```
config
 ├── security
 |    ├── SecurityConfig
 |    ├── CustomUserDetailsService
 |    ├── AuthenticationEntryPoint
 |    ├── RoleHierarchyConfig
 |    └── CorsConfig
 └── web
      ├── OpenApiConfig
      └── WebConfig
```

Instead of random endpoints, group them clearly:

```
/auth/...
/jobs/...
/companies/...
/users/...
/employers/...
/admin/...
/support/...
```

Spring Boot Console Logging Pattern Configuration

eazy bytes

What is logging.pattern.console?

`logging.pattern.console` is a Spring Boot property used to customize how logs appear in the console.

- Controls how log messages appear in the console output
- Defines the format, structure, and styling of log entries
- Part of Spring Boot's built-in logging configuration

Example

```
logging.pattern.console=${CONSOLE_LOG_PATTERN:
  %green(%d{HH:mm:ss.SSS}) %blue(%-5level)
  %red([%thread]) %yellow(%logger{15}) - %cyan(%msg%n)}
```

What does this mean?

The value uses environment variable substitution:

- CONSOLE_LOG_PATTERN → Environment variable name
- If not provided → Uses default pattern

Spring Boot Console Logging Pattern Configuration

eazy bytes

Explanation of Pattern Components

Component	Purpose	Example output
%green(%d{HH:mm:ss.SSS})	Timestamp in green	14:23:45.678
%blue(%-5level)	Log level (5 chars, left-aligned) in blue	INFO, ERROR
%red([%thread])	Thread name in red	[main]
%yellow(%logger{15})	Logger name (max 15 chars) in yellow	c.e.UserService
%cyan(%msg)	Actual log message in cyan	User created successfully
%n	New line character	

Spring Boot Console Logging Pattern Configuration

eazy bytes

Environment Variable Syntax

Structure: `${ENV_VARIABLE:default_value}`

Example: `${CONSOLE_LOG_PATTERN:fallback}`

Before colon (:): Environment variable name. After colon: Fallback/default value if env variable not set

Benefits:

- Override without changing code
- Different patterns for different environments
  - ✓ Development: Detailed colored logs
  - ✓ Production: JSON format for log aggregators
  - ✓ Testing: Minimal logging

Spring Boot DevTools

eazy bytes

Spring Boot DevTools is a developer tool that improves the development experience by:

- Auto-restarting the application when code changes.
- Disabling caching in development mode so changes take effect immediately.

To enable DevTools, add the dependency in your pom.xml,

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
```

Spring Boot automatically disables DevTools in production for security and performance. You don't need to remove it—it runs only in development mode.

`scope=runtime` → DevTools is available only at runtime, not at compile time.
`optional=true` → Prevents DevTools from being included automatically in other dependent projects.

H2 Database in Spring Boot

eazy bytes

What is H2 Database?

H2 is a lightweight, fast, and in-memory database used for:

- Development & Testing (because it resets on restart)
- Embedded Mode (no installation needed)
- Standalone Mode (can store data on disk)

Why Use H2 in Spring Boot?

- No need to install – It runs in memory.
- Fast & lightweight – Perfect for testing.
- Supports SQL – Similar to MySQL, PostgreSQL.
- Provides a Web Console – Easy to view data.

H2 Database in Spring Boot

eazy bytes

Adding H2 to Your Spring Boot Project – By adding below dependencies in pom.xml, Spring Boot automatically detects H2 DB and configures it

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-h2console</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

H2 Database in Spring Boot

eazy bytes

Configuring H2 Database in application.properties

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:testdb  # In-memory database
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Enable H2 Console (for viewing tables & data)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

What does above properties do?

- jdbc:h2:mem:testdb → Uses an in-memory database (testdb).
- sa (default username), empty password.
- H2 Console enabled → View database in the browser.
- H2 Dialect → Configures Hibernate for H2.

Running H2 Database in Spring Boot

1. Run your Spring Boot application.
2. Open a browser and go to: http://localhost:8080/h2-console
3. Enter database details:
    - JDBC URL: jdbc:h2:mem:testdb
    - Username: sa
    - Password: (leave empty)
4. Click Connect

Now, you can execute SQL queries and view tables!

H2 Persistent Mode

By default, H2 resets data on restart (because it’s in-memory). To keep data between restarts, change the URL:

spring.datasource.url=jdbc:h2:file:~/jobportal;AUTO_SERVER=true

Spring Boot Database Initialization

eazy bytes

Spring Boot auto-creates DB schema & inserts data from SQL scripts.

Default script locations :

- Schema (DDL) → optional:classpath*:schema.sql
- Data (DML) → optional:classpath*:data.sql

Custom locations :

- Set using spring.sql.init.schema-locations and spring.sql.init.data-locations

The optional : prefix means that the application will start even when the files do not exist. To have the application fail to start when the files are absent, remove the optional: prefix.

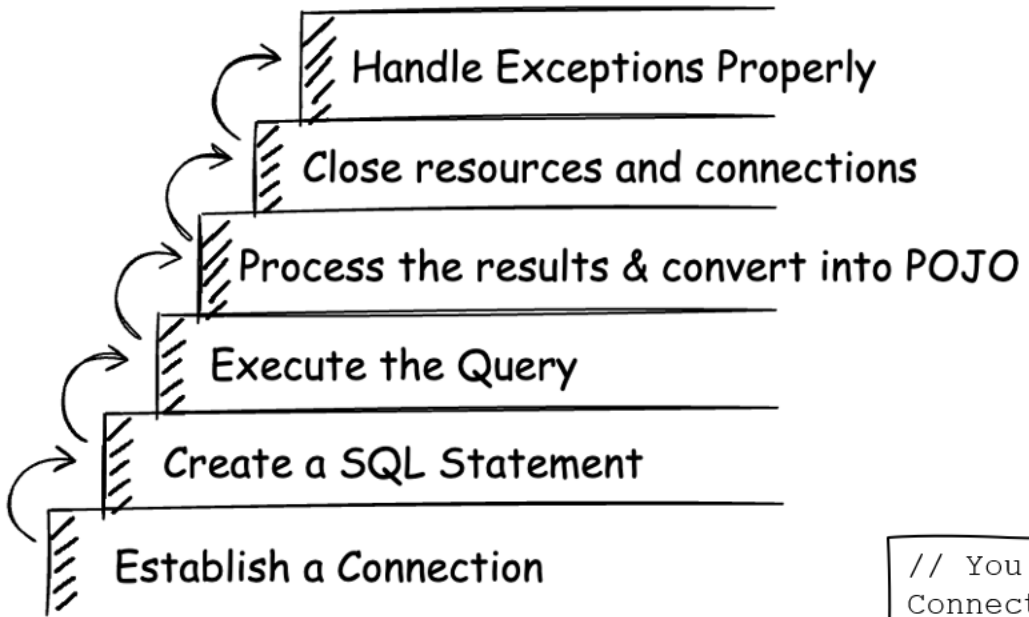
When does initialization run?

- By default: embedded (Only for in-memory databases)
- Force always: Set spring.sql.init.mode=always
- Disable: Set spring.sql.init.mode=never

Typical Steps while working with a DB using Java

eazy bytes

All the below steps needs to be managed by Developer manually when not using any frameworks like Spring Data JPA,



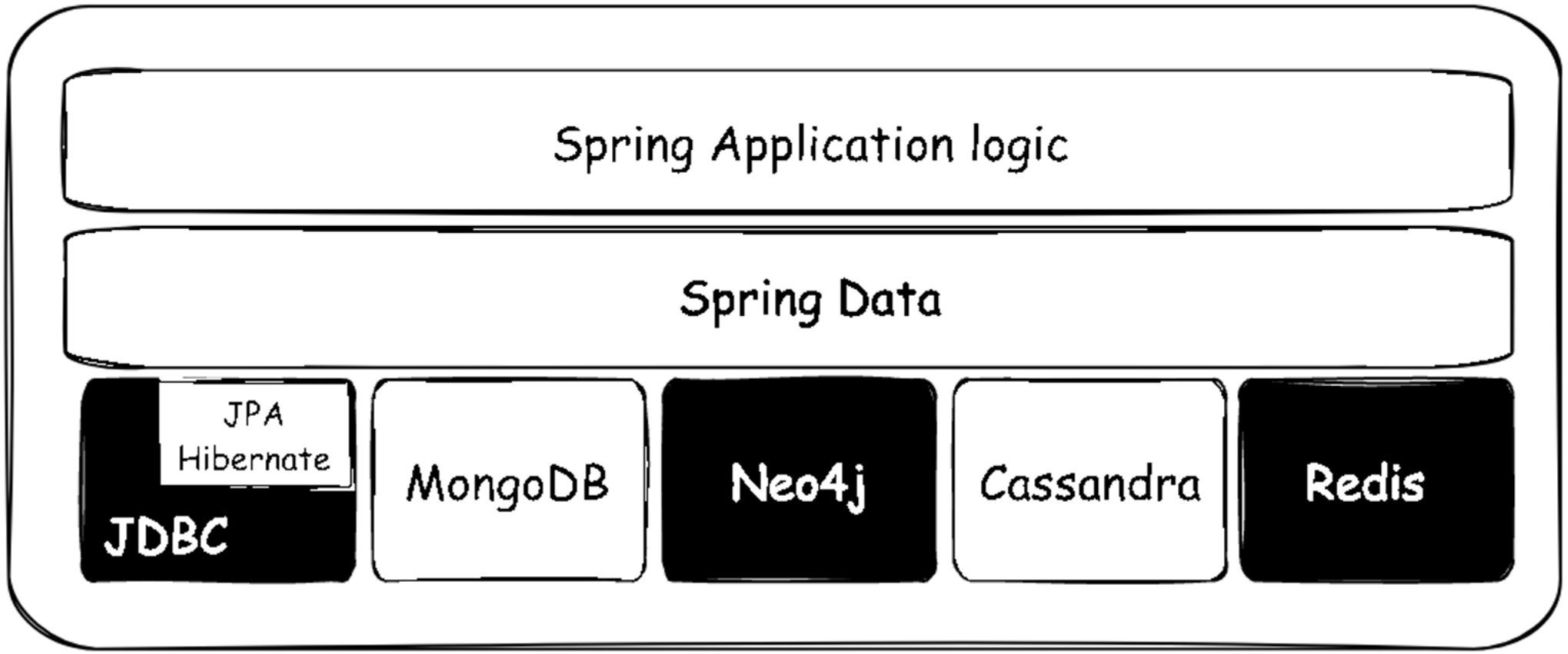

```
// You'd have to write code like this everywhere:
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, username, password);
    PreparedStatement stmt = conn.prepareStatement("SELECT * FROM
                                                    companies");
    ResultSet rs = stmt.executeQuery();
    // Process results...
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    if (conn != null) conn.close(); // Don't forget to close!
}
```

Spring Data

eazy bytes

Spring Data is a part of the Spring Framework that simplifies database access in Java applications. It provides an easy and consistent way to interact with databases without writing boilerplate code.

Spring Data is an umbrella project that includes several modules to work with different types of databases (relational, NoSQL, etc.).



Why Use Spring Data?

Spring Data helps you to:

- Reduce boilerplate code for database operations
- Simplify CRUD (Create, Read, Update, Delete) operations
- Work seamlessly with Spring Boot

Spring Data with Different Databases

Spring Data supports:

- Relational Databases (MySQL, PostgreSQL, Oracle)
- NoSQL Databases (MongoDB, Redis, Cassandra)
- Graph Databases (Neo4j)

Spring Data vs Spring Data JPA

eazy bytes

Spring Data and Spring Data JPA are closely related, but they are not the same. Let's break it down in simple terms so you can understand the differences clearly.

- ✓ **Spring Data** is an umbrella project that simplifies working with databases.
- ✓ It provides common functionalities for different types of databases, including SQL, NoSQL, Redis, Neo4j etc.
- ✓ Think of Spring Data as a toolbox that contains different tools for working with databases

- ✓ **Spring Data JPA** is a subproject of Spring Data that focuses on JPA (Java Persistence API)
- ✓ It helps you interact with relational databases like MySQL, PostgreSQL, Oracle, and H2 without writing complex SQL queries.
- ✓ Spring Data JPA is a part of Spring Data but specifically for JPA-based databases. It uses Hibernate as the default JPA provider to manage the database.

The most commonly used sub module of Spring Data is **Spring Data JPA**, which simplifies working with **relational databases**

How Spring Data JPA and Hibernate are related?

eazy bytes

JPA (Java Persistence API) is a specification (not an implementation) that defines a standard way to manage relational data in Java applications. Think of it as a contract or set of rules that describes how Java objects should be mapped to database tables.

Hibernate is a JPA implementation - the actual ORM (Object-Relational Mapping) framework that does the heavy lifting. It handles the actual database operations, SQL generation, caching, lazy loading, transaction management at the ORM level. Sits between your Java objects and the database.

Spring Data JPA is an abstraction layer on top of JPA implementations (like Hibernate). It provides repository pattern, query derivation, pagination, and reduces boilerplate code. Sits on top of any JPA provider (Hibernate, EclipseLink, OpenJPA, etc.)

`Spring Data JPA hides MOST Hibernate complexity.`

Concept	What it is ?	Provides
JPA	Specification	Rules, annotations
Hibernate	Implementation of JPA	Actual ORM behavior
Spring Data JPA	Abstraction on top of JPA	Repositories, less boilerplate

`Spring Data JPA → uses → JPA → implemented by → Hibernate`

How Spring Data JPA and Hibernate are related?

eazy bytes

JPA is like the international car standards (what every car must have: steering, brakes, gears) –
just a rulebook, you can't drive it!

Hibernate is like Toyota - an actual car manufacturer that builds real cars following those standards, plus adds extra features like cruise control.

Spring Data JPA is like Tesla Autopilot - you just say "take me to work" and it handles all the driving
(gear shifts, navigation, parking) automatically!

What is ORM Framework?

eazy bytes

ORM = Object-Relational Mapping
In simple words: ORM is a translator between your Java objects and database tables.

The Problem ORM Solves

The Mismatch:

Java thinks in Objects: User, Order, Product (with properties and methods)
Databases think in Tables: rows, columns, foreign keys, SQL

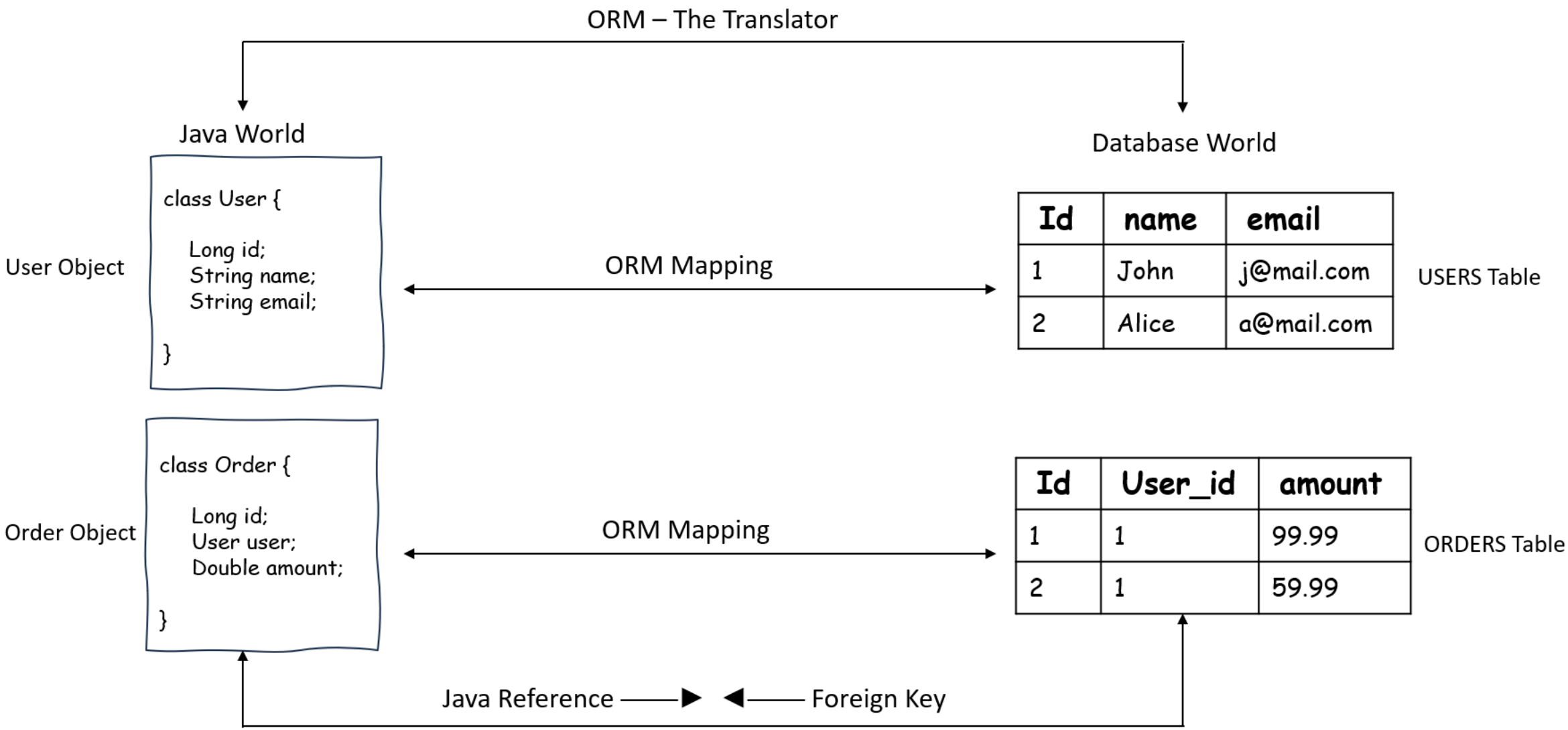
ORM is the bridge that automatically converts between these two different worlds!

Simple Benefits:

Write Java, not SQL - Think in objects, not tables
No manual conversion - ORM does the translation
Less code - One line vs 10+ lines of JDBC
Fewer errors - No typos in SQL strings
Database independent - Same code works with MySQL, PostgreSQL, Oracle

What is ORM Framework?

eazy bytes



What is ORM Framework?

eazy bytes

Your Code Says	ORM Transalates to
user.setName("John")	UPDATE users SET name='John'
orderRepo.save(order)	INSERT INTO orders VALUES(...)
user.getOrders()	SELECT * FROM orders WHERE user_id = ?
userRepo.findAll()	SELECT * FROM users

ORM is a tool that automatically converts Java objects into database rows and vice-versa, so you don't need to write SQL manually. It makes database interaction easier.

Setup Spring Data JPA

eazy bytes

Step 1: Add Dependencies

Include the dependencies of Spring Data JPA and relational DB like H2, MySQL etc.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-h2console</artifactId>
</dependency>
```

Step 2: Configure Database details in application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.show-sql=true
```

Important components of Spring Data JPA

eazy bytes

1. Entity

What it does:

- Represents a table in the database.
- Each instance of the entity is a row in the table.

```
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = ”name" )
    private String name;

    @Column(name = ”email" )
    private String email;

    // Getters and Setters
}
```

- @Entity → Marking a class to represents a table in the database.
- @Table(name = "users") → Defines the table name (Optional when table name and class name is same)
- @Id → Marks the primary key.
- @GeneratedValue → Allow to configure the primary key value strategy
- @Column(name= ”name") → Defines the column name (Optional when column name and field name is same)

Important components of Spring Data JPA

eazy bytes

2. Repository

What it does:

- Allows you to interact with the database without writing queries.
- Spring Data JPA provides support to perform CRUD operations with out writing any code

```
import org.springframework.data.jpa.repository.JpaRepository;

@Repository // Optional
public interface UserRepository extends JpaRepository<User, Long> {

}
```

- JpaRepository<User, Long> → Provides ready-made CRUD supported methods like save(), findById(), delete(), etc.
- UserRepository can be injected into the service layer to trigger the DB related operations

- If your repositories and entities are in the different package as @SpringBootApplication, then you should mention @EnableJpaRepositories, @EntityScan annotations along with the package details.

Spring Data important interfaces

eazy bytes

No matter which persistence technology your application uses, Spring Data offers a unified set of interfaces that you can extend to define its persistence capabilities.

At the core of Spring Data's repository abstraction is the **Repository** interface, which serves as the foundation for building data access layers efficiently.

The **Repository** interface is the most abstract contract in Spring Data. Extending it marks your interface as a Spring Data repository, but it provides no predefined operations like saving or retrieving records. It acts purely as a marker interface without declaring any methods.

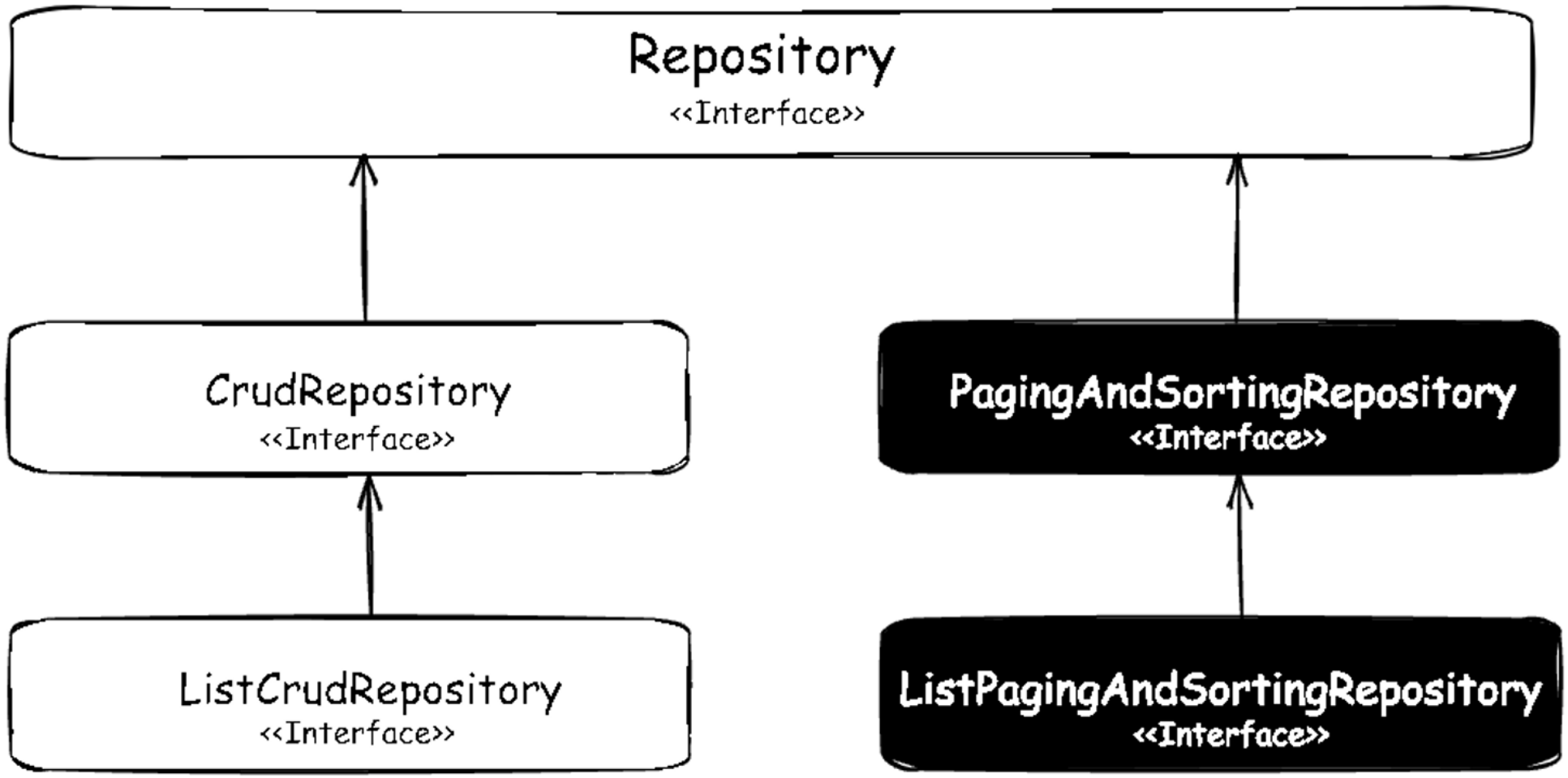
CrudRepository is the simplest Spring Data contract that provides basic persistence operations—create, retrieve, update, and delete. Extending it enables these functionalities in your application. **ListCrudRepository** extends CrudRepository, returning List instead of Iterable where applicable.

PagingAndSortingRepository provides methods for retrieving entities with pagination and sorting.
ListPagingAndSortingRepository extends it, returning List instead of Iterable where applicable.

Spring Data important interfaces

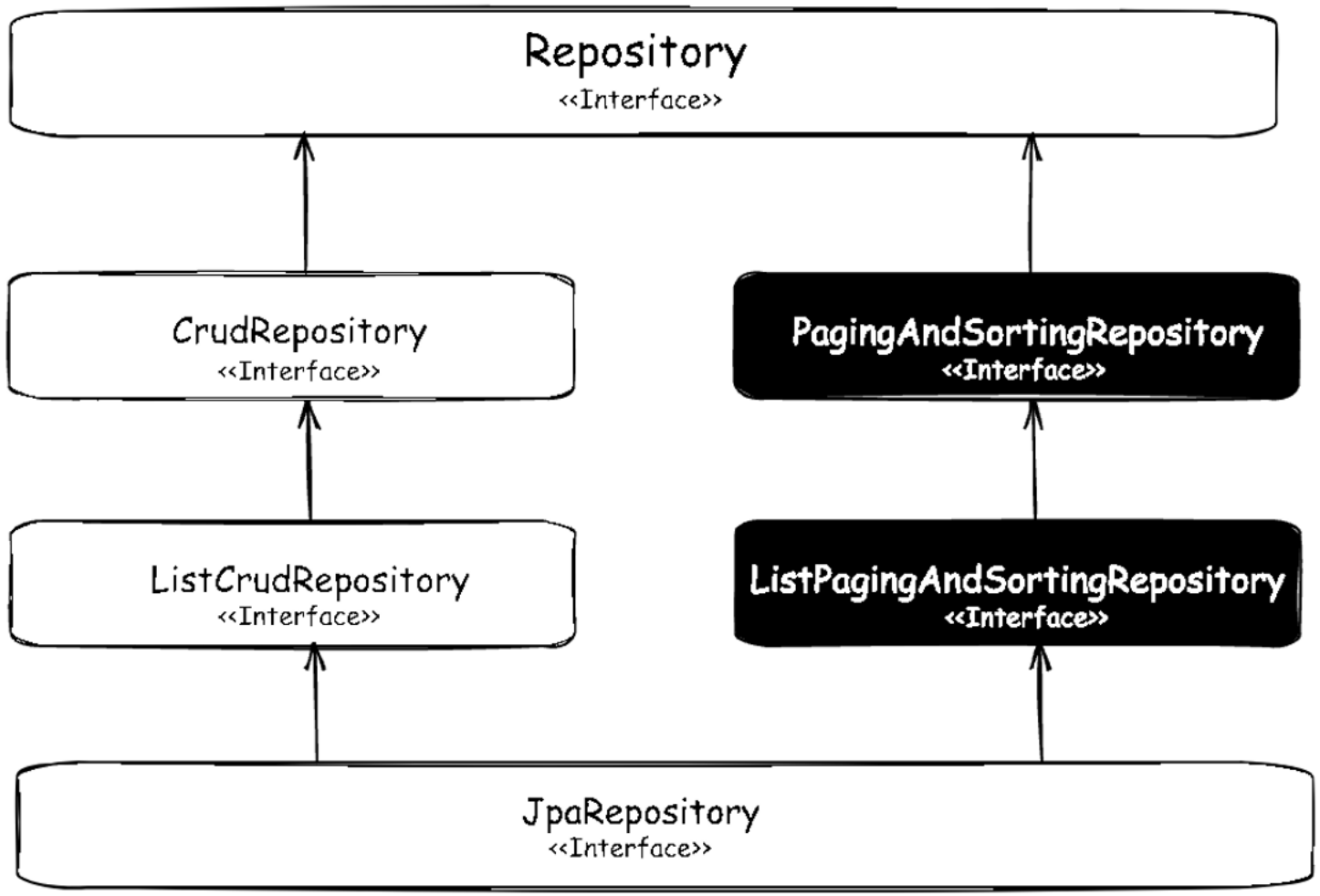
eazy bytes

To implement repositories in Spring Data, extend one of its core interfaces: CrudRepository, ListCrudRepository, PagingAndSortingRepository, or ListPagingAndSortingRepository. Each provides different levels of persistence functionality.



Spring Data JPA important interfaces

eazy bytes



Choose JpaRepository from Spring Data JPA, when working with relational databases like MySQL, PostgreSQL, or Oracle, where data is structured in tables with relationships.

Quick Tips

It’s important to distinguish between the @Repository annotation and the Spring Data Repository interface.

Spring Data follows the Interface Segregation Principle, providing multiple repository interfaces that extend one another. This allows applications to adopt only the functionality
they need, avoiding unnecessary complexity.

Different Spring Data modules offer technology-specific repository contracts. For instance:

- Spring Data JPA provides JpaRepository for relational databases.
- Spring Data MongoDB offers MongoRepository for working with MongoDB.

By extending the appropriate interface, applications can leverage Spring Data’s powerful abstraction while maintaining flexibility and efficiency.

Lombok — Reduce Boilerplate, Boost Productivity

eazy bytes

Lombok is a lightweight Java library that helps you write less code and keeps your classes clean and readable. It is a annotation-based Java library that auto-generates boilerplate code at compile time for the below components,

- Getters / Setters
- Constructors
- Builder pattern
- Logging
- Equals & HashCode

Example: Without Lombok (~30 lines for simple POJO)

```
public class Employee {
    private String name;
    private int age;

    public Employee() {}

    public Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

Example: With Lombok (Just 8 lines → Same functionality)

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Employee {
    private String name;
    private int age;
}
```

Lombok — Reduce Boilerplate, Boost Productivity

eazy bytes

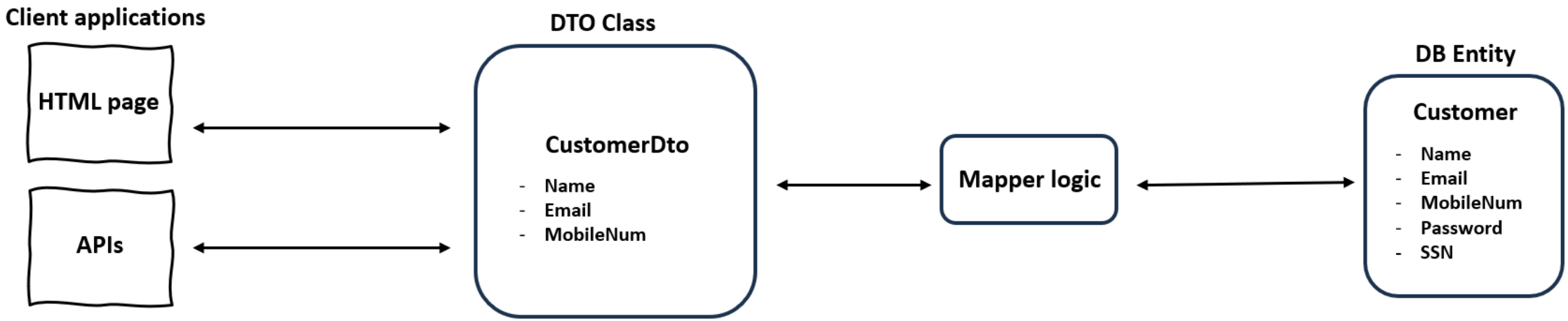
Key Lombok Annotations

Annotation	Purpose
@Getter / @Setter	Auto-generate methods
@Data	Getter + Setter + toString + equals + hashCode
@AllArgsConstructor	Constructor with all fields
@NoArgsConstructor	Default constructor
@Builder	Build objects easily
@Slf4j	Auto-create logger

DTO (Data Transfer Object) pattern

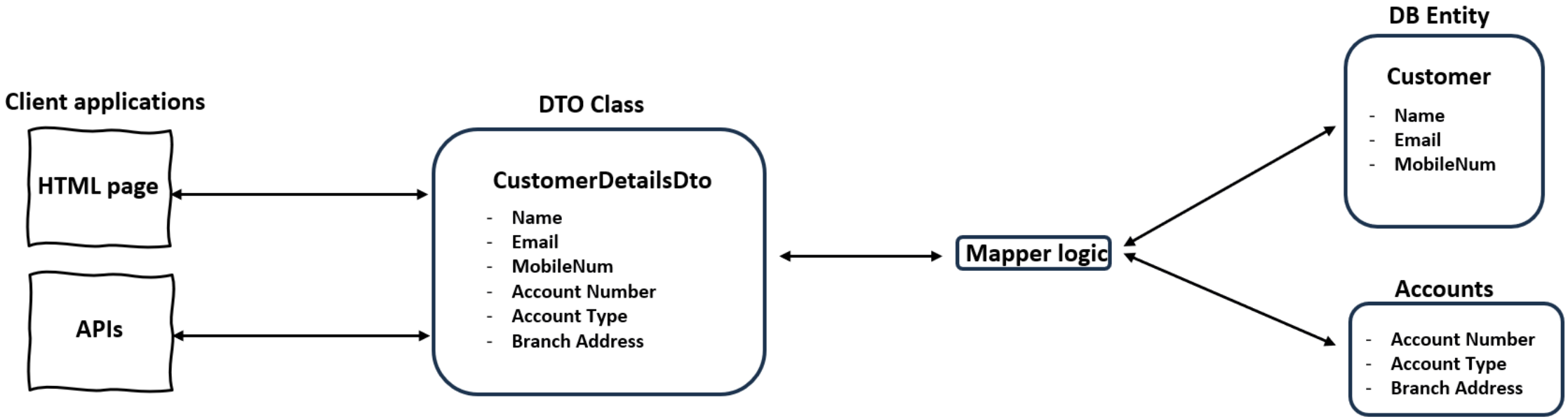
eazy bytes

The Data Transfer Object (DTO) pattern is a design pattern that allows you to transfer data between different parts of your application. DTOs are simple objects that contain only data, and they do not contain any business logic. This makes them ideal for transferring data between different layers of your application, such as the presentation layer and the data access layer.



DTO (Data Transfer Object) pattern

eazy bytes



Here are some of the benefits of using the DTO pattern:

Reduces network traffic: DTOs can be used to batch up multiple pieces of data into a single object, which can reduce the number of network requests that need to be made. This can improve performance and reduce the load on your servers.

Encapsulates serialization: DTOs can be used to encapsulate the serialization logic for transferring data over the wire. This makes it easier to change the serialization format in the future, without having to make changes to the rest of your application.

Decouples layers: DTOs can be used to decouple the presentation layer from the data access layer. This makes it easier to change the presentation layer without having to change the data access layer.

CORS (Cross-Origin Resource Sharing)

eazy bytes

What is CORS?

CORS is a security feature built into web browsers. It prevents a website from making requests to a different domain unless explicitly allowed.

Example of a CORS Issue

Imagine you have:

- A frontend running at http://localhost:5173 (React)
- A backend API running at http://localhost:8080 (Spring Boot)

If your frontend tries to fetch data from the backend below CORS error will be thrown by the browser:

CORS Error:

```
Access to fetch at 'http://localhost:8080/api/companies' from origin
'http://localhost:5173' has been blocked by CORS policy.
```

CORS (Cross-Origin Resource Sharing)

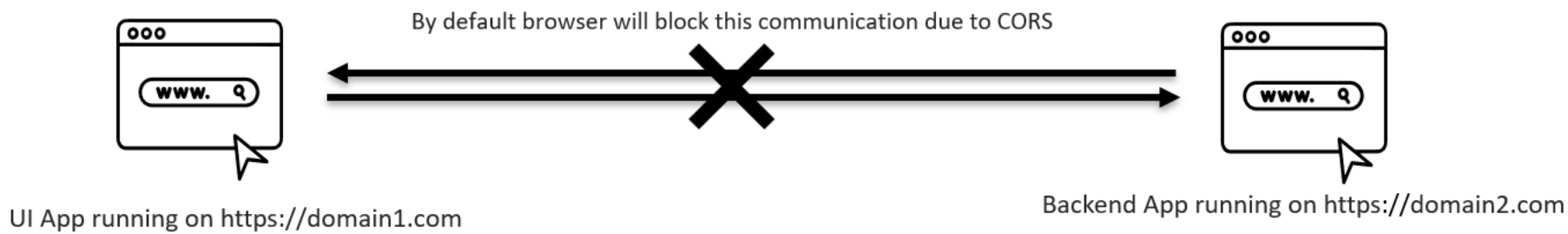
eazy bytes

CORS is not a security issue/attack but the default protection provided by browsers to stop sharing the data/communication between different origins.

When does two origins consider different ?

Two origins consider different if they have:

- a different scheme (HTTP or HTTPS)
- a different domain
- a different port



CORS (Cross-Origin Resource Sharing)

eazy bytes

How to fix CORS?

If a web application's UI is deployed on one server and needs to communicate with a REST service hosted on another, we can enable this interaction by making changes in the backend app. Backend can decide from which origins, it want to communicate.

How to Fix CORS using Spring Boot?

Option 1: Using @CrossOrigin (Simple & Quick):

By default, mentioning @CrossOrigin alone allows:
All origins, All headers, All HTTP methods to which the controller method is mapped.

Enable CORS on a Specific Endpoint

```
@RestController
@RequestMapping("/companies")
public class CompanyController {

    @CrossOrigin(origins = "http://localhost:5173")
    @GetMapping
    public String getAllCompanies() {
        return "List of companies";
    }
}
```

Enable CORS for the Entire Controller

```
@RestController
@RequestMapping("/companies")
@CrossOrigin(origins = "http://localhost:5173")
public class CompanyController {

    @GetMapping
    public String getAllCompanies() {
        return "List of companies";
    }
}
```

CORS (Cross-Origin Resource Sharing)

eazy bytes

Option 2: Using a Filter

If you have many controllers, it's better to configure CORS globally.

```
@Configuration
public class CorsConfig {

    @Bean
    public CorsFilter corsFilter() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(Arrays.asList("http://localhost:5173"));
        config.setAllowedMethods(Collections.singletonList("*"));
        config.setAllowedHeaders(Arrays.asList("Content-Type"));
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return new CorsFilter(source);
    }

}
```

Option 3: Using Spring Security (will be discussed in Spring Security sections)

CORS (Cross-Origin Resource Sharing)

eazy bytes

Option 4: Using WebMvcConfigurer

We can override the method addCorsMappings() from the WebMvcConfigurer interface,

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

        @Override
        public void addCorsMappings(CorsRegistry registry) {

                registry.addMapping("/api/**")
                        .allowedOrigins("https://domain2.com")
                        .allowedMethods("PUT", "DELETE")
                        .allowedHeaders("header1", "header2", "header3")
                        .exposedHeaders("header1", "header2")
                        .allowCredentials(true).maxAge(3600);

                // Add more mappings...
        }
}
```

CORS (Cross-Origin Resource Sharing)

eazy bytes

When CORS is enabled, Spring Boot adds headers like below:

Header	Description
Access-Control-Allow-Origin	Specifies which origins are allowed (e.g., http://localhost:5173)
Access-Control-Allow-Methods	Lists allowed HTTP methods (GET, POST, PUT, DELETE)
Access-Control-Allow-Headers	Specifies allowed request headers
Access-Control-Allow-Credentials	Allows sending cookies (e.g., JWT tokens)

When NOT to Use CORS config?

If your frontend and backend run on the SAME domain, CORS is not needed.

Example:

- https://mywebsite.com (frontend)
- https://mywebsite.com/api (backend) → No CORS required.

What is Docker & Why Should Anyone Use It?

eazy bytes

Docker is a tool that helps to package an application and run it anywhere. Think of Docker as a magic box that contains:

- Application
- All the software it needs
- All the configurations it depends on

So wherever you carry that box, the app works exactly the same way.

Docker is like a shipping container for a software. Just as shipping containers revolutionized cargo transport (same container works on ships, trains, trucks)

Docker packages a application with everything it needs to run. The app becomes portable - works the same on any computer with Docker installed.

Before Docker:

- “It works on my system!”
- Different environments: DEV ≠ QA ≠ PROD
- Installing dependencies manually
- Painful setup for new developers
- Server crashes due to mismatched software versions

Docker solves this problem by providing:

- Consistency — same behavior everywhere
- Isolation — each app runs in its own box
- Easy setup — run one command → app is ready
- Portability — from laptop → server → cloud
- Faster deployments — ship entire environment as an image

What is Docker & Why Should Anyone Use It?

eazy bytes

Before Docker, Developers used to face lot of problems like,

- “It works on my system!”
- Different environments: DEV ≠ QA ≠ PROD
- Installing dependencies manually
- Painful setup for new developers
- Server crashes due to mismatched software versions

For example, to setup a Spring Boot app requires:

- JDK 25 (hope you don't have Java 8 projects!)
- Maven or Gradle
- MySLQ database server
- Redis for caching
- RabbitMQ for messaging
- Configure database connections, ports, environment variables

With Docker: One command -> `docker-compose up`

Your Spring Boot app + PostgreSQL + Redis + RabbitMQ all start together. No Java version conflicts on your machine.
Same environment from developer laptop to production.
New team member joins? They're running the app in 2 minutes!

Docker Jargons

eazy bytes

Docker Image - The Blueprint

A Docker Image is a blueprint / template of an application. It contains:

- Application code
- Runtime (Java, Python, Node, etc.)
- Required libraries
- OS-level dependencies

Think of it as a cake recipe with all ingredients pre-measured and included.
Read-only template - you don't modify the recipe itself
Can be shared like sharing a photo - upload to Docker Hub, others can download

Docker Container - The Living Application

A Container is a running instance of an image.
If Image is the recipe, Container is the actual cake you baked.
You can create multiple containers from one image (like baking multiple cakes from one recipe)

Docker Containers are,

- Lightweight
- Fast to start
- Isolated from each other - like apps running in separate bubbles
- Disposable (stop/delete anytime)

Docker Jargons

eazy bytes

Docker Volume - The Permanent Storage

Containers do not keep data permanently. If you restart/delete a container → data is gone.

Volumes solve this problem:

- They provide permanent storage
- Data survives container restarts
- Multiple containers can share the same data

Think of a volume as a USB drive connected to your container.

More Essential Terms

Dockerfile: A text file with instructions to build your image (like writing your own recipe)

Docker Hub: The "App Store" for Docker images - download or share images

Port Mapping: Containers have their own internal network. To access the app from your host machine, you must map ports.

Example: 8080 (your laptop) --> 80 (container)

It's like giving the container a door so the outside world can enter.

Docker Compose: Tool to run multiple containers together (like conducting an orchestra)

Registry: Storage location for images (Docker Hub is a public registry)

Setup MySQL DB using Docker

eazy bytes

This command starts a MySQL database in a Docker container and makes it behave just like a locally installed MySQL server — but cleaner, safer, and fully isolated.

```
docker run -p 3306:3306 --name jobportaldb \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=jobportal \
  -v /Users/eazybytes/Desktop/jobportal-data:/var/lib/mysql \
  -d mysql
```

Breaking Down the Command

`docker run` - Starts a new container from an image.

`-p 3306:3306` (Port Mapping)

Maps MySQL's internal port (3306) → your machine's port (3306)
Allows tools like MySQL Workbench, IntelliJ, or your Spring Boot app to connect.
Think of it as opening a door from your machine to the container.

`--name jobportaldb`
Gives your container a custom name: jobportaldb Easier to manage
Use docker stop jobportaldb instead of writing container ID
Think of it as giving a nickname to your container.

Setup MySQL DB using Docker

eazy bytes

Environment Variables (-e)

`-e MYSQL_ROOT_PASSWORD=root` -> Sets MySQL's root password inside the container.
`-e MYSQL_DATABASE=jobportal` -> Automatically creates a database named jobportal when the container starts.
No manual SQL commands needed.

`-v /Users/eazybytes/Desktop/jobportal-data:/var/lib/mysql`

This is a volume — permanent storage.

Left side: folder on your laptop
Right side: MySQL data folder inside container

Why this matters:

Even if container is deleted → your database is safe
Data is stored on your machine, not inside the container
Think of it as attaching a USB drive to your container.

`-d` (Detached Mode)

Runs the container in the background.
Does not block your terminal
Container keeps running silently
You can continue typing other commands

`mysql` (Image Name)
Tells Docker to use the official MySQL image from Docker Hub.
If it isn't available locally, Docker will automatically download it.

What is Docker Compose?

eazy bytes

Docker Compose helps developers run multiple containers together — like a database, a backend app, and maybe a message broker — all defined in one file. Instead of typing long docker run commands, you define everything in a single file called compose.yml. Then just run:

```
docker compose up
docker compose down
```

and all your services start & stop together!

Setting Up MySQL with compose file

compose.yml

```
services:
  jobportal:
    image: mysql:latest
    container_name: jobportaldb
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: jobportal
    volumes:
      - /Users/eazybytes/Desktop/jobportal-data:/var/lib/mysql
```

Spring Boot + Docker Compose

eazy bytes

Spring Boot has built-in support for Docker Compose! You just need to add one dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-docker-compose</artifactId>
  <optional>true</optional>
</dependency>
```

What Happens Automatically ?

When this module is added, Spring Boot can:

Detect your compose.yml file inside the project root folder
Run docker compose up when your app starts
Auto-create connection beans for supported services (like MySQL)
Run docker compose stop when your app shuts down

If containers are already running — no problem! Spring Boot will just connect to them without restarting.

Spring Boot + Docker Compose

eazy bytes

Configuring Compose File & Lifecycle

Use a Custom Compose File

```
spring.docker.compose.file=../my-compose.yml
```

Control Lifecycle Behavior

```
spring.docker.compose.lifecycle-management=start-and-stop
spring.docker.compose.start.command=up
spring.docker.compose.stop.command=stop
```

This gives you full control over when and how Docker Compose runs.

In Simple Words

Docker Compose is like a universal power button for your app environment. Just write a small YAML file, and you can:

- Start MySQL, Redis, RabbitMQ, or any service
- Connect them automatically to your app
- Tear them down cleanly with one command

“No installation. No pain. Just one file and one command — and your environment is alive!”

Spring Boot: `spring.jpa.hibernate.ddl-auto`

eazy bytes

Based on the value assigned to property `spring.jpa.hibernate.ddl-auto` , the Hibernate decides what to do with your database schema

What Does ddl-auto Even Mean ?

Hibernate can auto-manage your DB schema (tables, columns, constraints) every time your app starts.
Depending on the value, it can create, update, validate, or delete your schema. Think of it like:

- Builder mode,
- Cleaner mode,
- Upgrader mode,
- Inspector mode,
- Hands-off mode.

Spring Boot: `spring.jpa.hibernate.ddl-auto`

eazy bytes

Common Values

create — Builder Mode (with Demolition!)

Meaning: Drops existing tables and creates new ones.
Analogy: “Break the old house and rebuild it fresh every time.”
Use Case: Early development, testing fresh schema.
⚠ All existing data is LOST.

create-drop — Temporary Builder
Meaning: Creates tables on startup, drops them on shutdown.
Analogy: “A tent set up for the day — gone when the session ends.”
Use Case: Unit tests, in-memory DBs.

create-only
Meaning: Only create schema. No dropping.
Analogy: “Build it once and walk away — never tear anything down.”
Use Case: Initial migrations or clean tables.

drop
Meaning: Drops all tables.
Analogy: “Level the entire building, don’t construct anything new.”
Use Case: Rare — mostly for tools or scripted cleanup.

Spring Boot: `spring.jpa.hibernate.ddl-auto`

eazy bytes

none — No Interference Mode
Meaning: Hibernate will NOT manage the schema.
Analogy: “Hands off! The DB admin is the boss.”
Use Case: Production (recommended)

truncate
Meaning: Deletes all rows but keeps table structure.
Analogy: “Clear the house but keep the foundation.”
Use Case: Resetting test data without dropping tables.

update — Auto-Fix Mode
Meaning: Updates schema if necessary (adds columns, tables).
Analogy: “Fixing small things in your house as needed.”
Use Case: Development only.
⚠ Not safe for production — can break schema or cause silent issues.

validate — Inspector Mode
Meaning: Checks if DB schema matches entities. No changes made.
Analogy: “Inspector visits your house, but can’t touch anything.”
Use Case: Staging/Prod

Spring Boot: `spring.jpa.hibernate.ddl-auto`

eazy bytes

Recommended Usage

Environment	Recommended Value
Local Dev	update, create, create-drop
Testing	create-dro, validate
Production	none or validate

Defaults to "create-drop" when using an embedded database and no schema manager was detected. Otherwise, defaults to "none".

JPA Buddy Plugin in IntelliJ IDEA

eazy bytes

What is JPA Buddy ?

- A powerful IntelliJ IDEA plugin for working with JPA, Hibernate, and Spring Data.
- Helps generate entities, repositories, DTOs, and database mappings.
- Provides easy tools to navigate, validate, and improve your JPA layer.
- Reduces boilerplate and eliminates common mistakes in JPA development.

Top Features Developers Love

- Entity Generation Wizard
- Relationship Builder (@OneToOne, @OneToMany, etc.)
- Repository/DAO Generator
- JSON → Entity conversion
- Database Schema Synchronization
- Attribute/Column Type Suggestions
- JPA Error Inspection (real-time warnings)

Installing JPA Buddy

- Go to File → Settings → Plugins
- Search for “JPA Buddy” & “Jakarta EE: JPA Model”
- Install both of them & Restart IntelliJ IDEA

`JetBrains premium version gives extra features compared to free version`

Global Exception Handling in Spring Boot

eazy bytes

In a Spring Boot application, when an exception occurs, it may cause an application crash or return an unclear error response to the client. Global Exception Handling ensures that errors are handled consistently and meaningful error messages are returned.

Why Use Global Exception Handling?

Provides consistent error responses across the application.
Improves readability and maintainability of exception handling logic.
Helps in logging and debugging by capturing errors in a central place.
Ensures better user experience with meaningful error messages.
No need to write try and catch blocks in all the methods

Ways to Handle Exceptions in Spring Boot

Spring Boot provides two main ways to handle exceptions globally:

Using @ExceptionHandler (Controller-Level Handling)
Using @RestControllerAdvice & ExceptionHandler(Global Exception Handling)

Global Exception Handling in Spring Boot

eazy bytes

This code defines a global exception handler using @RestControllerAdvice that catches all unhandled exceptions in the application.

When any exception occurs, it creates an ErrorResponseDto containing the request details, error message, status, and timestamp, and returns it with HTTP 500 – Internal Server Error.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleGlobalException(Exception exception,
                                                                   WebRequest webRequest) {
        ErrorResponseDto errorResponseDto = new ErrorResponseDto(
                webRequest.getDescription(false), HttpStatus.INTERNAL_SERVER_ERROR,
                exception.getMessage(), LocalDateTime.now());
        return new ResponseEntity<>(errorResponseDto, HttpStatus.INTERNAL_SERVER_ERROR);
    }

}
```

Validations in Spring Boot REST APIs

eazy bytes

Validation ensures that the data sent in API requests is correct before processing.

Why Use Validation?

Ensures correct and expected data input.
Prevents invalid data from reaching the database.
Reduces errors and improves API reliability.
Helps maintain clean and secure APIs.

To get started with validation, add the below dependency,

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Spring Boot provides two key annotations to trigger validation:

@Valid – Used on method custom datatype parameters to enable validation.
@Validated – Used on method primitive datatype parameters to enable validation.

Validations in Spring Boot REST APIs

eazy bytes

Spring Boot supports field-level validation using Jakarta Validation annotations inside DTOs and Entity classes

```
public record ContactRequestDto(

        @NotBlank(message = "Name cannot be empty")
        @Size(min = 5, max = 30, message = "Name must be between 5 and 30 characters")
        String name,

        @NotBlank(message = "Email cannot be empty")
        @Email(message = "Invalid email address")
        String email,

        @NotBlank(message = "UserType cannot be empty")
        @Pattern(regexp = "Job Seeker|Employer|Other",
                message = "UserType must be one of: Job Seeker, Employer, Other")
        String userType,

        @NotBlank(message = "Subject cannot be empty")
        @Size(min = 5, max = 255, message = "Subject must be between 5 and 255 characters")
        String subject,

        @NotBlank(message = "Message cannot be empty")
        @Size(min = 10, max = 1000, message = "Message must be between 10 and 1000 characters")
        String message

        ) {}
```

Validations in Spring Boot REST APIs

eazy bytes

To apply validation on RequestBody data, use @Valid inside a controller method.

```
@RestController
@RequestMapping("api/v1/contacts")
@RequiredArgsConstructor
public class ContactController {

    private final IContactService iContactService;

    @PostMapping
    public ResponseEntity<String> saveContact(@Valid @RequestBody ContactRequestDto contactRequestDto) {
        boolean isSaved = iContactService.saveContact(contactRequestDto);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body("Request processed successfully");
    }
}
```

Validations in Spring Boot REST APIs

eazy bytes

To apply validation on RequestParam, use below kind of logic,

```
@GetMapping
public ResponseEntity<String> fetchOpenContacts(
                  @RequestParam @Validated @NotBlank(message = "Status can not be blank") String status)
{
    return ResponseEntity.ok("These are the contacts with the given status: " + status);
}
```

Best Practices for Validations

- Use DTOs instead of entity classes for request validation.
- Always return meaningful error messages to the client.
- Centralize validation error handling using @RestControllerAdvice.

Validations in Spring Boot REST APIs

eazy bytes

Global Exception Handling for Validation Errors

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationExceptions(
            MethodArgumentNotValidException exception) {
        Map<String, String> errors = new HashMap<>();
        List<FieldError> fieldErrorList = exception.getBindingResult().getFieldErrors();
        fieldErrorList.forEach(error -> errors.put(error.getField(), error.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(HandlerMethodValidationException.class)
    public ResponseEntity<Map<String,String>> handleException(HandlerMethodValidationException exception) {
        Map<String, String> errors = new HashMap<>();
        List<ParameterValidationResult> results = exception.getParameterValidationResults();
        results.forEach(result -> {
            String paramName = result.getMethodParameter().getParameterName();
            String combinedMessages = result.getResolvableErrors().stream()
                    .map(error -> error.getDefaultMessage()).collect(Collectors.joining(", "));
            errors.put(paramName, combinedMessages);
            });
        return ResponseEntity.badRequest().body(errors);
    }
}
```

Auditing in Spring Boot with JPA

eazy bytes

Auditing in JPA helps track changes to entities by automatically recording who created or updated a record and when the changes occurred.

Why Use Auditing?

- Helps in maintaining logs of data modifications.
- Useful for tracking user actions in the database.
- Reduces manual effort by automatically capturing timestamps and user details.
- Ensures better data integrity and accountability.

Enable JPA Auditing

To enable auditing, we need to:

1. Create a BaseEntity class to handle auditing fields.
2. Use Spring Security or a custom implementation to capture the user who modified the data.
3. Enable Auditing in Spring Boot using @EnableJpaAuditing.

Auditing in Spring Boot with JPA

eazy bytes

Step 1: Create a BaseEntity Class

```
@Getter
@Setter
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    @CreationTimestamp
    private Instant createdAt;

    @CreatedBy
    @Column(name = "created_by", nullable = false, length = 20, updatable = false)
    private String createdBy;

    @LastModifiedDate
    @Column(name = "updated_at”, insertable = false)
    @UpdateTimestamp
    private Instant updatedAt;

    @LastModifiedBy
    @Column(name = "updated_by", length = 20, insertable = false)
    private String updatedBy;
}
```

Auditing in Spring Boot with JPA

eazy bytes

Step 2: Modify the Entity classes to extend BaseEntity

```
@Getter
@Setter
@Entity
@Table(name = "contacts")
public class Contact extends BaseEntity {

   // other fields

}
```

Step 3: Create an AuditorAware Implementation

We need to tell Spring who the current user is when saving/updating entities. If you use Spring Security, modify the getCurrentAuditor() method to return the authenticated username.

```
@Component("auditorAwareImpl")
public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        return Optional.of("Anonymous user");
    }
}
```

Auditing in Spring Boot with JPA

eazy bytes

Step 4: Enable Auditing in the Main Class

```
@SpringBootApplication
@EnableJpaAuditing(auditorAwareRef = "auditorAwareImpl")
public class EazystoreApplication {

  public static void main(String[] args) {
    SpringApplication.run(EazystoreApplication.class, args);
  }

}
```

Springdoc OpenAPI

eazy bytes

Documenting REST APIs is essential for making them understandable and usable by other developers.
springdoc-openapi is a Java library that automatically generates API documentation for your Spring Boot REST APIs. It follows the OpenAPI (Swagger) standard.

Why Document REST APIs?

Helps developers understand available endpoints and their request/response formats.
Reduces dependency on manual explanations.
Improves API usability and onboarding.
Makes integration with other systems easier.

Developers can further enhance the default API documentation using annotations like @Schema, @Tag, @Operation, @ApiResponse etc.

Springdoc OpenAPI

eazy bytes

To use the Springdoc library, Developer need to add the below dependency in pom.xml,

```
<dependency>
   <groupId>org.springdoc</groupId>
   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
   <version>2.8.14</version>
</dependency>
```

Once the dependency added, Start your Spring Boot application, and open:

Swagger UI:

`http://localhost:8080/swagger-ui.html`

OpenAPI JSON:

`http://localhost:8080/v3/api-docs`

Spring automatically generates the API documentation!

For more details, refer https://springdoc.org/

Bean scopes in Spring

eazy bytes

Bean scope defines how Spring creates and manages bean instances. It answers two key questions:

- How many instances of a bean will Spring create?
- When will Spring create these instances?

Spring provides six bean scopes:

01 **Singleton (Default)**

02 **Prototype**

03 **Request** (inside web apps)

04 **Session** (inside web apps)

05 **Application** (inside web apps)

06 **Websocket** (inside web apps)

Request Bean Scope

eazy bytes

Request Scope (@RequestScope)

What is it? - A new bean instance is created for each HTTP request.
When to use? - When a bean should be valid only for the duration of a request (e.g., storing request-specific data).
Use case - Storing temporary user inputs like form data

```
@Component
@RequestScope
@Getter @Setter
public class RequestScopedBean {

    private String username;

}
```

How it works?

- Each HTTP request gets a new instance of RequestScopedBean.
- If multiple users send requests, each gets a separate instance.
- As soon as the request is complete, the bean is destroyed.

Session Bean Scope

eazy bytes

Session Scope (@SessionScope)

What is it? - A new bean instance is created for each user session.
When to use? - When you need to store user-specific data across multiple requests in a single session.
Use case - Storing logged-in user details, shopping cart information.

```
@Component
@SessionScope
@Getter @Setter
public class SessionScopedBean {

    private String username;

}
```

How it works?

- When a user logs in, a SessionScopedBean is created.
- The same bean instance is used for multiple requests from the same user.
- When the session expires, the bean is destroyed.

Application Bean Scope

eazy bytes

Application Scope (@ApplicationScope)

What is it? - A single bean instance is shared across the entire application.
When to use? - When you need global data shared among all users and requests.
Use case - Storing global statistics like visitor count

```
@Component
@ApplicationScope
@Getter
public class ApplicationScopedBean {
    private int visitorCount = 0;

    public void incrementVisitorCount() {
        visitorCount++;
    }

}
```

How it works?

- Only one instance of ApplicationScopedBean is created.
- It is shared across all users and requests.
- The bean is destroyed only when the application stops.

Websocket Bean Scope

eazy bytes

Websocket Scope (@WebSocketScope)

WebSocket Scope = a bean that is created for each WebSocket connection and destroyed when that connection closes. A WebSocket is a continuous connection between the client (browser/app) and the server.
Unlike HTTP, which is request–response, WebSocket stays open and allows real-time, two-way communication.

Examples: Live chat, Stock price updates, Notifications, Multiplayer games

```
@Component
@WebSocketScope
public class ChatSessionData {

    private String username;
    private String currentRoom;

}
```

How it works?

- Each WebSocket client gets its own ChatSessionData object.
- They don’t interfere with each other.

Spring Bean Scopes - Request, Session, Application

eazy bytes

Using Scoped Beans in Controllers

```
@RestController @RequiredArgsConstructor
@RequestMapping("/api/v1/scope")
public class ScopeController {

    private final RequestScopedBean requestScopedBean;
    private final SessionScopedBean sessionScopedBean;
    private final ApplicationScopedBean applicationScopedBean;

    @GetMapping("/request")
    public String getRequestScope() {
        requestScopedBean.setUsername("JohnDoe");
        return "Request Scope: " + requestScopedBean. getUsername();
    }

    @GetMapping("/session")
    public String getSessionScope() {
        sessionScopedBean.setUsername("JohnDoe");
        return "Session Scope User: " + sessionScopedBean.getUsername();
    }

    @GetMapping("/application")
    public String getApplicationScope() {
        applicationScopedBean.incrementVisitorCount();
        return "Application Visitor Count: " + applicationScopedBean.getVisitorCount();
    }
}
```

Difference between @ApplicationScope and @Singleton scope

eazy bytes

Use @ApplicationScope when working with web applications to store global web-related state.
Use Singleton (default scope) for services, repositories, or shared business logic in all types of Spring applications.
Singleton beans are managed by the Spring container, whereas Application-scoped beans are managed by the Servlet context.

Feature	@ApplicationScope	@Singleton
Scope	One instance per Spring web application	One instance per Spring container (default scope)
Usage	Works only in web applications (Spring MVC, Spring Boot Web)	Works in all Spring applications (Web, CLI, Batch, etc.)
Annotation	@ApplicationScope	No annotation required (default), but can use @Scope("singleton")
Instance Sharing	Shared across the entire application for all users and requests	Shared across the entire Spring container
Bean Destruction	Destroyed when the web application stops	Destroyed when the Spring container shuts down
Use Case	Storing global application state (e.g., visitor count, app-wide config)	Storing singleton service beans (e.g., @Service, @Repository, @Component)

Understanding Relationships in Spring Data JPA

eazy bytes

In real-world apps, data is often connected (e.g., a customer has an address, a book has an author). JPA relationships help model these connections between entities (tables) in your database.

- JPA relationships define how one entity connects to another
- They help in maintaining data consistency and navigating between entities easily
- These relationships mimic foreign key relationships in databases
- With out relationships, Developers need to write complex join queries

Spring Data JPA supports four main types of relationships:

OneToOne: One entity is linked to exactly one other entity.

OneToMany: One entity is linked to multiple entities.

ManyToOne: Multiple entities are linked to one entity.

ManyToMany: Multiple entities are linked to multiple entities.

Each relationship has a specific use case and way to implement.

One-to-Many and Many-to-One Relationship in Spring Data JPA

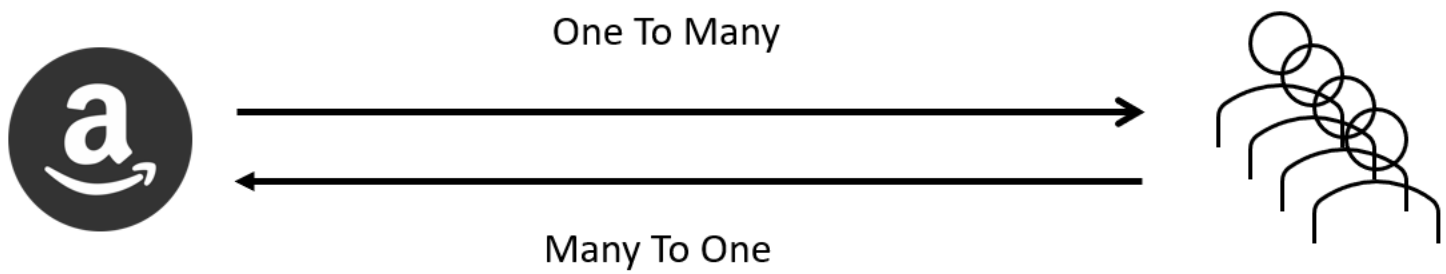
eazy bytes

Company & Jobs — Understanding One-to-Many and Many-to-One

Imagine a big company like Amazon. The company posts multiple job openings:

- Java Developer
- Tester
- DevOps Engineer
- Cloud Architect

So One Company → Many Jobs
Each Job belongs to only one Company
This is exactly what OneToMany and ManyToOne represent.



One-to-Many and Many-to-One Relationship in Spring Data JPA

eazy bytes

How the Database Looks

The company_id column in the Job table is the foreign key that creates the relationship.

Companies Table

ID	Name
1	Amazon

ID	Title	company_id
101	Java Dev	1
102	QA Automation	1
103	DevOps Eng	1
104	Cloud Architect	1

Why NOT @OneToOne Between Company and Job?

You might think:
"Every job belongs to one company → so it should be OneToOne." But that logic looks only at the job side, not both sides.
In JPA (and databases), a OneToOne means:
One company → only one job and one job → only one company
Both sides must have only one. But is that true in real life?

Key Principle to Remember
OneToOne means both sides have exactly one.
OneToMany means one has many, while the other has only one.

One-to-Many and Many-to-One Relationship in Spring Data JPA

eazy bytes

Entities involved:

Company.java

```
@Entity
@Table(name = ”companies")
public class Company extends BaseEntity {

    @OneToMany(mappedBy = "company", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Job> jobs = new ArrayList<>();
}
```

mappedBy = "company" means:
The Job entity contains a field named company, and that field owns the foreign key.

Cascade means: “If something happens to the parent (Company), do the same to its children (Jobs).”

Examples:

- Saving Company → Jobs are automatically saved
- Updating Company → Jobs are updated
- Deleting Company → All Jobs are deleted

This makes life easier because you don’t need to save/delete jobs manually.

One-to-Many and Many-to-One Relationship in Spring Data JPA

eazy bytes

orphanRemoval = true

This is like saying: “If a job is removed from the jobs list, delete it from the database also.”

Example: `company.getJobs().remove(job);`

That removed job becomes an “orphan” and Hibernate will delete it from the jobs table.

Without orphanRemoval: The job stays in the database even if not linked to any company.
With orphanRemoval = true:The job is automatically removed → clean database, no garbage data.

private List<Job> jobs = new ArrayList<>();

This simply initializes the list so that:

- Company always has an empty list instead of null
- You can add jobs easily:

```
company.getJobs().add(job);
```

One-to-Many and Many-to-One Relationship in Spring Data JPA

eazy bytes

Job.java

```
@Entity
@Table(name = "jobs")
public class Job extends BaseEntity {

    @NotNull
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JoinColumn(name = "company_id", nullable = false)
    private Company company;
}
```

@NotNull

Meaning → Every job must belong to a company. No job can exist without a company.
If someone tries to save a Job with company = null → Result into an error.

@ManyToOne(fetch = FetchType.LAZY, optional = false)

Why ManyToOne? Because: Many Jobs → belong to One Company
A company can have many job posts. But each job is linked to exactly one company only
Hence, Many-to-One.

One-to-Many and Many-to-One Relationship in Spring Data JPA

eazy bytes

optional = false
Meaning: The Company relationship is mandatory. Every Job must have a Company. This reinforces @NotNull.

fetch = FetchType.LAZY
This improves performance. Lazy loading means: “When you load a job, DO NOT load the company immediately.”
Only when you call: `job.getCompany()` THEN Hibernate will fetch the company.

Why is this good?
Because: You usually need job details first. You don't always need the company details. Saves memory and reduces queries

@JoinColumn(name = "company_id", nullable = false)
This tells Hibernate: “There is a column named company_id in the jobs table which stores company’s ID”.
This is the foreign key.

@OnDelete(action = OnDeleteAction.CASCADE)
This is very powerful. It tells the database (not just Hibernate):“If a company is deleted, automatically delete all its jobs.”
This ensures referential integrity.

Example:
If Company 1 is deleted → All job rows with company_id = 1 are automatically deleted by the database.
No orphan jobs. No mismatch errors.

What is a Bidirectional Relationship ?

eazy bytes

In JPA, a bidirectional relationship means:

- Both entities (Company and Job) know about each other
- You can navigate the relationship in both directions

That means:

- From Company, you can get all its Jobs (`company.getJobs()`)
- From Job, you can get the Company it belongs to (`job.getCompany()`)
- It's like two friends who both have each other's phone numbers
  — they can call each other anytime.

In Our Case:

Company → has many jobs (OneToMany side)
Job → belongs to one company (ManyToOne side)

Because both sides reference each other, the relationship becomes bidirectional.

Who Owns the Relationship?

eazy bytes

From JPA perspective:

Job is the owner - Because it has the foreign key column company_id.

Company is the "inverse" side

- mappedBy = "company" tells JPA:
- “Use the foreign key in the Job table — I don’t have one.”

This prevents duplicate join tables and keeps the database clean.

Understanding FETCH Types in JPA

eazy bytes

What Are Fetch Types in JPA?

Fetch Types define when related entities are loaded from the database. When you define relationships like @OneToOne, @OneToMany, @ManyToOne, @ManyToMant, JPA needs to know:

```
Should the related data be loaded immediately when the parent is
loaded?

Or should it be loaded later, only when needed?
```

That's where fetch = FetchType.LAZY or fetch = FetchType.EAGER comes in.

Fetch Type	Meaning	Behavior
EAGER	Fetch immediately	Loads the related entity right away
LAZY	Fetch when accessed (on demand)	Loads the related entity only when used

Understanding FETCH Types in JPA

eazy bytes

Think of It Like Ordering Food

EAGER = You order the main dish + side dish at the same time
→ Even if you're not hungry for the side, it still comes

LAZY = You order only the main dish
→ You ask for the side dish only if/when you want it

Default Fetch Types

Relationship Annotation	Default Fetch Type
@OneToOne	EAGER
@ManyToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

Understanding Cascade Types in JPA

eazy bytes

What Are Cascade Types in JPA?

Cascade in JPA means:

"If I do something to the parent entity, automatically do it to the child entity too."
Think of it like a chain reaction — changes made to one entity get automatically applied to related entities.

Real-World Analogy

Imagine a Company and it's Jobs:

If a Company is saved, it's Jobs should be saved too.
If the Company is deleted, it's Jobs should be deleted too.
That's cascading: you don't need to save or delete related objects separately.

Why Do We Use Cascade?

- To reduce boilerplate code
- To avoid manually saving/deleting related entities
- To keep entities in sync automatically

Understanding Cascade Types in JPA

eazy bytes

Available Cascade Types

Cascade Type	What it does
PERSIST	Saves the child entity when the parent is saved
MERGE	Updates the child entity when the parent is updated
REMOVE	Deletes the child entity when the parent is deleted
REFRESH	Refreshes the child when parent is refreshed from DB
DETACH	Detaches the child from the persistence context when parent is detached
ALL	Applies all the above actions

```
Company company = new Company();
Job job1 = new Job();
Job job2 = new Job();
company.getJobs().add(job1);
company.getJobs().add(job2);

// This saves company AND both jobs automatically
companyRepository.save(company);
```

Because of cascade configurations:

When a company is saved, the associated jobs also will be saved

Understanding @OnDelete(action = OnDeleteAction.CASCADE)

eazy bytes

What is @OnDelete(action = OnDeleteAction.CASCADE)?

@OnDelete is an annotation in Hibernate/JPA (Java).
Purpose: Tells the database to automatically delete related records when a parent record is deleted.
Uses ON DELETE CASCADE at the database level (e.g., in SQL).

Code Example

```
@Entity
@Table(name = "jobs")
public class Job extends BaseEntity {

    @NotNull
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JoinColumn(name = "company_id", nullable = false)
    private Company company;
}
```

If a Company is deleted, the associated job records are automatically deleted by the database.

Understanding @OnDelete(action = OnDeleteAction.CASCADE)

eazy bytes

The enum org.hibernate.annotations.OnDeleteAction defines the strategy Hibernate should apply when a parent entity is deleted, in relation to its child/associated entities — directly in the database (not through Java).

OnDeleteAction Values

Enum Value	Description
NO_ACTION	Default behavior. If a parent is deleted while children still exist, the database raises a foreign key constraint violation (maybe later in transaction).
CASCADE	Automatically deletes the child rows when the parent row is deleted (via ON DELETE CASCADE at DB level).
RESTRICT	Immediately blocks the deletion of the parent if any child exists (stricter than NO_ACTION).
SET_NULL	Sets the child's foreign key to NULL when the parent is deleted (child remains, but unlinked).
SET_DEFAULT	Sets the child's foreign key to its default value (if any) when the parent is deleted.

@OnDelete vs. CascadeType.REMOVE

eazy bytes

Feature	@OnDelete	CascadeType.REMOVE
Where it runs	Database level (SQL foreign key)	Application level (Java/Hibernate)
How it works	Database deletes child records automatically	Hibernate deletes child records in code
Performance	Faster (database handles it)	Slower (Hibernate queries each record)
Code example	@OnDelete(action = OnDeleteAction.CASCADE)	cascade = CascadeType.REMOVE
When to use	When you want database to manage deletions	When you need app-level control

Key Difference: @OnDelete is a database rule; CascadeType is a Hibernate instruction.

Spring Security Introduction

eazy bytes

Spring Security is a powerful framework that helps you protect your Spring applications.

- Handles authentication (who are you?) and authorization (what can you access?)
- Works seamlessly with Spring Boot
- Can protect both web and REST APIs
- Highly customizable

Why Use Spring Security?

- Protects Against common attacks like CSRF, CORs, Session Fixation
- Easy Login Systems: Adds username/password checks quickly.
- Customizable: You can tweak it to fit your needs
- Easily integrate with Databases, LDAP, OAuth2 / JWT / SAML

Default Behavior (Zero Configuration) of Spring Security

eazy bytes

When you add this to your pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

What happens automatically?

- A login form is shown
- A default user is created
- Console logs show the default password
- All URLs are secured by default

Default Credentials in Spring Security & Custom Configuration

eazy bytes

By default, Spring Security provides the following credentials,

- Default Username -> user
- Default password -> Randomly generated in console at startup

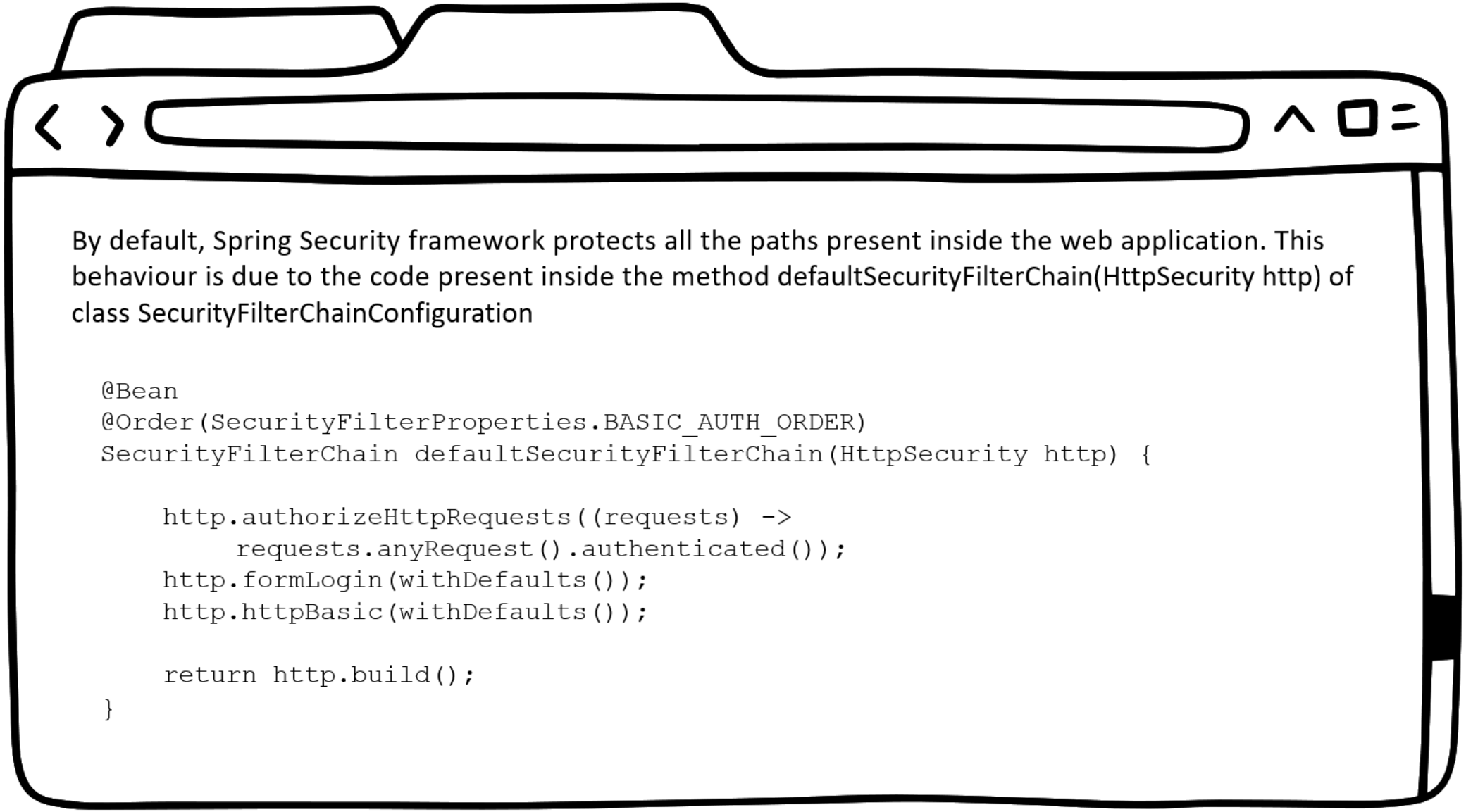
Password changes every time application restarts
Meant only for development / testing

We can disable the generated password and define our own using the following properties:

```
spring.security.user.name=madan
spring.security.user.password=Madan@123
spring.security.user.roles=USER,ADMIN
```

Default Spring Security configurations

eazy bytes



By default, Spring Security framework protects all the paths present inside the web application. This behaviour is due to the code present inside the method defaultSecurityFilterChain(HttpSecurity http) of class SecurityFilterChainConfiguration

```
@Bean
@Order(SecurityFilterProperties.BASIC_AUTH_ORDER)
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) {

    http.authorizeHttpRequests((requests) ->
         requests.anyRequest().authenticated());
    http.formLogin(withDefaults());
    http.httpBasic(withDefaults());

    return http.build();
}
```

Disable Form Login

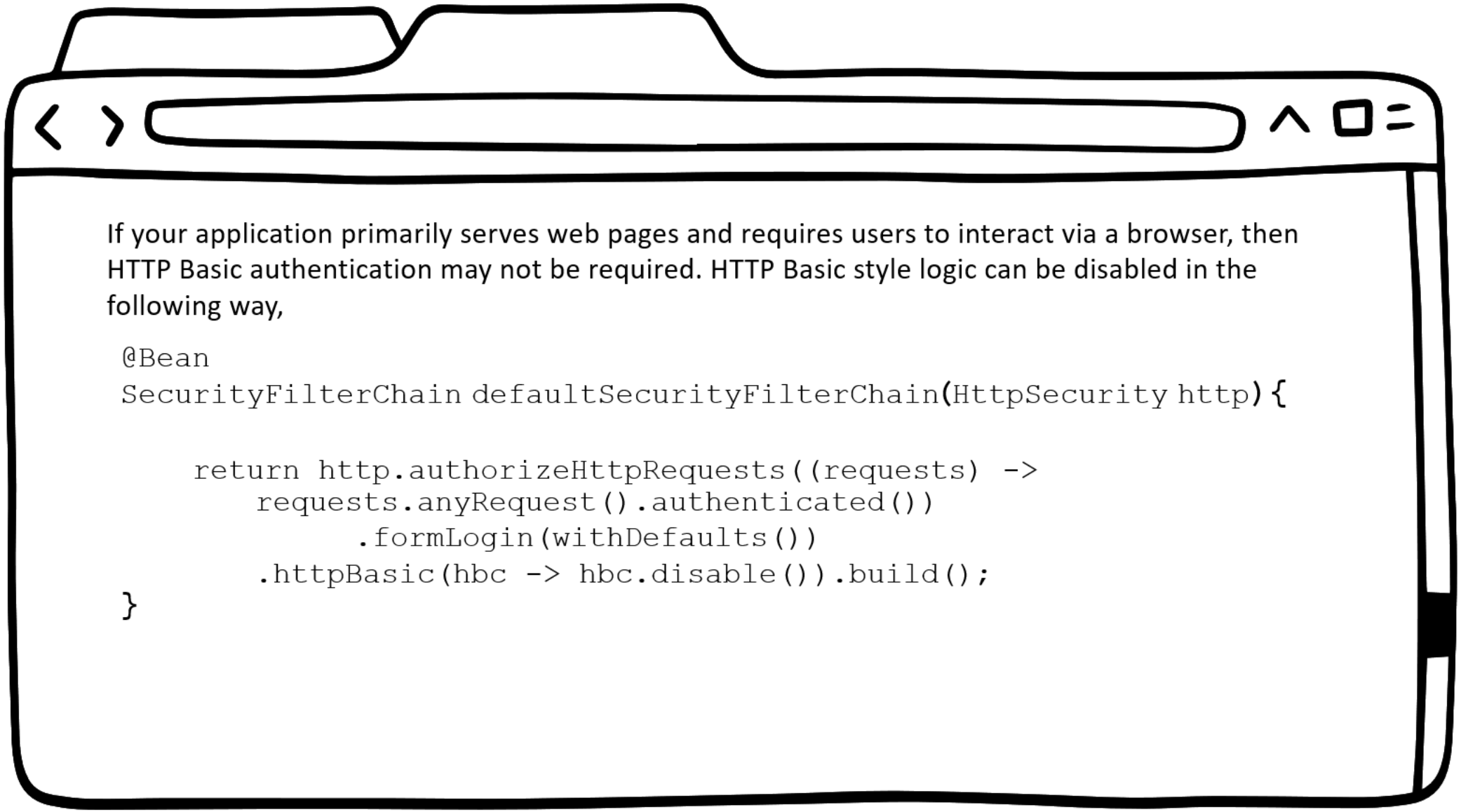
eazy bytes

If your Spring Security-protected endpoints are only accessed programmatically (using APIs) by clients (such as mobile apps or other services) and there's no need for a web-based login form, disabling form login can be appropriate. This can be done in the following way,

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http){

    return http.authorizeHttpRequests((requests) ->
       requests.anyRequest().authenticated())
            .formLogin(flc -> flc.disable())
       .httpBasic(withDefaults()).build();
}
```

Disable HTTP Basic Login

eazy bytes

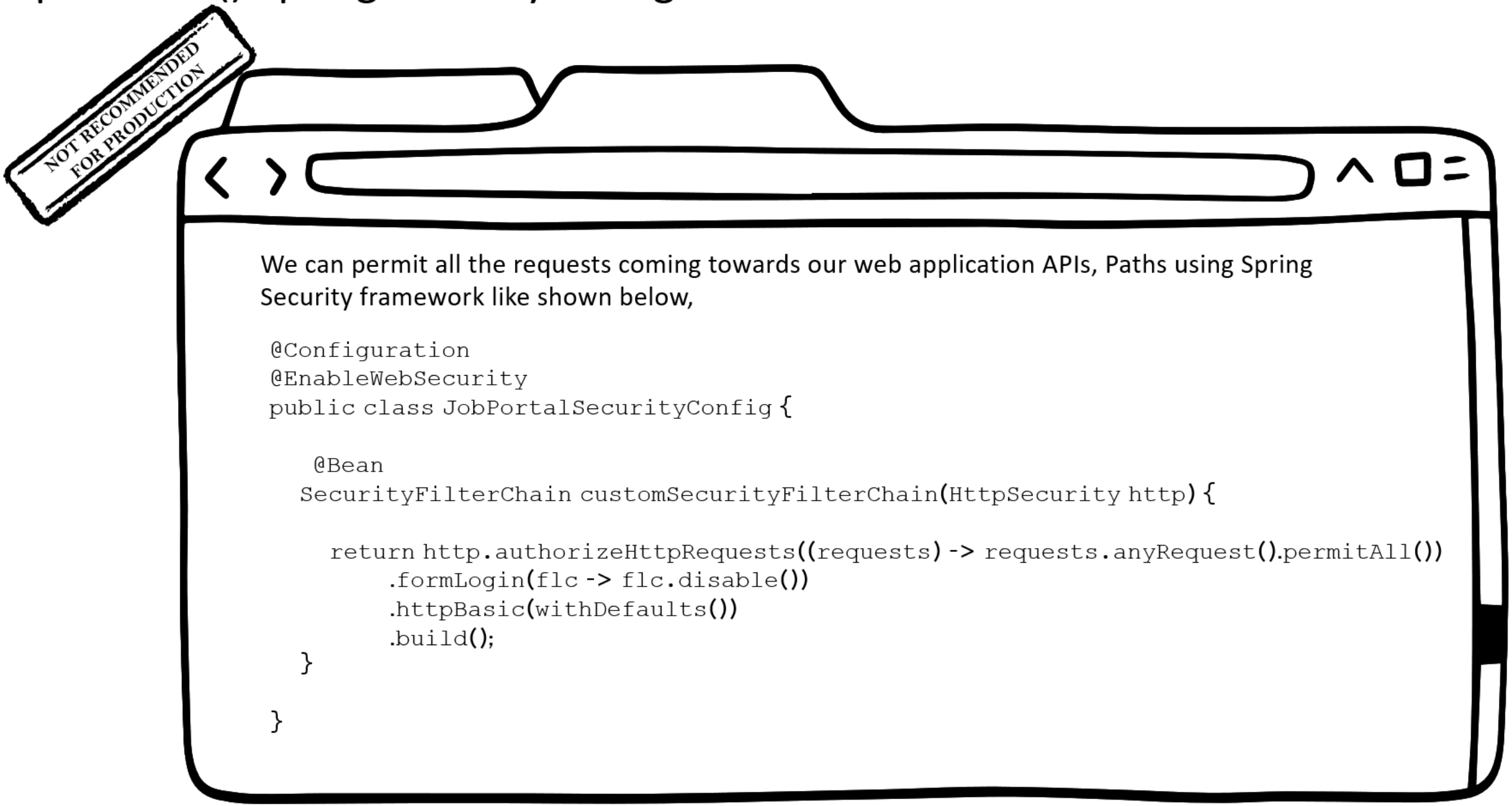


If your application primarily serves web pages and requires users to interact via a browser, then HTTP Basic authentication may not be required. HTTP Basic style logic can be disabled in the following way,

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http){

    return http.authorizeHttpRequests((requests) ->
        requests.anyRequest().authenticated())
            .formLogin(withDefaults())
        .httpBasic(hbc -> hbc.disable()).build();
}
```

permitAll() Spring Security configurations

eazy bytes



We can permit all the requests coming towards our web application APIs, Paths using Spring Security framework like shown below,

```
@Configuration
@EnableWebSecurity
public class JobPortalSecurityConfig {

    @Bean
   SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

     return http.authorizeHttpRequests((requests) -> requests.anyRequest().permitAll())
          .formLogin(flc -> flc.disable())
          .httpBasic(withDefaults())
          .build();
   }

}
```

denyAll() Spring Security configurations

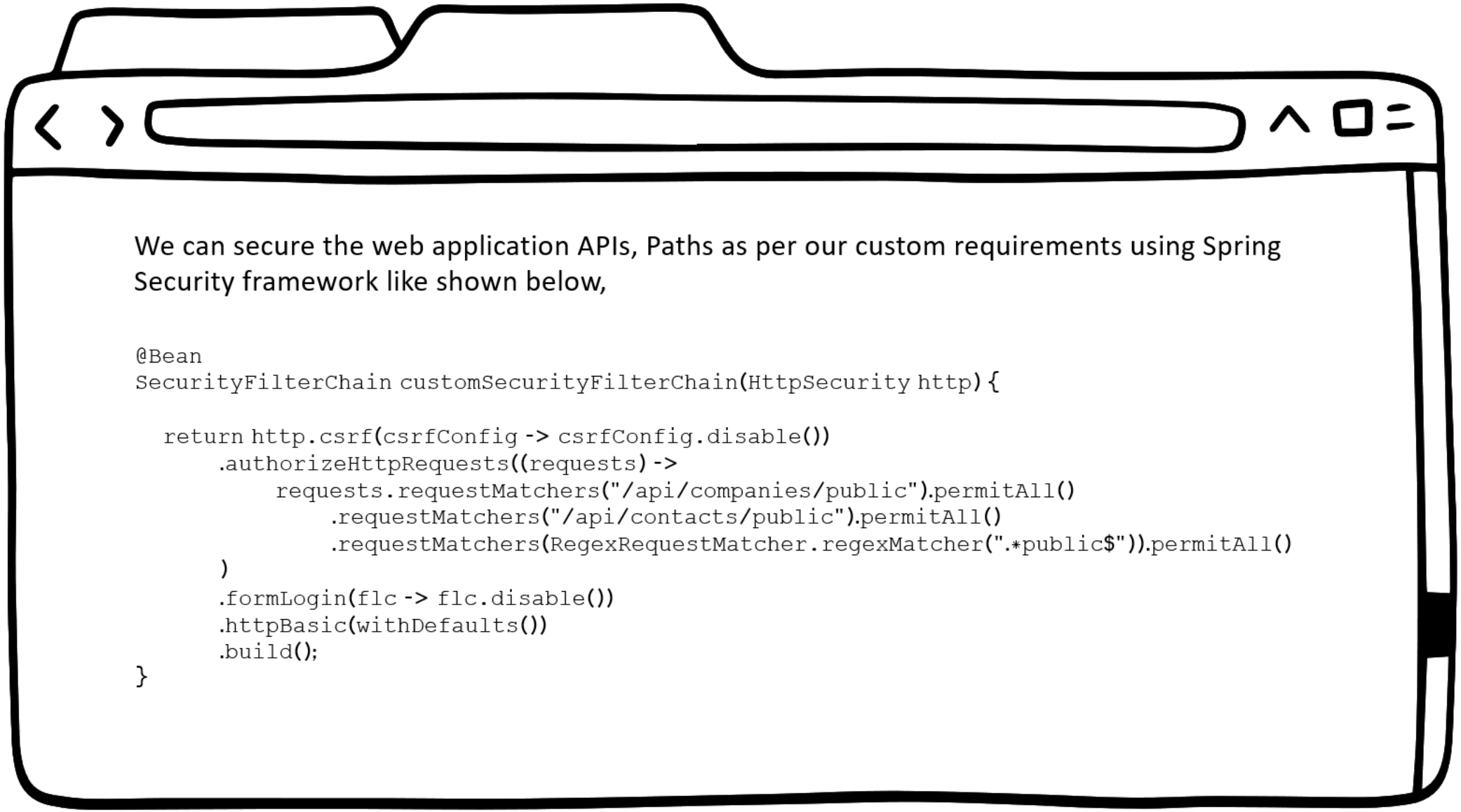
eazy bytes

We can deny all the requests coming towards our web application APIs, Paths using Spring Security framework like shown below,

```
@Configuration
@EnableWebSecurity
public class JobPortalSecurityConfig {

    @Bean
  SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

    return http.authorizeHttpRequests((requests) -> requests.anyRequest().denyAll())
        .formLogin(flc -> flc.disable())
        .httpBasic(withDefaults())
        .build();
  }

}
```

Custom Spring Security configurations

eazy bytes

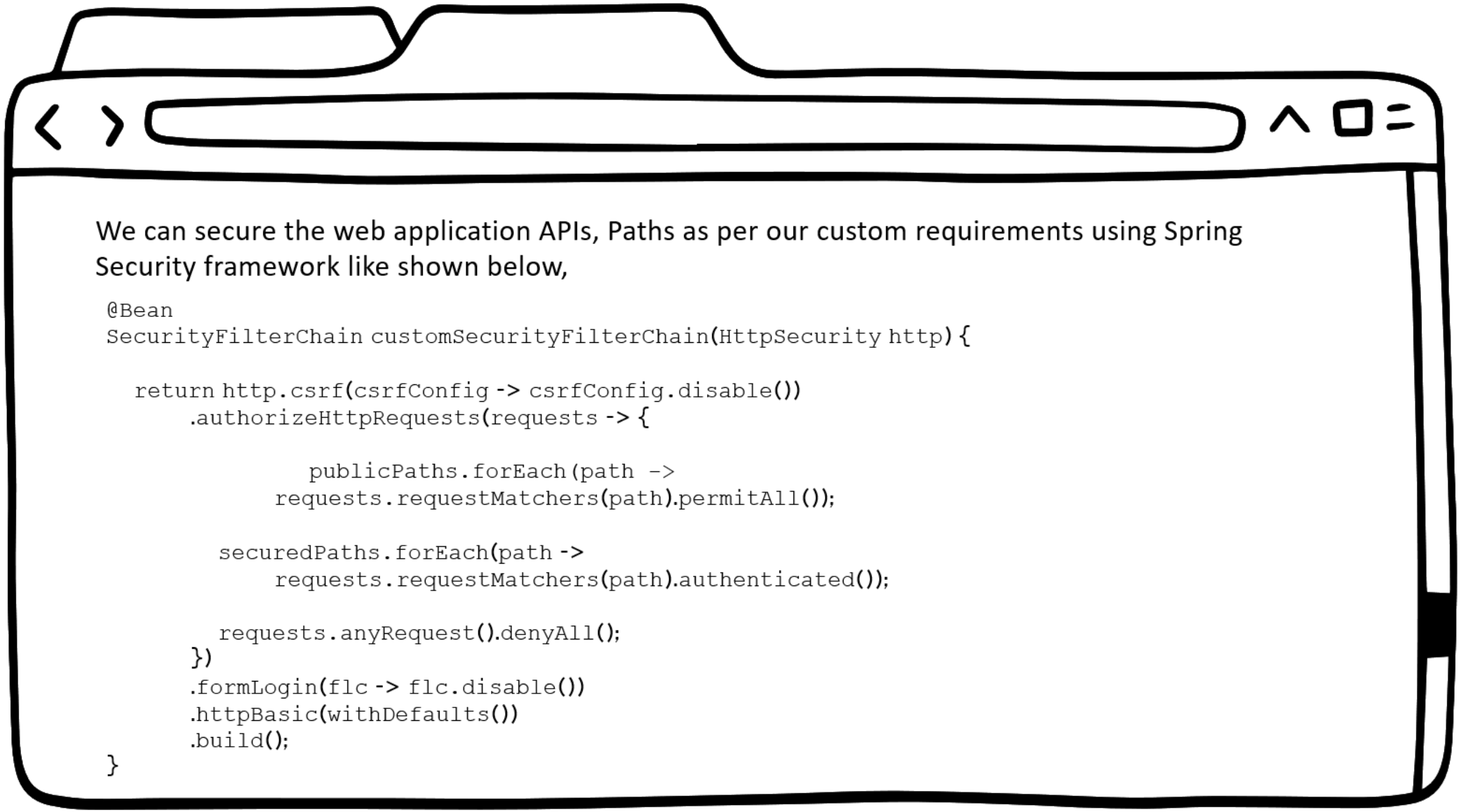


We can secure the web application APIs, Paths as per our custom requirements using Spring Security framework like shown below,

```
@Bean
SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

  return http.csrf(csrfConfig -> csrfConfig.disable())
      .authorizeHttpRequests((requests) ->
          requests.requestMatchers("/api/companies/public").permitAll()
              .requestMatchers("/api/contacts/public").permitAll()
              .requestMatchers(RegexRequestMatcher.regexMatcher(".*public$")).permitAll()
      )
      .formLogin(flc -> flc.disable())
      .httpBasic(withDefaults())
      .build();
}
```

Custom Spring Security configurations

eazy bytes

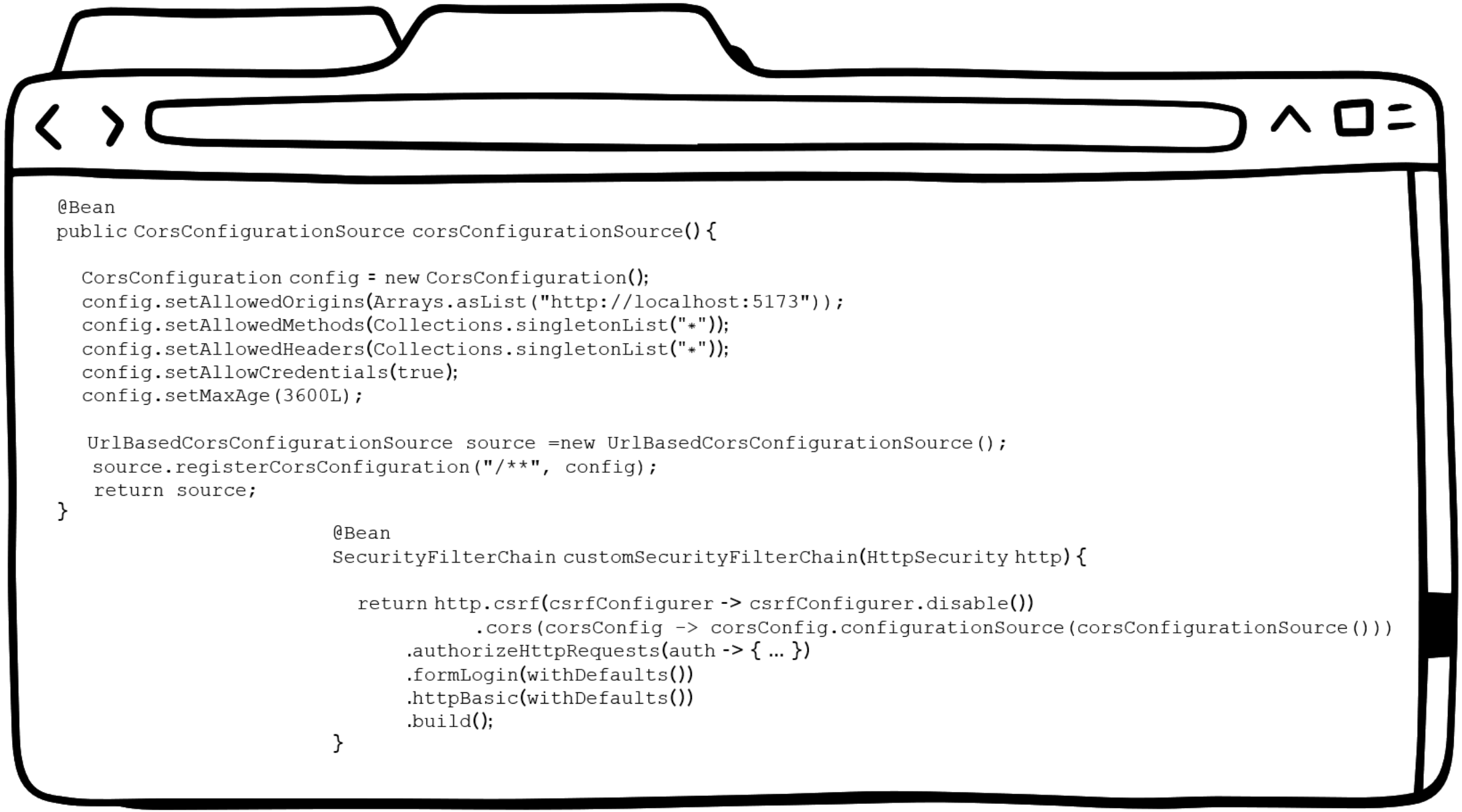


We can secure the web application APIs, Paths as per our custom requirements using Spring Security framework like shown below,

```
@Bean
SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

  return http.csrf(csrfConfig -> csrfConfig.disable())
      .authorizeHttpRequests(requests -> {

              publicPaths.forEach(path ->
            requests.requestMatchers(path).permitAll());

        securedPaths.forEach(path ->
            requests.requestMatchers(path).authenticated());

        requests.anyRequest().denyAll();
      })
      .formLogin(flc -> flc.disable())
      .httpBasic(withDefaults())
      .build();
}
```

Spring Security CORs configurations

eazy bytes

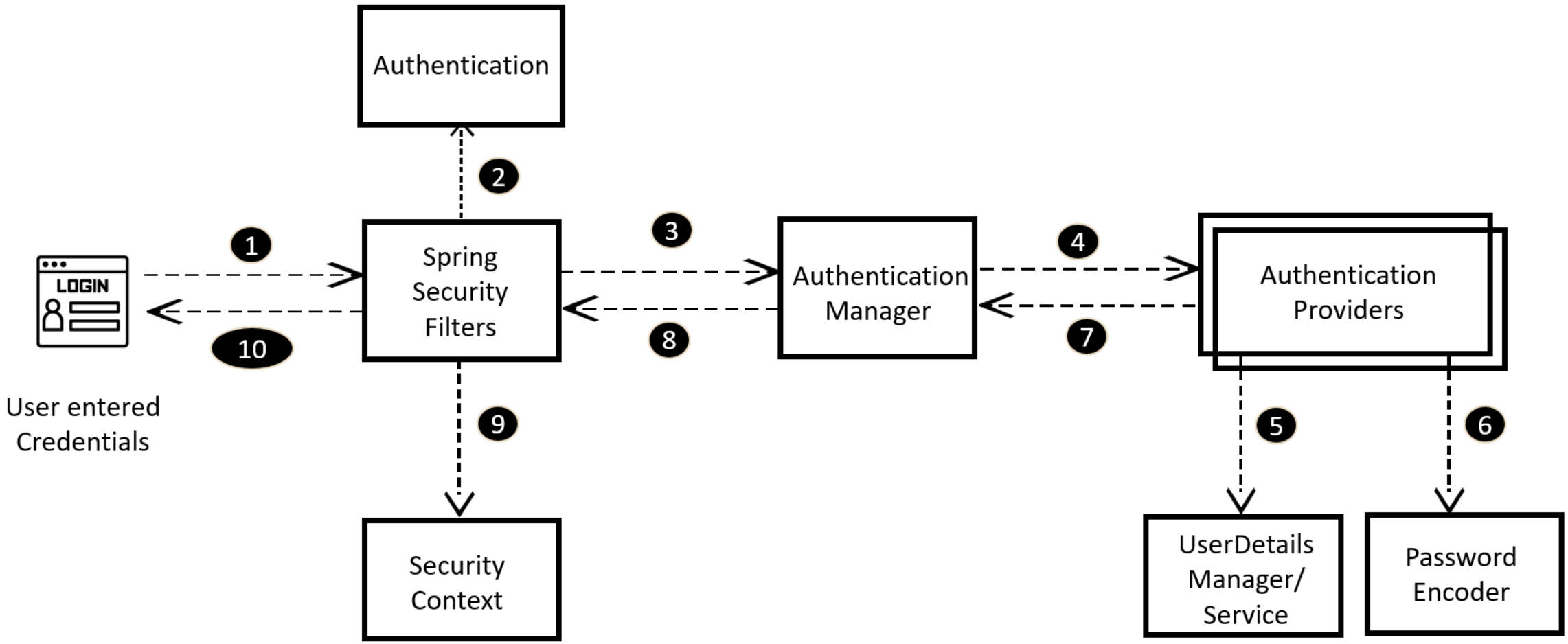


```
@Bean
public CorsConfigurationSource corsConfigurationSource() {

  CorsConfiguration config = new CorsConfiguration();
  config.setAllowedOrigins(Arrays.asList("http://localhost:5173"));
  config.setAllowedMethods(Collections.singletonList("*"));
  config.setAllowedHeaders(Collections.singletonList("*"));
  config.setAllowCredentials(true);
  config.setMaxAge(3600L);

   UrlBasedCorsConfigurationSource source =new UrlBasedCorsConfigurationSource();
   source.registerCorsConfiguration("/**", config);
   return source;
}
```

```
@Bean
SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

  return http.csrf(csrfConfigurer -> csrfConfigurer.disable())
            .cors(corsConfig -> corsConfig.configurationSource(corsConfigurationSource()))
      .authorizeHttpRequests(auth -> { ... })
      .formLogin(withDefaults())
      .httpBasic(withDefaults())
      .build();
}
```

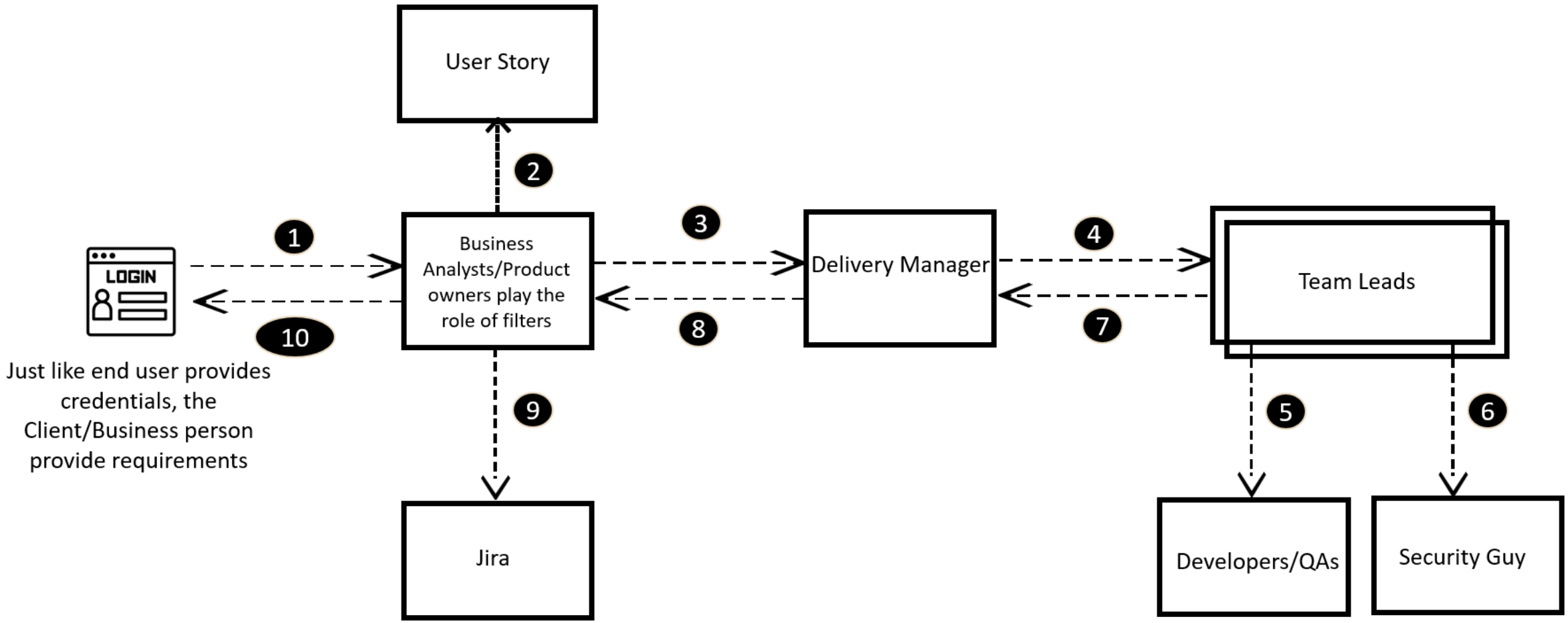
Spring Security Internal flow

eazy bytes



Analogy of Spring Security Internal flow with SDLC

eazy bytes



Spring Security Internal flow

eazy bytes

Spring Security Filters

A series of Spring Security filters intercept each request & work together to identify if Authentication is required or not. If authentication is required, accordingly navigate the user to login page or use the existing details stored during initial authentication.

Authentication

Filters like UsernamePasswordAuthenticationFilter will extract username/password from HTTP request & prepare Authentication type object. Because Authentication is the core standard of storing authenticated user details insdie Spring Security framework.

AuthenticationManager

Once received request from filter, it delegates the validating of the user details to the authentication providers available. Since there can be multiple providers inside an app, it is the responsibility of the AuthenticationManager to manage all the authentication providers available. In simple words, the authentication manager takes the responsibility for authentication.

AuthenticationProvider

AuthenticationProviders has all the core logic of validating user details for authentication.

UserDetailsManager/UserDetailsService

UserDetailsManager/UserDetailsService helps in retrieving, creating, updating, deleting the User Details from the DB/storage systems.

PasswordEncoder

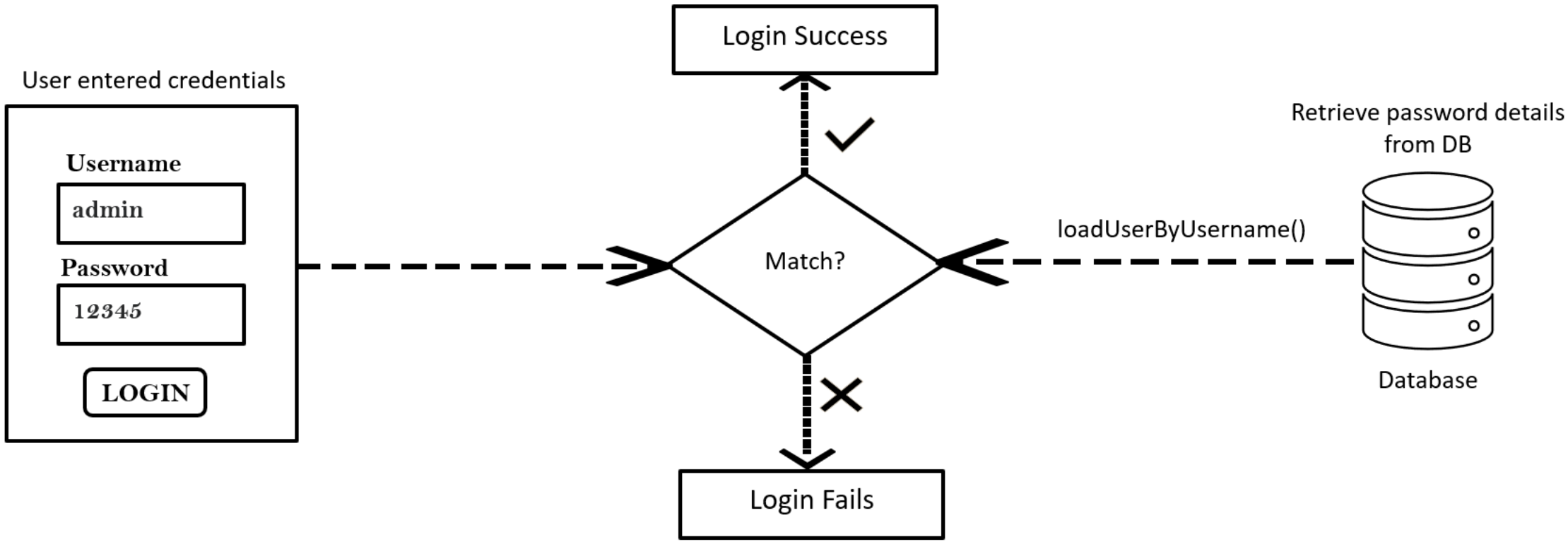
Service interface that helps in encoding & hashing passwords. Otherwise we may have to live with plain text passwords

SecurityContext

Once the request has been authenticated, the Authentication will usually be stored in a thread-local SecurityContext managed by the SecurityContextHolder. This helps during the upcoming requests from the same user.

How passwords validated with plain passwords

eazy bytes



Storing the passwords in a plain text inside a storage system like DB will have Integrity & Confidentiality issues. So this is not a recommended approach for Production applications.

eazy bytes

Different ways for Data privacy

Encoding

- ✓ Encoding is defined as the process of converting data from one form to another and has nothing to do with cryptography.
- ✓ It involves no secret and completely reversible.
- ✓ Encoding can't be used for securing data. Below are the various publicly available algorithms used for encoding.

Ex: ASCII, BASE64, UNICODE

Encryption

- ✓ Encryption is defined as the process of transforming data in such a way that guarantees confidentiality.
- ✓ To achieve confidentiality, encryption requires the use of a secret which, in cryptographic terms, we call a "key".
- ✓ Encryption can be reversible by using decryption with the help of the "key". As long as the "key" is confidential, encryption can be considered as secured.

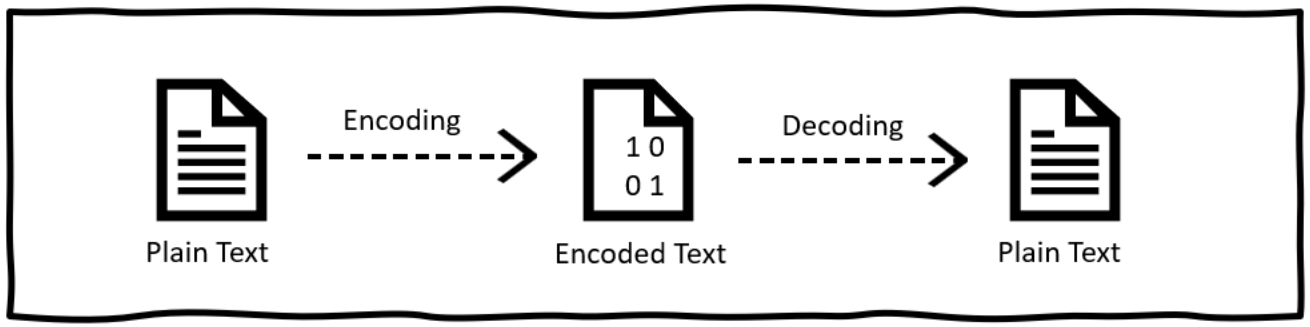
Hashing

- ✓ In hashing, data is converted to the hash value using some hashing function.
- ✓ Data once hashed is non-reversible. One cannot determine the original data from a hash value generated.
- ✓ Given some arbitrary data along with the output of a hashing algorithm, one can verify whether this data matches the original input data without needing to see the original data

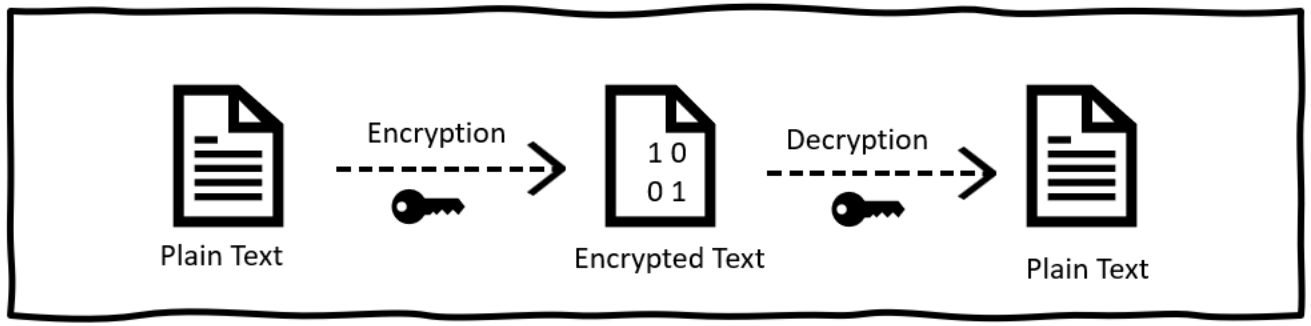
Encoding Vs Encryption Vs Hashing

eazy bytes

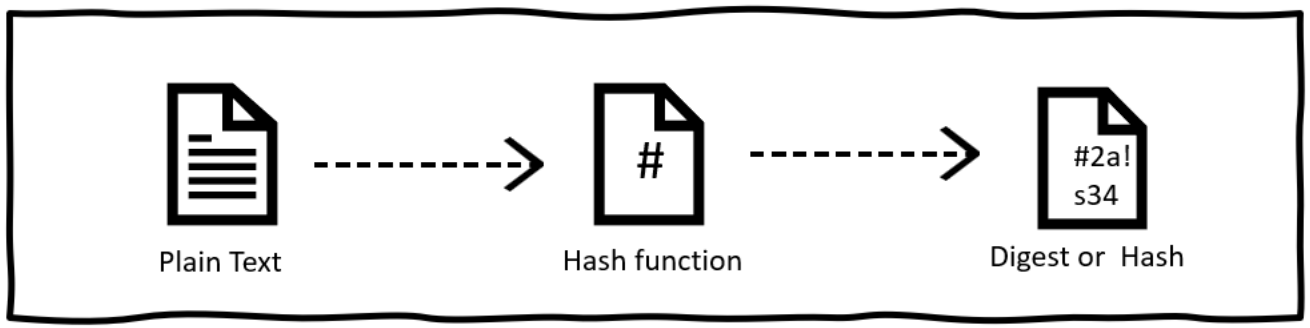
Encoding & Decoding



Encryption & Decryption



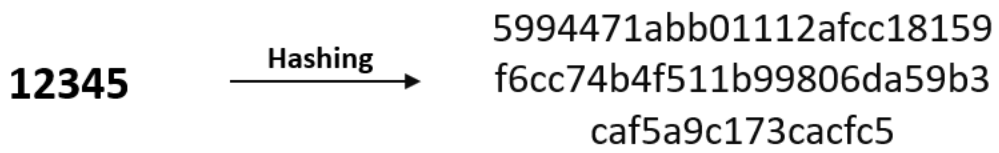
Hashing (Only 1-way)



Deep dive on Hashing

eazy bytes

01 Hash functions take any data as input and produce a unique string of bytes in return. Given the same input, the hash function always reproduces the same string of bytes. The output of a hash function is often called a digest or a HASH.



A hash function transforms input of any length (such as a file, message, or video) into a consistent, fixed-length output (e.g., 256 bits for SHA-256). When the same input is hashed, it always generates the same resulting digest or hash.

---

02 A fundamental characteristic of a hash function is its irreversibility, meaning that it should not be possible to deduce the input from its output alone. This property is often referred to as being "one-way."

In other words, given the output produced by a hash function (depicted as a blender), it should be computationally infeasible, or practically impossible, to reverse the process and uncover the original input.

Hashing passwords

eazy bytes

The typical approach for storing passwords within an organization involves hashing user passwords and retaining only the resulting digests. When a user attempts to log in to your website, the process typically follows these steps:

1. You receive the user's password during the registration process.
2. You hash the provided password and discard the original. Store it in a storage system like Database
3. During login operation, you compare the resulting digest with the stored version; if they match, the user gains access.

While this may look like a perfect solution as no one can get to know the plain plassword from the hash or digest value even if they manage to steal the user data from DB. It has some loop holes. Let's imagine few users they used simple passwords and there is also a good chance that multiple user choosed to have the same password like 123456 or password etc. Below is how, it is going to look,

User ID	Username	Plain password	Hash password
1	johndoe	12345	5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5
2	mike123	password	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
3	pavan189	12345	5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5
4	saanvi180	password	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8

Hashing passwords

eazy bytes

There will be couple of problems if we use simple hashing mechanisms for passwords management.

1. If an attacker gains access to hashed passwords in a DB, they could potentially launch a brute force attack or an exhaustive search (on Rainbow table) by trying all possible passwords. This method tests each attempt against the entire database of hashed passwords. Ideally, we aim to limit attackers to targeting only one hashed password at a time.

2. Hash functions are designed to be fast, which attackers can exploit to try numerous passwords per second during a brute force attack. Ideally, we should implement mechanisms to impede such attacks, slowing down the process significantly.

What is a Brute force attack ?

A brute force attack is a trial-and-error method used by hackers to decode encrypted data, such as passwords or encryption keys. In this type of attack, the attacker systematically tries all possible combinations of characters until the correct one is found. It's essentially like trying every possible key in a lock until the right one opens it.

What is a Rainbow table or Dictionary table attack ?

A rainbow table or Dictionary table attacks are a type of cryptographic attack used to crack password hashes. It involves precomputing and storing a large number of password hashes along with their corresponding plaintext passwords in a table known as a "rainbow table" or a "dictionary table"

When a hashed password is obtained, instead of computing hashes for all possible passwords as in a brute force attack, the attacker can simply look up the hash in the rainbow or dictionary table to find the corresponding plaintext password. This significantly reduces the time and computational resources required to crack the password.

Hashing passwords

eazy bytes

Let’s try to understand how to overcome the challenges of Brute force attacks and Rainbow table attacks,

To prevent rainbow table attacks, salts are commonly used. Salts are random values that are unique for each user and are made public. The salt value is stored as part of the resulting hash itself. Specifically, the salt is included in the hash string along with the actual hashed password.

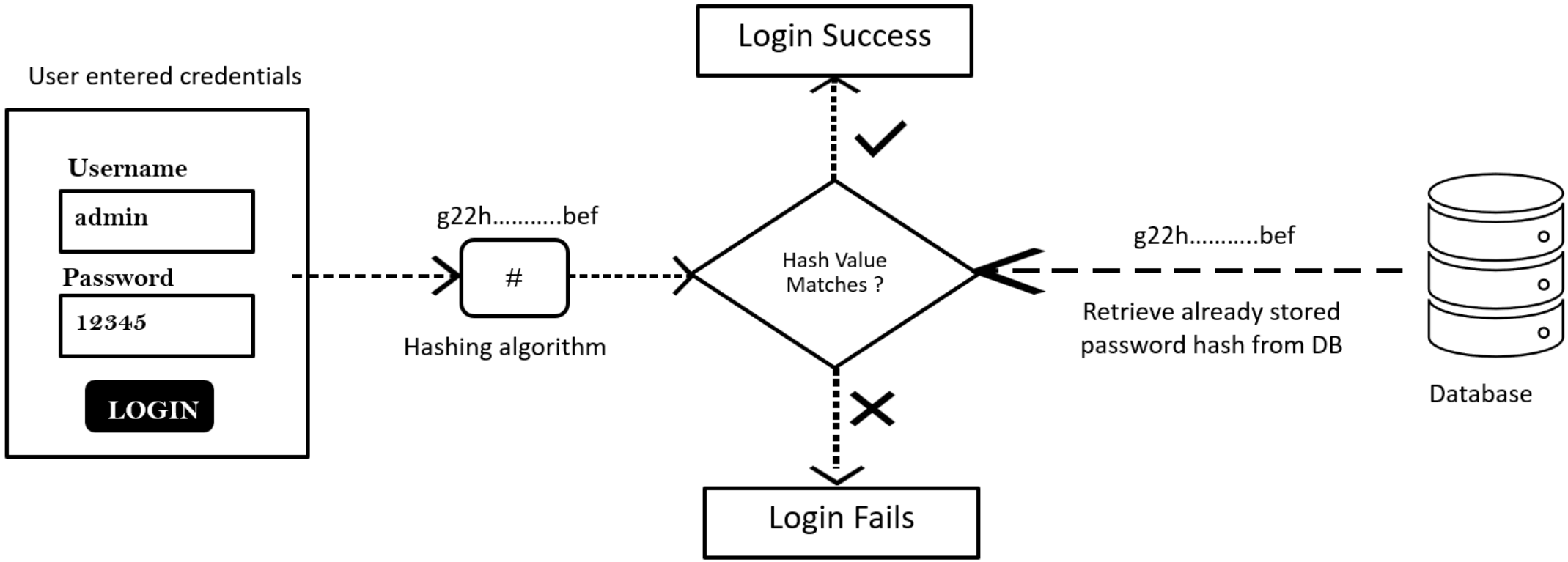
The salt is typically stored at the beginning of the hash string and is used during the password verification process to generate the same hash again for comparison. Since each user has a unique hash function, attackers can't precompute large tables of passwords (rainbow tables) to test against the entire database of stolen password hashes.

Plain text password + Random salt value + Hashing Algo = Strong Hash Value

Spring Security provides industry recommended Password Encoders that are capable of generating random salt and leverage password hashing algorithms like bcrypt

How passwords validated with hashed passwords

eazy bytes



Storing & managing the passwords with hashing is the recommended approach for Production applications. With various PasswordEncoders available inside Spring Security, it makes our life easy.

Details of PasswordEncoder

eazy bytes

Methods inside PasswordEncoder Interface

```
public interface PasswordEncoder {

	String encode(CharSequence rawPassword);

	boolean matches(CharSequence rawPassword, String encodedPassword);

	default boolean upgradeEncoding(String encodedPassword) {
		return false;
	}
}
```

Different implementations of PasswordEncoder inside Spring Security

NoOpPasswordEncoder (Not recommended for Prod apps)

StandardPasswordEncoder (Not recommended for Prod apps)

Pbkdf2PasswordEncoder

BCryptPasswordEncoder

SCryptPasswordEncoder

Argon2PasswordEncoder

Configure users using InMemoryUserDetailsManager

eazy bytes

Instead of defining a single user inside application.properties, we can define multiple users along with their authorities with the help of InMemoryUserDetailsManager & UserDetails

```
@Bean
public UserDetailsService userDetailsService(){

    var user1 = User.builder().username("madan")
      .password("$2a$12$38udQLdI5.SbyZAZFIs1IuJ9/W3hqkujMTicFNXA18AfhJH5GLUPy") // Madan@123
            .roles("USER").build();

    var user2 = User.builder().username("admin")
      .password("$2a$12$lqclT6YT2aEVd1/45bIrCquO8t.z3R5ygQo1uKNGT7AAj7FjsQsUis") // Admin@123
            .roles("ADMIN").build();

  return new InMemoryUserDetailsManager(user1, user2);
}

@Bean
public PasswordEncoder passwordEncoder(){
  return new BCryptPasswordEncoder();
}
```

Manual Invocation of Spring Security Authentication

eazy bytes

Spring Security usually triggers authentication automatically when we access a secured path via:

- Filter chain (UsernamePasswordAuthenticationFilter)
- Login form / HTTP Basic

But we also have an option of manually triggering the authentication using the AuthenticationManager inside a REST endpoint.

This gives us:

- Full control over login API
- Ability to return custom JSON instead of default Spring responses
- Integration with JWT generation workflow

Manual Invocation of Spring Security Authentication

eazy bytes

AuthenticationManager Setup

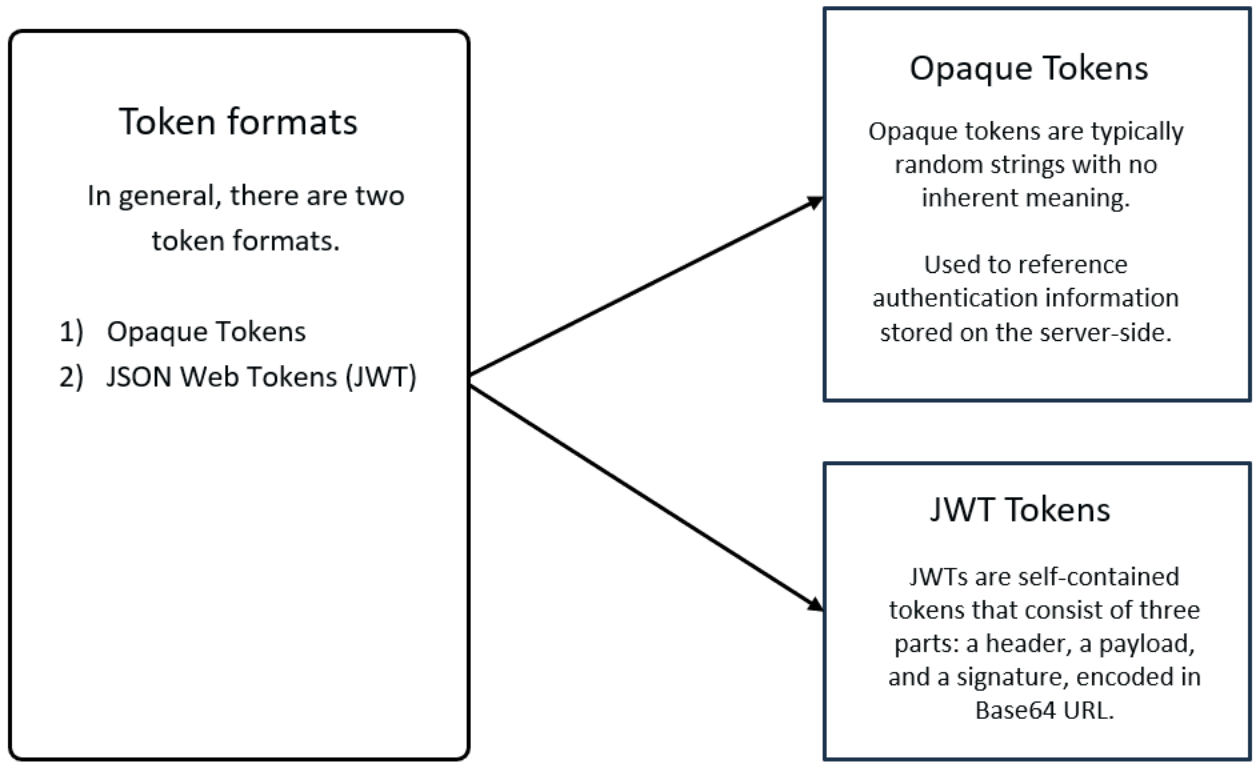
```
@Bean
public AuthenticationManager authenticationManager() {
    var provider = new DaoAuthenticationProvider(userDetailsService());
    provider.setPasswordEncoder(passwordEncoder());
    return new ProviderManager(provider);
}
```

How Authentication can be manually triggered

```
var authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
                loginRequestDto.username(),loginRequestDto.password()));
```

Token formats

eazy bytes



JWT tokens

eazy bytes

- ✓ JWT means JSON Web Token. It is a token implementation which will be in the JSON format and designed to use for the web requests.
- ✓ JWT is the most common and favorite token type that many systems use these days due to its special features and advantages.
- ✓ JWT tokens can be used both in the scenarios of Authorization/Authentication along with Information exchange which means you can share certain user related data in the token itself which will reduce the burden of maintaining such details in the sessions on the client/server side.

A JWT token has 3 parts each separated by a period(.) Below is a sample JWT token,

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

1. Header
2. Payload
3. Signature (Optional)

JWT tokens

eazy bytes

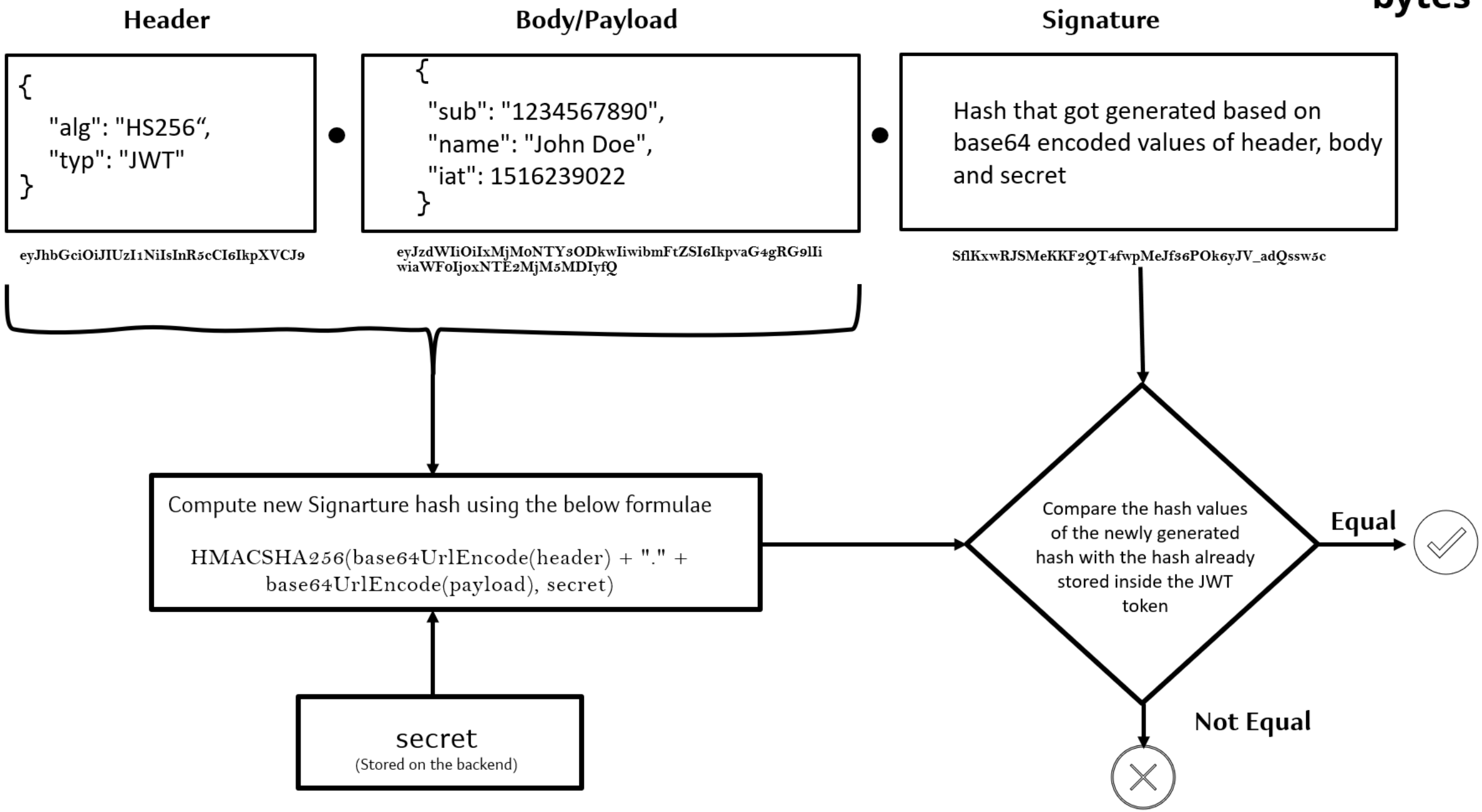
- ✓ The last part of the token is the digital signature. This part can be optional if the party that you share the JWT token is internal and that someone who you can trust but not open in the web.
- ✓ But if you are sharing this token to the client applications which will be used by all the users in the open web then we need to make sure that no one changed the header and body values like Authorities, username etc.
- ✓ To make sure that no one tampered the data on the network, we can send the signature of the content when initially the token is generated. To create the signature part you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

- ✓ For example if you want to use the HMAC SHA256 algorithm, the signature will be created in the following way:

```
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)
```

- ✓ The signature is used to verify the message wasn't changed along the way, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

eazy bytes


Header
Body/Payload
Signature
{
"alg": "HS256“,
"typ": "JWT"
}
{
"sub": "1234567890",
"name": "John Doe",
"iat": 1516239022
}
Hash that got generated based on base64 encoded values of header, body and secret
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
Compute new Signarture hash using the below formulae
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)
secret
(Stored on the backend)
Compare the hash values of the newly generated hash with the hash already stored inside the JWT token
Equal
Not Equal

Generating JWT Token after Successful Authentication

eazy bytes

JWT tokens can be issued after successful Spring Security authentication. The token contains user identity & roles, signed using a secure key, and returned to the client for stateless access.

Dependencies (jjwt)

Add below dependencies,

- jjwt-api
- jjwt-impl
- jjwt-jackson

api → Core JWT specification
impl → Signing, parsing
jackson → JSON serialization/deserialization

Why JJWT?

- Mature JWT implementation
- Simple builder API
- Flexible signing options
- Secure key handling

Generating JWT Token after Successful Authentication

eazy bytes

Flow Overview

```
Client → /auth/login → AuthenticationManager → If Valid → JwtUtil.generateJwtToken() → Return JWT
```

- jjwt-api
- jjwt-impl
- jjwt-jackson

Core JWT Generation Logic

```
String token = Jwts.builder()
        .issuer("Job Portal")
        .subject("JWT Token")
        .claim("username", user.getUsername())
        .claim("roles", authorities)
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis() + 24*60*60*1000))
        .signWith(secretKey)
        .compact();
```

Generating JWT Token after Successful Authentication

eazy bytes

Choosing the Signing Key

```
String secret = env.getProperty("jwt.secret");
SecretKey secretKey = Keys.hmacShaKeyFor(secret.getBytes());
```

Login Endpoint

```
var authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(dto.username(),
            dto.password()));

String jwt = jwtUtil.generateJwtToken(authentication);
return ResponseEntity.ok(new LoginResponse("OK", userDto, jwt));
```

Validating JWT Token Using Custom Filter

eazy bytes

We can authenticate every incoming request without sessions, purely using the Bearer JWT token

Where Validation Happens?

Inside JwtTokenValidatorFilter. Runs before Spring's default authentication filter
Extends OncePerRequestFilter → executes exactly once per request

```
Client → Request → JWT Filter → SecurityContext → Controller
```

Step-by-Step JWT Validation

1) Extract Bearer Token from Header

```
String authHeader = request.getHeader("Authorization");
String jwt = authHeader.substring(7);
```

2) Verify Signature & Parse Claims

```
Claims claims = Jwts.parser().verifyWith(secretKey)
        .build().parseSignedClaims(jwt).getPayload();
```

Validating JWT Token Using Custom Filter

eazy bytes

3) Read User & Roles

```
String username = claims.get("username");
String roles = claims.get("roles");
```

4) Build Authentication Object

```
Authentication auth =
    new UsernamePasswordAuthenticationToken(username, null,
        AuthorityUtils.commaSeparatedStringToAuthorityList(roles));
```

5) Set Authentication into Security Context

```
SecurityContextHolder.getContext().setAuthentication(auth);
```

6) Expired / Invalid Token Handling

```
catch(ExpiredJwtException) → 401 + "Token Expired"
catch(Exception) → BadCredentialsException("Invalid Token received")
```

Validating JWT Token Using Custom Filter

eazy bytes

Paths Skipped from JWT Validation

```
protected boolean shouldNotFilter(HttpServletRequest request) {
   String path = request.getRequestURI();
   return publicPaths.stream().anyMatch(publicPath ->
               pathMatcher.match(publicPath, path));
}
```

Security Configuration Integration

```
.addFilterBefore(new JwtTokenValidatorFilter(publicPaths),
          BasicAuthenticationFilter.class)
```

Placed before Spring's default authentication filter
Ensures that JWT establishes identity early in the filter chain

Registration of new user

eazy bytes

Database Design

Tables involved

- `roles`
- `users`

Key relationships

- A user must have one role → ManyToOne
- A role can be assigned to many users

`users.role_id → roles.id`

Default roles inserted

- ROLE_JOB_SEEKER
- ROLE_EMPLOYER
- ROLE_ADMIN

Generate required Entity classes, DTOs, Repository interfaces for the above tables

Registration of new user

eazy bytes

Checking for Compromised Password

```
CompromisedPasswordDecision decision = compromisedPasswordChecker
                .check(registerRequestDto.password());
        if (decision.isCompromised()) {
            return ResponseEntity
                    .status(HttpStatus.BAD_REQUEST)
                    .body(Map.of("password", "Choose a strong password"));
        }
```

Using CompromisedPasswordChecker implementation, we can check if a password has been compromised. This feature is released as part of Spring Security 6.3 version.

```
@Bean
public CompromisedPasswordChecker compromisedPasswordChecker() {
  return new HaveIBeenPwnedRestApiPasswordChecker();
}
```

Registration of new user

eazy bytes

Duplicate User Validation

```
Optional<JobPortalUser> existingUser = userRepository.findByEmailOrMobileNumber
                (registerRequestDto.email(), registerRequestDto.mobileNumber());
        if (existingUser.isPresent()) {
            Map<String, String> errors = new HashMap<>();
            JobPortalUser jobPortalUser = existingUser.get();
            if (jobPortalUser.getEmail().equalsIgnoreCase(registerRequestDto.email())) {
                errors.put("email", "Email is already registered");
            }
            if (jobPortalUser.getMobileNumber().equals(registerRequestDto.mobileNumber())) {
                errors.put("mobileNumber", "Mobile number is already registered");
            }
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
        }
```

Password Hashing

```
jobPortalUser.setPasswordHash(
        passwordEncoder.encode(registerRequestDto.password())
);
```

Registration of new user

eazy bytes

Assigning Default Role

```
Role role = roleRepository.findByName(ApplicationConstants.ROLE_JOB_SEEKER)
                .orElseThrow(() -> new IllegalArgumentException("Role not found: " +
                        ApplicationConstants.ROLE_JOB_SEEKER));
        jobPortalUser.setRole(role);
        userRepository.save(jobPortalUser);
```

Saving the User to DB

```
userRepository.save(jobPortalUser);
return ResponseEntity
     .status(HttpStatus.CREATED)
     .body("Registration successful");
```

Derived Query Methods in Spring Data JPA

eazy bytes

What are Derived Query Methods ?

Derived query methods are auto-generated queries based on method names in Spring Data JPA.

Instead of writing SQL queries manually, you just name a method in your repository, and Spring Data JPA will automatically figure out the query for you.

How Do Derived Query Methods Work?

Spring Data JPA understands method names and automatically generates queries based on specific keywords in the method name.

For example, with the below method defined,

```
@Repository
public interface CompanyRepository extends JpaRepository<Company, Long> {

    // Spring will automatically generate the SQL query for this method!
    Optional<Company> findByStatus(String status);
}
```

This will automatically generate and execute:

```
SELECT * FROM companies WHERE status = ?;
```

Derived Query Methods in Spring Data JPA

eazy bytes

Do you know ?

A derived query method in Spring Data JPA consists of two main components separated by the first By keyword:

Introducer Clause

- This part specifies the type of operation to be performed on the database.
- Common introducers include find, read, query, count, and get.
- You can also include Distinct to ensure unique results in the query.

Criteria Clause

- This part begins after the first By keyword and defines conditions on entity properties.
- Conditions can be combined using And and Or keywords.

Example Usage:

- `findByStatus(String status)`
- `readByStatus(String status)`
- `queryByStatus(String status)`
- `countByStatus(String status)`

Using readBy, getBy, and queryBy instead of findBy does not change the behavior of the query.

Derived Query Methods in Spring Data JPA

eazy bytes

Common Keywords in Derived Query Methods

Keyword	Meaning	Example method name	Generated SQL
findBy	Find records by a field	findByDepartment(String dept)	SELECT * FROM customers WHERE department = ?
countBy	Count records by a field	countByDepartment(String dept)	SELECT COUNT(*) FROM customers WHERE department = ?
existsBy	Check if record exists	existsByName(String name)	SELECT CASE WHEN COUNT(*) > 0 THEN TRUE ELSE FALSE END FROM customers WHERE name = ?
deleteBy	Delete records by a field	deleteByName(String name)	DELETE FROM customers WHERE name= ?
removeBy	Remove records by a field	removeByAge(int age)	DELETE FROM customers WHERE age = ?

Using Multiple Fields in Queries

Example method name	Generated SQL
findByNameAndAge(String name, int age)	SELECT * FROM customers WHERE name = ? AND age = ?
findByDepartmentOrAge(String dept, int age)	SELECT * FROM customers WHERE department = ? OR age = ?
findDistinctByNameAndAge(String name, int age)	SELECT DISTINCT(*) FROM customers WHERE name = ? AND age = ?

Derived Query Methods in Spring Data JPA

eazy bytes

Using Comparisons in Queries

Example method name	Generated SQL
findByAgeGreaterThan(int age)	SELECT * FROM customers WHERE age > ?;
findByAgeLessThan(int age)	SELECT * FROM customers WHERE age < ?;
findByAgeBetween(int start, int end)	SELECT * FROM customers WHERE age BETWEEN ? AND ?;
findByNameLike(String pattern)	SELECT * FROM customers WHERE name LIKE ?;
findByNameNotLike(String pattern)	SELECT * FROM customers WHERE name NOT LIKE ?;
findByNameStartingWith(String prefix)	SELECT * FROM customers WHERE name LIKE ?%;
findByNameEndingWith(String suffix)	SELECT * FROM customers WHERE name LIKE %?;
findByNameContaining(String substring)	SELECT * FROM customers WHERE name LIKE %?%;

Sorting with Derived Queries

Example method name	Generated SQL
findByDepartmentOrderByAgeAsc(String dept)	SELECT * FROM customers WHERE department = ? ORDER BY age ASC
findByDepartmentOrderByNameDesc(String dept)	SELECT * FROM customers WHERE department = ? ORDER BY name DESC

Derived Query Methods in Spring Data JPA

eazy bytes

Limiting the Number of Results

Example method name	Generated SQL
findFirstByOrderByAgeAsc()	SELECT * FROM customers ORDER BY age ASC LIMIT 1
findTop3ByOrderByAgeDesc()	SELECT * FROM customers ORDER BY age DESC LIMIT 3

Handling Null Values

Example method name	Generated SQL
findByDepartmentIsNull()	SELECT * FROM customers WHERE department IS NULL
findByDepartmentIsNotNull()	SELECT * FROM customers WHERE department IS NOT NULL

Working with Dates

Example method name	Generated SQL
findByStartDateAfter(Date startDate)	SELECT * FROM customers WHERE start_date > ?
findByStartDateBefore(Date startDate)	SELECT * FROM customers WHERE start_date < ?

Derived Query Methods in Spring Data JPA

eazy bytes

Checking multiple values

Example method name	Generated SQL
findByDepartmentIn(Arrays.asList("HR", "IT"))	SELECT * FROM customers WHERE department IN ('HR', 'IT')
findByDepartmentNotIn(Arrays.asList("HR", "IT"))	SELECT * FROM customers WHERE department NOT IN ('HR', 'IT')

Checking Boolean values

Example method name	Generated SQL
findByActiveTrue()	SELECT * FROM customers WHERE active = true
findByActiveFalse()	SELECT * FROM customers WHERE active = false

Ignoring case

Example method name	Generated SQL
findByFirstnameIgnoreCase(String name)	SELECT * FROM customers U WHERE UPPER(U.name) = UPPER(?)

Derived Query Methods in Spring Data JPA

eazy bytes

Combining Everything: A Complex Query

You can mix multiple conditions together!

Example:

```
List<Customer> findByDepartmentAndAgeGreaterThanOrderByNameAsc(String department, int age);
```

This will generate:

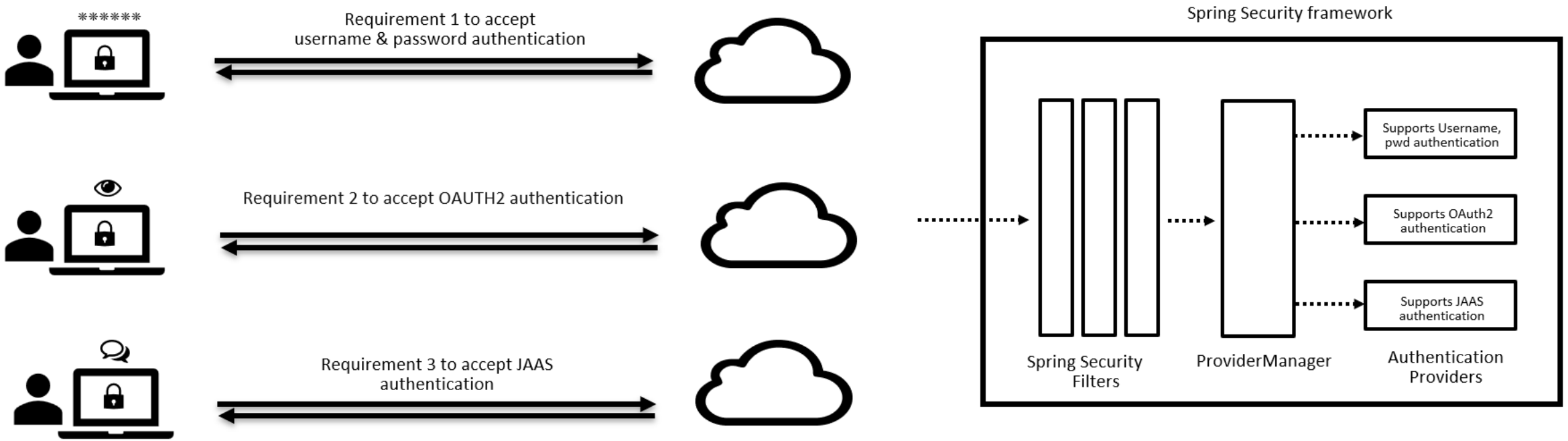
```
SELECT * FROM customers WHERE department = ? AND age > ? ORDER BY name ASC;
```

If a derived query gets too complex, it's better to use @Query

Authentication Provider

eazy bytes

Why we need to implement our own Authentication Provider ?



- ✓ The AuthenticationProvider in Spring Security takes care of the authentication logic. The default implementation of the AuthenticationProvider is to delegate the responsibility of finding the user in the system to a UserDetailsService implementation & PasswordEncoder for password validation. But if we have a custom authentication requirement that is not fulfilled by Spring Security framework, then we can build our own authentication logic by implementing the AuthenticationProvider interface.
- ✓ It is the responsibility of the ProviderManager which is an implementation of AuthenticationManager, to check with all the implementations of Authentication Providers and try to authenticate the user.

Authentication Provider

eazy bytes

Methods inside AuthenticationProvider Interface

```
public interface AuthenticationProvider {

        Authentication authenticate(Authentication authentication)
                throws AuthenticationException;

        boolean supports(Class<?> authentication);
}
```

The authenticate() method receives and returns authentication object. We can implement all our custom authentication logic inside authenticate() method.

The second method in the AuthenticationProvider interface is supports(Class<?> authentication). You'll implement this method to return true if the current AuthenticationProvider supports the type of the Authentication object provided.

Defining our own AuthenticationProvider implementation

eazy bytes

```
@Component
@RequiredArgsConstructor
public class JobPortalUsernamePwdAuthenticationProvider implements AuthenticationProvider {

    private final JobPortalUserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        String username = authentication.getName();
        String pwd = authentication.getCredentials().toString();
        JobPortalUser jobPortalUser = userRepository.findByEmail(username).orElseThrow(
                () -> new UsernameNotFoundException(
                        "User details not found for the user: " + username)
        );
        List<SimpleGrantedAuthority> authorities = List.of(
                                new SimpleGrantedAuthority(jobPortalUser.getRole().getName()));
        if (passwordEncoder.matches(pwd, jobPortalUser.getPasswordHash())) {
            return new UsernamePasswordAuthenticationToken(jobPortalUser, null, authorities);
        } else {
            throw new BadCredentialsException("Invalid password!");
        }
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return (UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication));
    }

}
```

Login using our own AuthenticationProvider implementation

eazy bytes

AuthenticationManager Setup

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationProvider authenticationProvider) {
    return new ProviderManager(authenticationProvider);
}
```

Login API logic

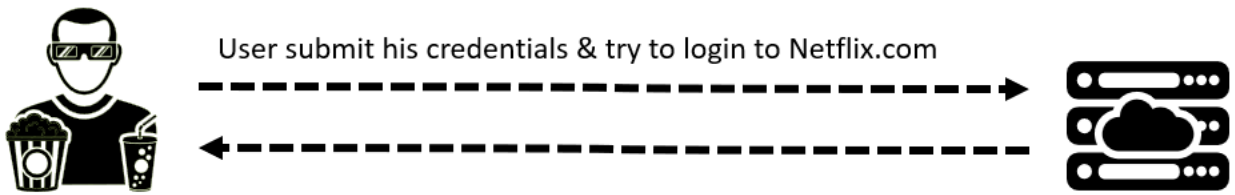
```
 PostMapping(value = "/login/public", version = "1.0")
    public ResponseEntity<LoginResponseDto> apiLogin(@RequestBody LoginRequestDto loginRequestDto) {
            var resultAuthentication = authenticationManager.authenticate(new
UsernamePasswordAuthenticationToken(loginRequestDto.username(),
                    loginRequestDto.password()));

         String jwtToken = jwtUtil.generateJwtToken(resultAuthentication);
         var userDto = new UserDto();
          var loggedInUser = (JobPortalUser) resultAuthentication.getPrincipal();
          BeanUtils.copyProperties(loggedInUser, userDto);
          userDto.setRole(loggedInUser.getRole().getName());
         return ResponseEntity.status(HttpStatus.OK)
           .body(new LoginResponseDto(HttpStatus.OK.getReasonPhrase(),userDto, jwtToken));
    }
```

CROSS-SITE REQUEST FORGERY (CSRF)

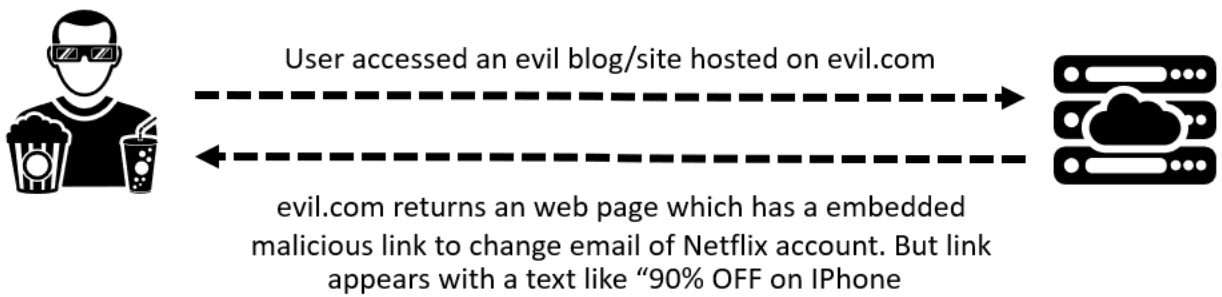
eazy bytes

- A typical Cross-Site Request Forgery (CSRF or XSRF) attack aims to perform an operation in a web application on behalf of a user without their explicit consent. In general, it doesn't directly steal the user's identity, but it exploits the user to carry out an action without their will.
- Consider you are using a website netflix.com and the attacker's website evil.com

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com



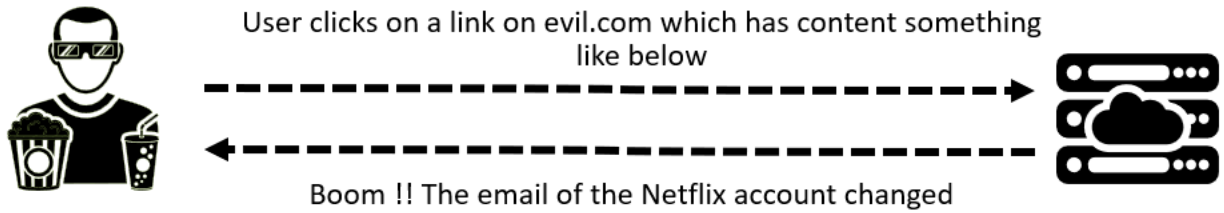
Step 2 : The same Netflix user opens an evil.com website in another tab of the browser.



CROSS-SITE REQUEST FORGERY (CSRF)

eazy bytes

Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com, the backend server of Netflix.com can't differentiate from where the request came. So here the evil.com forged the request as if it is coming from a Netflix.com UI page.



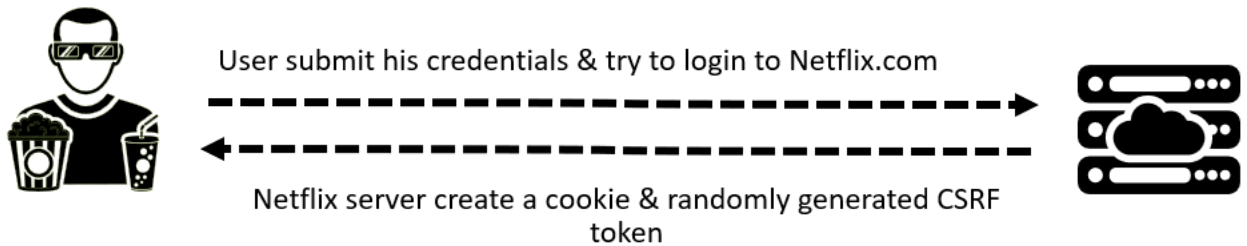

```
<form action="https://netflix.com/changeEmail"
   method="POST" id="form">
  <input type="hidden" name="email"
value="user@evil.com">
<form>

<script>
  document.getElementById('form').submit()
</script>
```

SOLUTION TO CSRF ATTACK

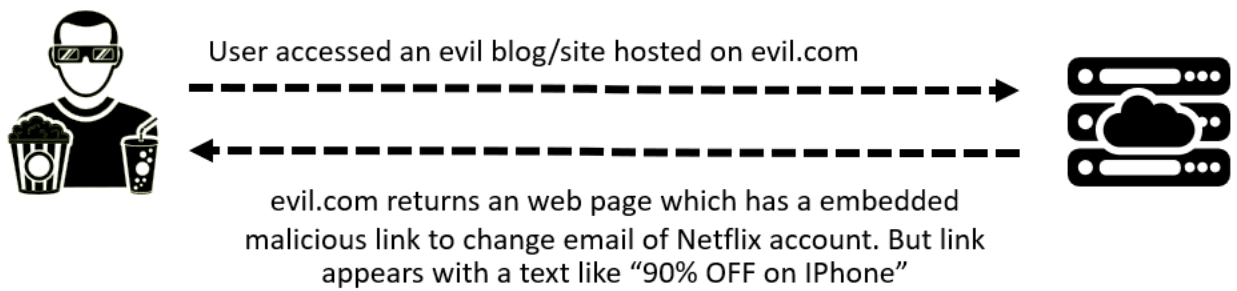
eazy bytes

- To defeat a CSRF attack, applications need a way to determine if the HTTP request is legitimately generated via the application's user interface. The best way to achieve this is through a CSRF token. A CSRF token is a secure random token that is used to prevent CSRF attacks. The token needs to be unique per user session and should be of large random value to make it difficult to guess.
- Let's see how this solve CSRF attack by taking the previous Netflix example again,

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com along with a randomly generated unique CSRF token for this particular user session.



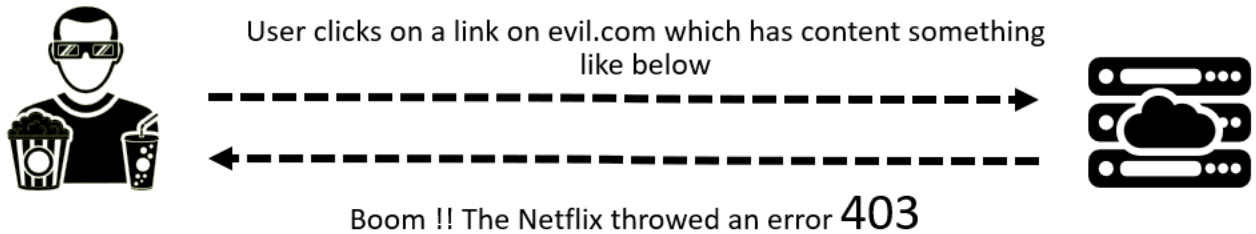
Step 2 : The same Netflix user opens an **evil.com** website in another tab of the browser.



SOLUTION TO CSRF ATTACK

eazy bytes

Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com. This time the Netflix.com backend server expects CSRF token along with the cookie. The CSRF token must be same as initial value generated during login operation



The CSRF token will be used by the application server to verify the legitimacy of the end-user request if it is coming from the same App UI or not. The application server rejects the request if the CSRF token fails to match the test.

DISABLE CSRF PROTECTION

eazy bytes

By default Spring Security block all HTTP POST, PUT, DELETE, PATCH operations with an error of 403, if there is no CSRF solution implemented insdie a web application. We can change this default behaviour by disabling the CSRF protection provided by Spring Security.

```
@Bean
SecurityFilterChain customSecurityFilterChain(HttpSecurity http) {

  return http.csrf(csrfConfig -> csrfConfig.disable())
      .authorizeHttpRequests(requests -> {

        publicPaths.forEach(path ->
            requests.requestMatchers(path).permitAll());

        securedPaths.forEach(path ->
            requests.requestMatchers(path).authenticated());

        requests.anyRequest().denyAll();
      })
      .formLogin(flc -> flc.disable())
      .httpBasic(withDefaults())
      .build();
}
```

CSRF ATTACK SOLUTION

eazy bytes

Spring Security blocks any modifying request unless:

- The request contains a valid CSRF token
- The backend validates it

This prevents:

- Session hijacking
- XSRF attacks in browser-based clients
- Unauthorized POST requests

Our job portal is a browser-based app → CSRF protection IS required.

How Spring Security CSRF Works

Spring Security provides:

- CSRF Token Generator
- CSRF Token Repository (cookie)

Token validator on each modifying request

We can use Cookie-based CSRF Token generation which will Store the token under a cookie: XSRF-TOKEN
Token must be sent by the client in a header: X-XSRF-TOKEN

CSRF ATTACK SOLUTION

eazy bytes

CSRF Configuration (Backend)

```
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    .csrfTokenRequestHandler(new CsrfTokenRequestAttributeHandler())
)
```

CookieCsrfTokenRepository -> Stores token in a cookie accessible by JavaScript → SPA-friendly.
withHttpOnlyFalse() -> Allows frontend to read the cookie value.
CsrfTokenRequestAttributeHandler -> Ensures token is added as request attribute properly.

Rest API to read CSRF token

```
@GetMapping(path = "/public", version = "1.0")
public CsrfToken csrfToken(HttpServletRequest request) {
    return (CsrfToken) request.getAttribute(CsrfToken.class.getName());
}
```

Frontend (React, Angular, Vue) must fetch CSRF token before calling NON SAFE HTTP METHOD APIs.
The token is sent in the response body.

CSRF ATTACK SOLUTION

eazy bytes

Below are the important components that handles CSRF attacks in Spring Security,

CsrfToken - Provides the information/contract about an expected CSRF token.

CsrfTokenRepository – Provides the contract to create, store, and load CSRF tokens. HttpSessionCsrfTokenRepository(stores the CsrfToken in the HttpSession), CookieCsrfTokenRepository (persists the CSRF token in a cookie named "XSRF-TOKEN" and reads from the header "X-XSRF-TOKEN" following the conventions of Angular) are the typically used implementation classes.

CsrfTokenRequestHandler – A callback interface that is used to make the CsrfToken created by the CsrfTokenRepository available as a request attribute. Implementations of this interface may choose to perform additional tasks or customize how the token is made available to the application through request attributes. CsrfTokenRequestAttributeHandler is the typically used implementation classes.

CsrfFilter – Spring Security in built filter responsible to validate the CSRF token received by client and to decide whether to throw an exception or not

Logging in Spring Boot

eazy bytes

What is Logging?

- Logging is the process of recording important events in an application.
- Helps in debugging, monitoring, and troubleshooting.
- Logs provide information like errors, warnings, execution flow, and performance insights.

Key benefits of logging:

1. Debugging – Helps in finding and fixing errors.
2. Monitoring – Tracks application behavior over time.
3. Auditing – Keeps a record of important actions.
4. Performance Analysis – Helps in identifying slow parts of the app.

Why NOT Use System.out.println()

Because System.out has,

- No log levels
- No file logging
- No formatting
- Not production-ready

In real apps, System.out.println() is not acceptable.

Logging in Spring Boot

eazy bytes

Spring Boot uses SLF4J (Simple Logging Facade for Java) with Logback by default. No need to add dependencies—it works out of the box.

What is SLF4J?
SLF4J is a common logging interface that Spring Boot uses to write logs, without caring which logging framework is underneath. Spring Boot supports different logging frameworks:

Framework	Description
SLF4J (Facade)	Generic logging API (recommends best practice)
Logback (Default)	The default Spring Boot logger
Log4j2	Powerful and feature-rich logging
Java Util Logging (JUL)	Built-in Java logging

Each one has different APIs. If your code depends directly on Logback:

- Switching to Log4j2 later becomes painful
- Your code becomes tightly coupled

SLF4J Solves This Problem

- SLF4J = Simple Logging Facade for Java
- Facade means wrapper / front door
- It provides one common logging API
- Internally, it forwards logs to the actual logging framework

Logging in Spring Boot

eazy bytes

Default Log Format

```
2100-11-20T16:37:12.913Z  INFO 127185 --- [myapp] [main] com.example.MyApp : Application started successfully!
```

The following items are output:

```
Date and Time: Millisecond precision and easily sortable

Log Level: ERROR, WARN, INFO, DEBUG, or TRACE

Process ID

A --- separator to distinguish the start of actual log messages

Application name: Enclosed in square brackets (logged by default only if spring.application.name is set)

Application group: Enclosed in square brackets (logged by default only if spring.application.group is set)

Thread name: Enclosed in square brackets (may be truncated for console output)

Correlation ID: If tracing is enabled (not shown in the sample above)

Logger name: This is usually the source class name (often abbreviated)

The log message
```

Logging in Spring Boot

eazy bytes

Formatting Logs (Pattern Configuration)

```
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
```

Example Output

```
2100-11-20 10:30:45 [main] INFO  com.example.MyService - Application started!
```

Logging in Spring Boot

eazy bytes

Log Levels in Spring Boot. By default, Spring Boot sets the root log level to INFO

Log Level	Description	Example Use Case
TRACE	Most detailed logs	Tracking method calls
DEBUG	Debugging information	Variable values, steps in execution
INFO	General application info	Startup messages, business events
WARN	Potential issues	Deprecated API usage, high memory usage
ERROR	Serious problems	Exceptions, failed transactions

Impact of Different Log Levels

Configured Log Level	Logs That Appear
TRACE	Shows TRACE, DEBUG, INFO, WARN, ERROR
DEBUG	Shows DEBUG, INFO, WARN, ERROR
INFO	Shows INFO, WARN, ERROR
WARN	Shows WARN, ERROR
ERROR	Shows only ERROR

Logging in Spring Boot

eazy bytes

Changing Log Levels in application.properties

Set log level for the entire application:

```
logging.level.root=DEBUG
```

Set log level for a specific package:

```
logging.level.com.example.myapp=DEBUG
```

Log groups let you assign a single log level to multiple packages at once

```
# Create a log group with a logical name

logging.group.jobportal_error= com.eazybytes.jobportal.aspects,
                               com.eazybytes.jobportal.controller

# Setting Log Level for the Group

logging.level.jobportal_error=ERROR
```

Logging in Spring Boot

eazy bytes

By default, logs are printed to the console. To save them in a file, configure:

```
# Save logs to a file
logging.file.name=/Users/eazybytes/Desktop/logs/app.log
```

Using JSON Logs for Structured Logging

To enable JSON logs for easy parsing in ELK Stack or Cloud Logging:

```
logging.pattern.file={"timestamp":"%d{yyyy-MM-dd HH:mm:ss}",
                "level":"%p","logger":"%c","message":"%m"}
```

For advanced use cases, we can use logback.xml which is a configuration file that controls how logs are written

- Using logback.xml, we can:
- Decide where logs go (console / file)
- Decide how logs look (format)
- Decide how much to log (INFO / ERROR / DEBUG)
- Control log rotation

To use this option, we just have to place logback.xml under src/main/resources/

Logging in Spring Boot

eazy bytes

Logging example,

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
@RequestMapping("/logging")
public class LoggingController {

    private static final Logger LOGGER = LoggerFactory.getLogger(LoggingController.class);


    @GetMapping(version = "1.0")
    public ResponseEntity<String> testLogging() {
       LOGGER.trace(" TRACE: This is a very detailed trace log. Used for tracking execution flow.");
       LOGGER.debug(" DEBUG: This is a debug message. Used for debugging.");
       LOGGER.info(" INFO: This is an informational message. Application events.");
       LOGGER.warn(" WARN: This is a warning! Something might go wrong.");
       LOGGER.error(" ERROR: An error occurred! This needs immediate attention.");
        return ResponseEntity.status(HttpStatus.OK).
                body("Logging tested successfully");
    }
}
```

Logging in Spring Boot

eazy bytes

Logging example with Lombok,

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
@RequestMapping("/logging")
@Slf4j
public class LoggingController {

    @GetMapping(version = "1.0")
    public ResponseEntity<String> testLogging() {
       log.trace(" TRACE: This is a very detailed trace log. Used for tracking execution flow.");
       log.debug(" DEBUG: This is a debug message. Used for debugging.");
       log.info(" INFO: This is an informational message. Application events.");
       log.warn(" WARN: This is a warning! Something might go wrong.");
       log.error(" ERROR: An error occurred! This needs immediate attention.");
       return ResponseEntity.status(HttpStatus.OK).
                body("Logging tested successfully");
    }
}
```

Lombok `@Slf4j` automatically creates the logger variable log ! No need for LoggerFactory!

eazy bytes

AOP

ASPECT-ORIENTED PROGRAMMING

Adding Superpowers to Your Code

eazy bytes

What is Spring AOP?

AOP is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns.

Cross-Cutting Concerns: These are features required in many different parts of an application, but they are not the primary business logic. Ex: Logging, Security, Auditing, Transaction Management, Caching etc

Problem without AOP (Code Tangling): Without AOP, the code for logging, security, and transaction management gets repeatedly mixed in with your core business methods. Your primary methods become cluttered and harder to read/maintain.

Solution with AOP (Code Weaving): AOP allows you to define this common logic in a single, reusable module called an Aspect.

Spring then "weaves" (automatically inserts) this logic into your methods at the specified points, keeping your business logic clean.

eazy bytes

Without AOP, the code for logging, security, and transaction management gets repeatedly mixed into your core business methods. This makes your primary methods cluttered and harder to read/maintain.

Traditional Approach - Code Tangling:

```
public void transferMoney() {
    // Security check - 5 lines
    // Logging start - 3 lines
    // Start transaction - 4 lines

    // ACTUAL BUSINESS LOGIC - 2 lines!

    // Commit transaction - 3 lines
    // Logging end - 3 lines
    // Audit trail - 4 lines
}
```

With Spring AOP:

```
public void transferMoney() {
    // JUST business logic!
    // Clean, readable, maintainable!
}
```

Key AOP Terminology: The 3 W's

To implement Aspect-Oriented Programming (AOP), Spring requires you to define three critical components:

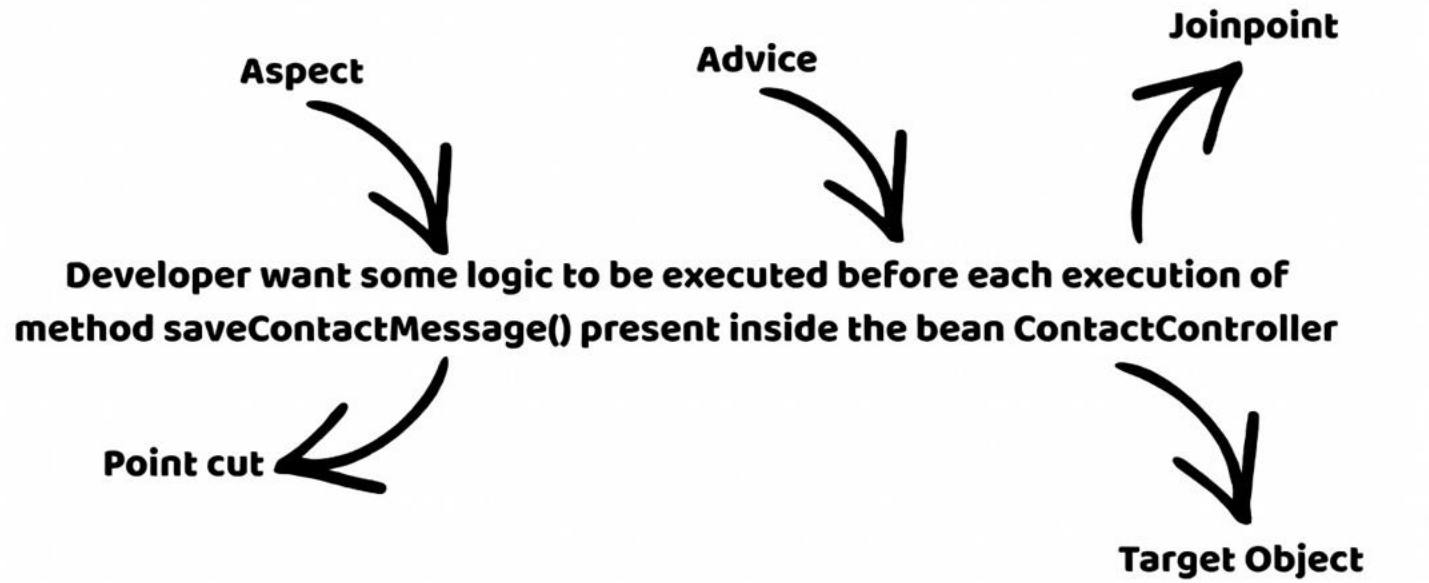
Term	What it Asks	Definition
Aspect (WHAT)	WHAT code to execute?	WHAT code or logic we want the Spring to execute when you call a specific method. This is called as Aspect.
Advice (WHEN)	WHEN method need to be executed ?	WHEN the Spring need to execute the given Aspect. For example is it before or after the method call. This is called as Advice.
Pointcut (WHICH)	WHICH method needs to be intercepted ?	WHICH method inside App that framework needs to intercept and execute the given Aspect. This is called as a Pointcut.

Join point which defines the event that triggers the execution of an aspect. Inside Spring, this event is always a method call.

Target object is the bean that declares the method/pointcut which is intercepted by an aspect.

eazy bytes

Typical Scenario of AOP implementation



eazy bytes

How Spring AOP Works: The Magic of Proxies

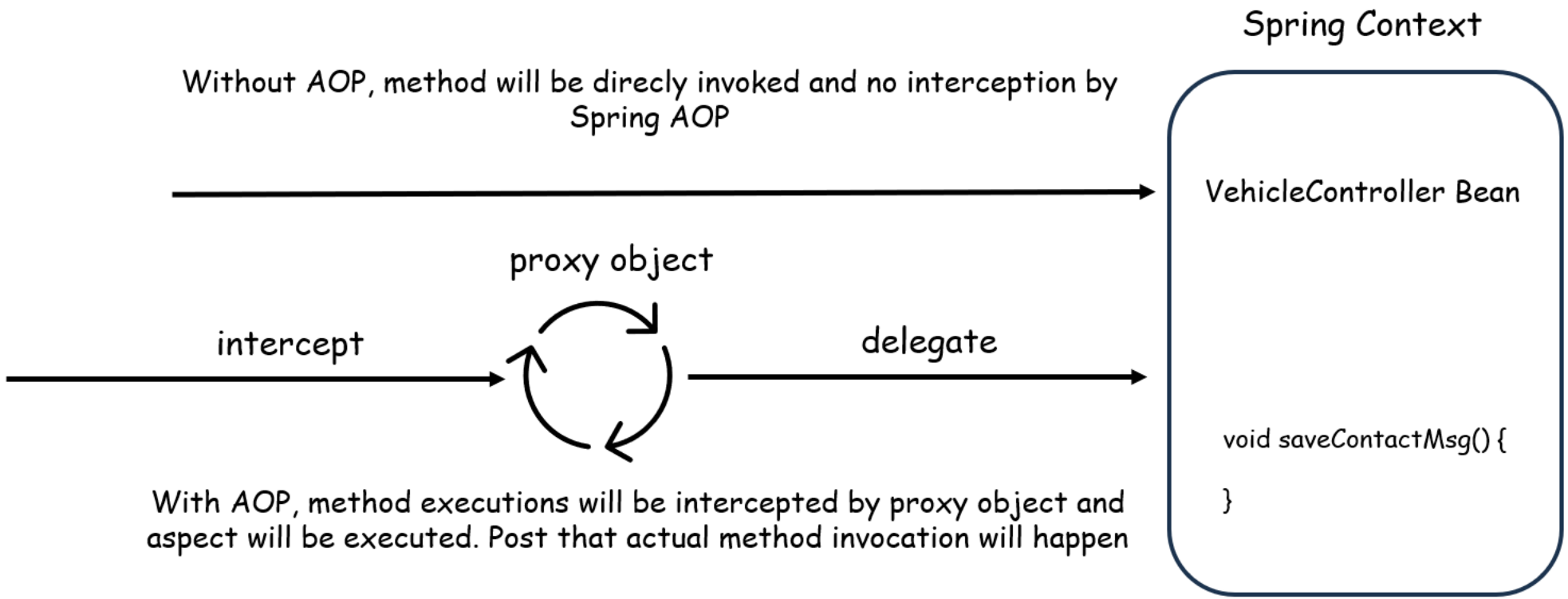
When we implement AOP inside our application using the Spring framework, it will intercept each method call and apply the logic defined in the Aspect. This process of inserting the Aspect logic into the target object is called Weaving.

The Mechanism: Proxy Objects

How does Spring achieve this interception and weaving without modifying your original business code? It uses the Proxy Design Pattern.

1) Original Bean: You define your original business class (e.g., ContactController).
2) The Proxy: Spring does not give you a direct reference to your original ContactController bean. Instead, it wraps the target bean with a Proxy Object.
3. Interception: When you call a method on the controller, you are actually calling the method on the Proxy.

- The Proxy intercepts the call.
- The Proxy executes the Advice (your logging/security logic).
- The Proxy then delegates the call to the actual method on the Target Object (your original bean).


Spring Context
Without AOP, method will be direcly invoked and no interception by Spring AOP
VehicleController Bean
proxy object
intercept
delegate
void saveContactMsg() {
}
With AOP, method executions will be intercepted by proxy object and aspect will be executed. Post that actual method invocation will happen

Advice Types

The Advice defines WHAT runs, but it also specifies the temporal WHEN (Before, After, or Around) relative to the intercepted method's execution:

Advice Type	WHEN it Runs
@Before	Before the target method starts
@AfterReturning	After the target method returns successfully (no exception)
@AfterThrowing	After the target method throws an exception
@After	After the target method finishes (regardless of success or failure)
@Around	Around the target method execution, giving full control to intercept, modify, or suppress the call

eazy bytes

What Is a Pointcut ?

A pointcut is a rule that tells Spring AOP where an aspect should run. Using pointcut, we tell “Run this advice ONLY for these methods”

Think of it as:

- A filter
- A selector
- A targeting rule

The Most Common Pointcut: execution

The execution() designator is the most frequently used and gives you the highest level of control. We usually say:
“Intercept this method execution”. That’s why the name execution(). It follows a specific pattern, like filling out a highly detailed form:

```
execution(
   modifiers-pattern?
   ret-type-pattern
   declaring-type-pattern?name-pattern(param-pattern)
   throws-pattern?
)
```

eazy bytes

Execution format :

```
execution(
   modifiers-pattern? ret-type-pattern declaring-type-pattern?name-pattern(param-pattern)
   throws-pattern? )
```

Pattern Component	Required ?	Example	Meaning
modifiers-pattern	Optional	public	The access modifier (e.g., public, private)
ret-type-pattern	Required	* or String	What the method must return. * means any return type.
declaring-type-pattern	Optional	com.app.service.	Where the method is defined (package/class).
name-pattern	Required	save* or *	The name of the method. * matches any method name.
param-pattern	Required	(..) or (*, String)	The number and type of arguments: () (no params), (..) (any number/type), (*) (one param of any type)
throws-pattern	Optional	throwable	Matches methods that declare throwing a specific exception.

eazy bytes

Common Wildcards:

Wildcards make the execution expression flexible:

* (Asterisk): Matches any single element (e.g., any return type, any method name).
+(Plus) : This class or its children"AccountService+"AccountService and all its kids
.. (Double Dot): Matches zero or more elements.

- In the parameters section, it matches any number of arguments: (..)
- In the package section, it matches the package and all its sub-packages: com.app.service..*

Goal	execution() Expression	Notes
Any Public Method	execution(public * *(..))	public modifier, any return type (*), any method name (*), any parameters ((..)).
Methods Starting with set	execution(* set*(..))	Any return type, method name starting with set, any parameters
Methods in a Specific Interface	execution(* com.svc.AccountService.*(..))	Targets all methods (*) within the AccountService interface
Methods in a Package & Subpackages	execution(* com.svc..*.*(..))	The ..* matches the svc package AND all sub-packages (e.g., svc.impl)

eazy bytes

@Around Advice in Spring AOP

@Around advice surrounds the method execution It allows us to Run logic before method execution, Control when the method runs, Run logic after method execution, Measure execution time, Access arguments & return value. This is the most powerful AOP advice

```
@Aspect
@Component
@Slf4j
public class LoggingAndPerformanceAspect {

    @Around("execution(* com.eazybytes.jobportal..*.*(..))")
    public Object logAndMeasureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable
{
         // Run before logic
         // Proceed with actual business method
        Object result = joinPoint.proceed();
         // Run after logic
         return result;
    }
}
```

eazy bytes

@Before Advice in Spring AOP

@Before advice runs before the target method executes. Perfect for Input validation, Pre-conditions, Security checks, Blocking invalid requests

```
@Before("""
    execution(* com.eazybytes.jobportal.auth.AuthController
    .registerUser(..))
    """)
public void validateBeforeRegister(JoinPoint joinPoint) {
    Object[] args = joinPoint.getArgs();

    //  Compromised password check

    //  Existing user check

    // Stop execution if validation fails by throwing an exception
}
```

eazy bytes

@AfterReturning Advice in Spring AOP

@AfterReturning runs only when a method completes successfully. No execution on exceptions. Access to returned value. Zero impact on controller logic

```
@Aspect
@Component
@Slf4j
public class LoginSuccessAuditAspect {

    @AfterReturning(pointcut = "execution(*
        com.eazybytes.jobportal.auth.AuthController.apiLogin(..))",
            returning = "response")
    public void logSuccessfulLogin(JoinPoint joinPoint, Object response) {

        // Log details of successful login
    }

}
```

eazy bytes

@AfterThrowing Advice in Spring AOP

@AfterThrowing runs only when a method throws an exception. It does NOT run on success. It does NOT change the response. It reacts after failure. Perfect for Error logging, Auditing, Alerts, Failure metrics

```
@Aspect
@Component
@Slf4j
public class ExceptionAuditAspect {

    @AfterThrowing(pointcut = "execution(* com.eazybytes.jobportal..*.*(..))",
            throwing = "ex")
    public void logAfterException(JoinPoint joinPoint, Exception ex) {

        String methodName = joinPoint.getSignature().toShortString();
        Object[] methodArgs = joinPoint.getArgs();

        // Here you could log details, Send metrics, Push audit events, Trigger alerts
    }
}
```

eazy bytes

Annotation-Based AOP

Instead of targeting packages or method names using Pointcuts, we can target specific methods by using Annotations as well

Step 1 – Create a Custom Annotation

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface LogAspect {
}
```

- @Retention(RUNTIME) → Available at runtime for AOP
- @Target(METHOD) → Can be applied only on methods
- This annotation is a marker (It doesn't contain logic)

Step 2 – Apply Annotation on Any Method

```
@LogAspect
public void createAccount() {
    // business logic
}
```

- No package rules
- No naming conventions
- Explicit & readable intent

eazy bytes

Annotation-Based AOP

Run this advice around any method that has @LogAspect annotation

```
@Aspect
@Component
@Slf4j
public class LoggingAndPerformanceAspect {

    @Around("@annotation(com.eazybytes.jobportal.aspects.LogAspect)")
    public Object logAndMeasureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        // Run before logic
        // Proceed with actual business method
        Object result = joinPoint.proceed();
        // Run after logic
        return result;
    }
}
```

eazy bytes

Writing Custom Queries using

@Query annotation

eazy bytes

Problem Scenario

Business Requirement

Fetch only those companies which have ACTIVE jobs, and fetch those jobs eagerly in a single query.

Entities

```
Company → One-to-Many → Job
```

Job.status should be ACTIVE

Can we solve this using Derived Query Methods ?

No. For example using below method will not help

```
List<Company> findByJobsStatus(String status);
```

Problems with Derived Queries

- Hard to express complex joins
- Cannot use FETCH JOIN
- Cannot control join type
- Cannot avoid N+1 problem

Derived queries are good for simple queries only

eazy bytes

N+1 problem

What is the N+1 Problem ?

The N+1 problem occurs when Hibernate executes one query to fetch parent entities and an additional query for each child collection, leading to performance issues.

For example, with our DB schema, one company can have many Jobs. By default jobs are LAZY loaded

```
@OneToMany(mappedBy = "company")
private List<Job> jobs;
```

How N+1 problem occurs ?

Step 1: Fetch Companies

```
List<Company> companies = companyRepository.findAll();
```

Hibernate executes 1 query:

```
SELECT * FROM companies;
```

Let's say: 30 companies are returned. So far → 1 query

Step 2: Access Jobs for Each Company

```
for (Company company : companies) {
    company.getJobs().size();
}
```

Now Hibernate does this:

SELECT * FROM jobs WHERE company_id = 1;

SELECT * FROM jobs WHERE company_id = 2;

...

One query per company. If N = 30 companies, then total queries = 31 SQL queries

This is called the N+1 problem

eazy bytes

N+1 problem

How Hibernate batch fetching can save from N+1 problem

Hibernate notices: “You are loading the same association (jobs) repeatedly for many parents.”

Instead of loading one company’s jobs at a time, it groups multiple parent IDs together and fetches their children in a single query.

For example,

```
SELECT * FROM jobs
WHERE company_id IN (1,2,3,4,5,6,7,8,9,10);
```

How N+1 can be solved using batch fetch ?

Approach 1: spring.jpa.properties.hibernate.default_batch_fetch_size=50

This is a global setting
Applies to all LAZY associations and entities
Hibernate will try to fetch up to 50 parent IDs at once

Approach 2: @BatchSize(size = 50)

```
@OneToMany(....)
@BatchSize(size = 50)
private List<Job> jobs = new ArrayList<>();
```

Applies batching only to this association
Overrides the global default
Useful when:
Some relations are accessed frequently
Others should remain untouched

Batch fetching does NOT remove N+1 entirely. Instead, it converts: N+1 queries into 1 + (N / batchSize) queries. Always prefer JOIN FETCH to completely solve N+1 problem

Solution using @Query

eazy bytes

```
public interface CompanyRepository extends JpaRepository<Company, Long> {

    @Query("SELECT DISTINCT c FROM Company c JOIN FETCH c.jobs j WHERE j.status =
:status")
    List<Company> findAllWithJobsByStatus(@Param("status") String status);

}
```

Why @Query is the RIGHT choice here

@Query lets you write exactly what you need – no compromises!

- Decide JOIN vs LEFT JOIN
- Decide FETCH strategy
- Decide filtering conditions

Solves N+1 problem

- Jobs are fetched in the same query due to FETCH
- No extra SQL calls

Clear intent

- Anyone reading the query understands business logic
- No guessing from method names

To the @Query annotation,
we need to provide the query by
using either JPQL or Native query

What is JPQL

eazy bytes

JPQL = Java Persistence Query Language

It works on Entities, not tables. Think of JPQL as "SQL that speaks Java"

// SQL thinks in tables and columns

```
SELECT DISTINCT c.* FROM companies c
JOIN jobs j ON c.id = j.company_id
WHERE j.status = 'ACTIVE'
```

// JPQL thinks in objects and properties

```
SELECT DISTINCT c FROM Company c
JOIN FETCH c.jobs j
WHERE j.status = 'ACTIVE'
```

Key Differences:

- Entity Names: Use Company not companies table
- Properties: Use c.jobs not foreign key column name
- Case Sensitive: Entity and property names matter!

Why JPQL is powerful ?

- Portable across databases
- Hibernate converts it to SQL
- Works well with entity relationships
- Perfect for business logic queries

What is a Native Query

eazy bytes

Native Query = Pure SQL

While providing native query, we need to pass `nativeQuery = true` as shown in the below example,

```
@Query(value = "SELECT DISTINCT c.* FROM companies c JOIN jobs j ON c.id = j.company_id
WHERE j.status = ?", nativeQuery = true)
List<Company> findAllWithJobsByStatusNative(String status);
```

When to use Native Queries?

- Database-Specific Features: Window functions, PIVOT
- Performance Optimization: Database-specific hints
- Legacy Databases: Complex stored procedures
- Complex Aggregations: That JPQL can't handle
- Not recommended for everyday use

Trade-offs:

- Database Dependent: Breaks portability
- No Compile-Time Checking: Errors at runtime
- Full SQL Power: Use any database feature
- Performance: Sometimes faster for complex queries

Golden Rule

Simple query → Derived Method
Complex query → @Query (JPQL)
DB-specific logic → Native Query

Probleme with Native Query

eazy bytes

```
SELECT DISTINCT c.* FROM companies c JOIN jobs j ON c.id = j.company_id WHERE j.status = 'ACTIVE'
```

The above native SQL query guarantees only ONE thing: Returned companies have at least one ACTIVE job. It does NOT guarantee:

- which jobs will be loaded
- how jobs will be filtered
- whether jobs are loaded eagerly or lazily

What happens WITHOUT @SQLRestriction

When you later do: `company.getJobs()`

Hibernate executes a separate SQL (lazy loading):

```
SELECT * FROM jobs WHERE company_id = ?
```

No status condition is applied. So Hibernate loads ALL jobs belonging to that company

Why @SQLRestriction("status = 'ACTIVE'") FIXES it

```
@SQLRestriction("status = 'ACTIVE'")
private List<Job> jobs;
```

This tells Hibernate: “Whenever you load Company.jobs, ALWAYS add AND status = 'ACTIVE' to the SQL” So Hibernate now executes:

```
SELECT * FROM jobs WHERE company_id = ?   AND status = 'ACTIVE’
```

Only ACTIVE jobs are loaded
Works with native queries, JPQL, lazy loading, batch loading

Probleme with Native Query

eazy bytes

Why @BatchSize fits nicely here

```
@SQLRestriction("status = 'ACTIVE’”)
@BatchSize(size = 25)
private List<Job> jobs;
```

This optimizes: Lazy loading of jobs

Reduces N+1 problem
Fetches jobs in batches: `WHERE company_id IN (?, ?, ?, ...)`
Combined with: AND status = 'ACTIVE’

- Efficient
- Clean
- Production-grade

Native SQL does NOT control relationship loading
Entity annotations control relationship loading

What is @NamedQuery

eazy bytes

Sometimes, you don't want the query cluttering your Repository interface. **@NamedQuery** defines queries right on your entity class and then, inside repo methods, we can refer using @Query. **@NamedQuery** always uses JPQL.

// As a first step, we can define our query as shown below,

```
@Entity
@NamedQuery(name =
"Company.fetchCompaniesWithJobsByStatus",
 query = "SELECT DISTINCT c FROM Company c
JOIN FETCH c.jobs j WHERE j.status = :status"
)
public class Company { ... }
```

// Spring Data JPA automatically looks for a NamedQuery that matches the name EntityName.MethodName.

```
@Repository
public interface CompanyRepository extends
                  JpaRepository<Company, Long> {

     List<Company> fetchCompaniesWithJobsByStatus
                  (@Param("status") String status);
}
```

Benefits:

- Centralized: All queries in one place (entity class)
- Validated at Startup: Errors caught early
- Reusable: Same query, multiple repositories
- Clean Repositories: No query clutter

// In case if the method name is not matching with the query name, then you need to explicitly provide query details using @Query

```
@Repository
public interface CompanyRepository extends
          JpaRepository<Company, Long> {
    @Query(name = "Company.fetchCompaniesWithJobsByStatus")
    List<Company> findAllWithActiveJobs
          (@Param("status") String status);
}
```

What is @NamedNativeQuery

eazy bytes

@NamedNativeQuery same as **@NamedQuery**, but with SQL.

```
@Entity
@NamedNativeQuery(
   name = "Company.fetchCompaniesWithJobsByStatusNative",
   query = "SELECT DISTINCT c.* FROM companies c JOIN jobs j ON c.id = j.company_id
            WHERE j.status = ?",
   resultClass = Company.class)
public class Company { ... }
```

Why resultClass:

- JPQL (@NamedQuery) works with entities, so JPA already knows which entity to return from the query itself.
- Native SQL (@NamedNativeQuery) works with tables and columns, so JPA has no idea how to map rows to an entity.
- Repository method signatures are not considered at query parsing time.

Hence, in native queries you must explicitly specify **resultClass** to tell JPA how to map the result.

@NamedQueries and @NamedNativeQueries

eazy bytes

We can define multiple JPQL queries and multiple native SQL queries on an entity using:

- @NamedQueries
- @NamedNativeQueries

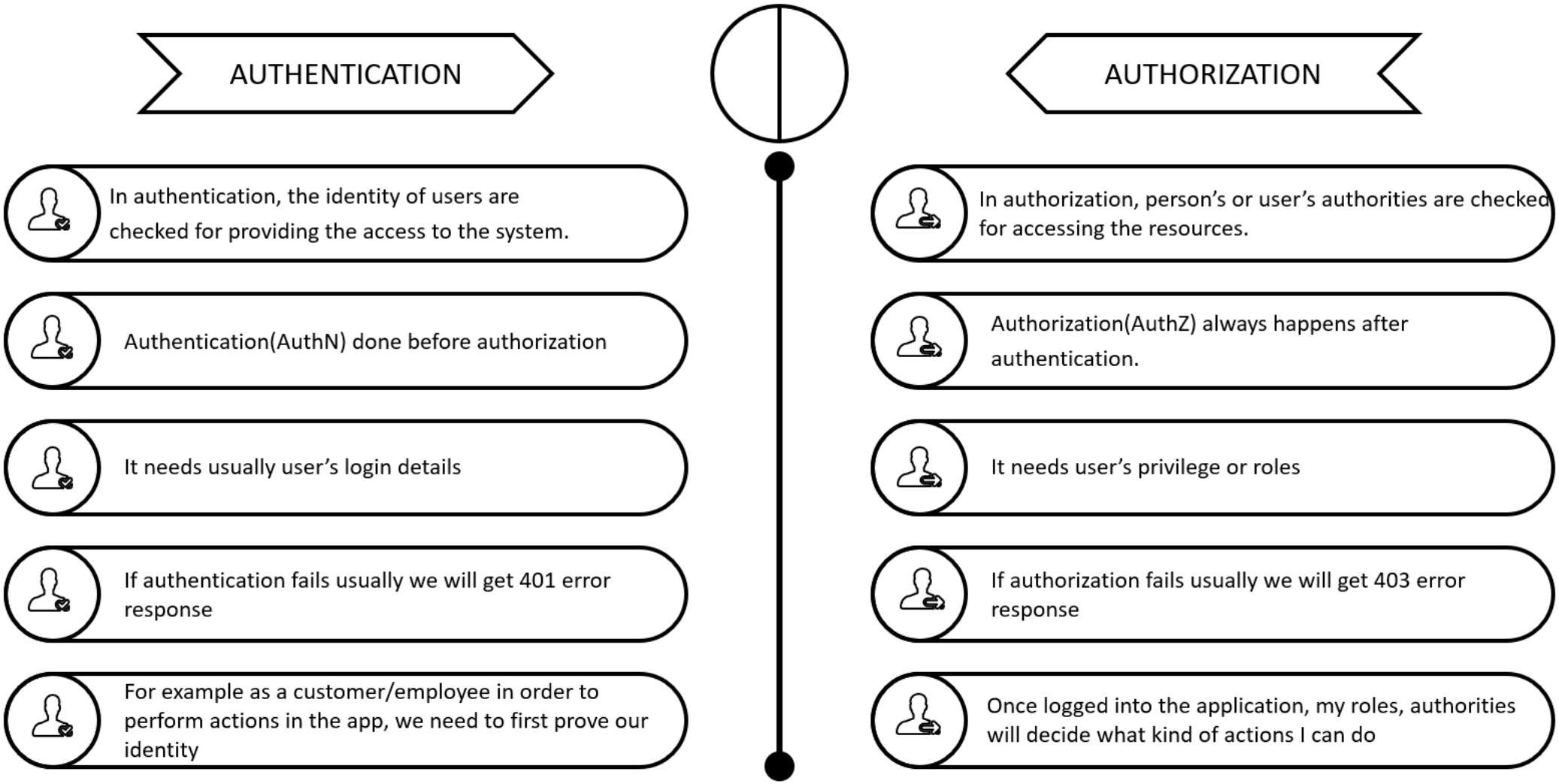
These annotations act as containers for grouping related queries together.

```
@NamedQueries({
    @NamedQuery(
        name = "EntityName.queryOne",
        query = "JPQL_QUERY_1"
    ),
    @NamedQuery(
        name = "EntityName.queryTwo",
        query = "JPQL_QUERY_2"
    )
})
```

```
@NamedNativeQueries({
    @NamedNativeQuery(
        name = "EntityName.nativeQueryOne",
        query = "NATIVE_SQL_QUERY_1",
        resultClass = EntityName.class
    ),
    @NamedNativeQuery(
        name = "EntityName.nativeQueryTwo",
        query = "NATIVE_SQL_QUERY_2",
        resultClass = EntityName.class
    )
})
```

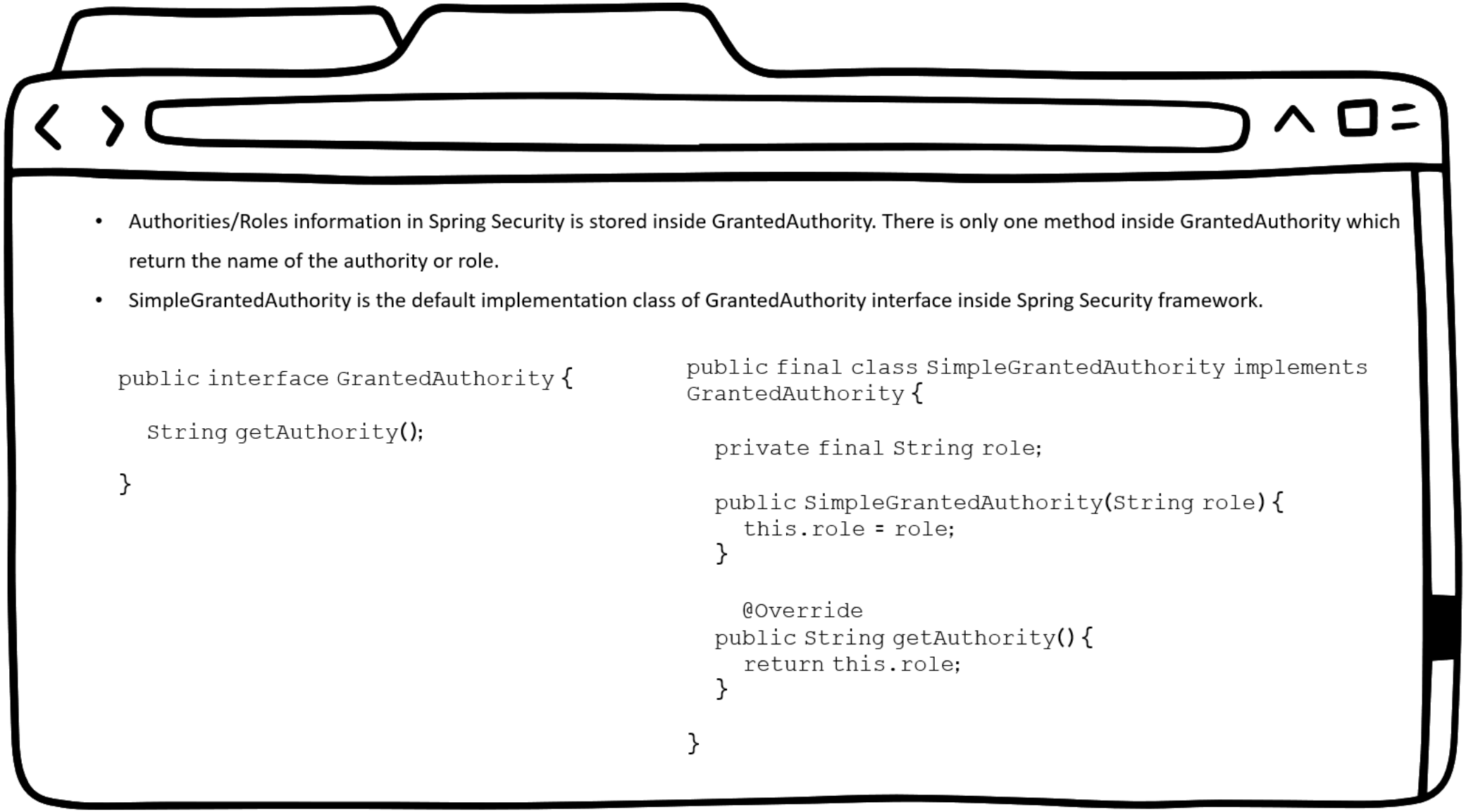
AUTHENTICATION & AUTHORIZATION

eazy bytes



HOW AUTHORITIES STORED ?

eazy bytes



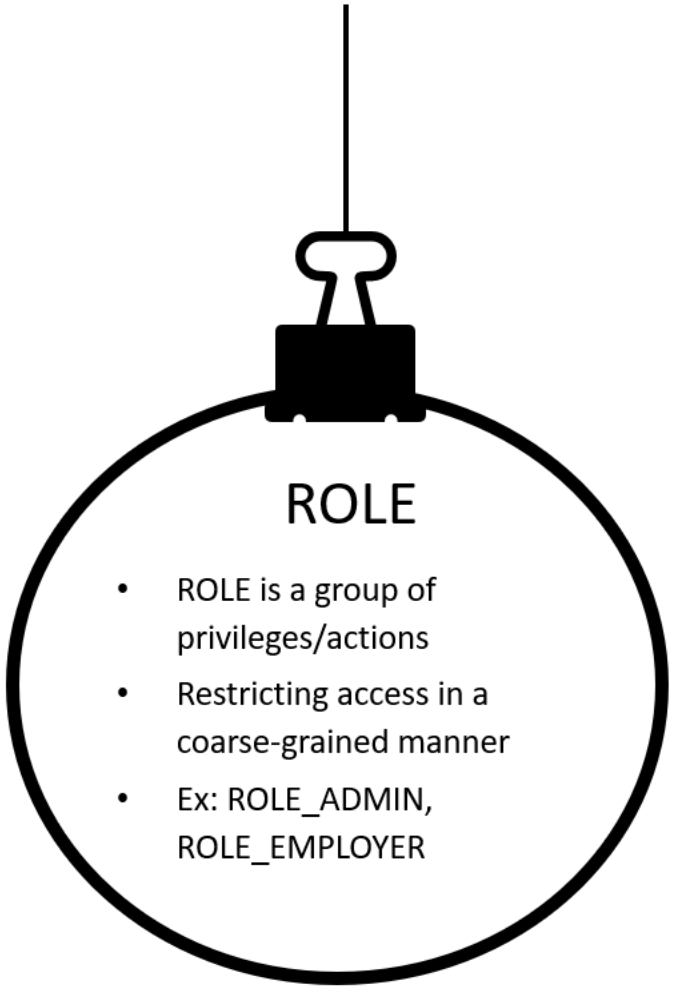
- Authorities/Roles information in Spring Security is stored inside GrantedAuthority. There is only one method inside GrantedAuthority which return the name of the authority or role.
- SimpleGrantedAuthority is the default implementation class of GrantedAuthority interface inside Spring Security framework.

```
public interface GrantedAuthority {

   String getAuthority();

}
```

```
public final class SimpleGrantedAuthority implements
GrantedAuthority {

   private final String role;

   public SimpleGrantedAuthority(String role) {
      this.role = role;
   }

      @Override
   public String getAuthority() {
      return this.role;
   }

}
```

AUTHORITY vs ROLE

eazy bytes



- The names of the authorities/roles are arbitrary in nature and these names can be customized as per the business requirement
- Roles are also represented using the same contract GrantedAuthority in Spring Security.
- When defining a role, its name should start with the ROLE_ prefix. This prefix specifies the difference between a role and an authority.

AUTHORITY vs ROLE

eazy bytes

You can configure the authorization rules to use a different prefix by providing a GrantedAuthorityDefaults bean, as shown below:

```
@Bean
static GrantedAuthorityDefaults grantedAuthorityDefaults() {

    return new GrantedAuthorityDefaults("MYPREFIX_");

}
```

You need to expose GrantedAuthorityDefaults using a static method to ensure that Spring publishes it before initializing Spring Security's method security @Configuration classes

CONFIGURING AUTHORITIES

eazy bytes

In Spring Security the authorities requirements can be configured using the following ways,

hasAuthority() — Accepts a single authority for which the endpoint will be configured and user will be validated against the single authority mentioned. Only users having the same authority configured can invoke the endpoint.

hasAnyAuthority() — Accepts multiple authorities for which the endpoint will be configured and user will be validated against the authorities mentioned. Only users having any of the authority configured can invoke the endpoint.

hasAllAuthorities() - Accepts multiple authorities for which the endpoint will be configured and user will be validated against the authorities mentioned. Only users having all of the authorities configured can invoke the endpoint.

access() — Using Spring Expression Language (SpEL) it provides you unlimited possibilities for configuring authorities which are not possible with the above methods. We can use operators like OR, AND inside access() method.

CONFIGURING AUTHORITIES

eazy bytes

You can extract path values from the request, as seen below by using access():

```
http.authorizeHttpRequests((authorize) -> authorize
        .requestMatchers("/profile/{name}").access(new
     WebExpressionAuthorizationManager("#name == authentication.name")))
```

You can check if the user has multiple roles, as seen below by using access():

```
.requestMatchers("/admin/**").access(allOf(hasAuthority(”admin"), hasAuthority(”manager"))
```

You can check if the user has multiple roles, as seen below by using access():

```
.requestMatchers("/admin/**").access(new WebExpressionAuthorizationManager("
hasAuthority('admin') && hasAuthority('manager')"))
```

CONFIGURING AUTHORITIES

eazy bytes

Like shown below, we can configure authority requirements for the APIs/Paths.

```
.authorizeHttpRequests(requests -> {

  publicPaths.forEach(path ->
      requests.requestMatchers(path).permitAll());

  adminPaths.forEach(path ->
      requests.requestMatchers(path).hasAuthority("ROLE_ADMIN"));

  securedPaths.forEach(path ->
      requests.requestMatchers(path).authenticated());

  requests.anyRequest().denyAll();
})
```

CONFIGURING ROLES

eazy bytes

In Spring Security the ROLES requirements can be configured using the following ways,

hasRole() — Accepts a single role name for which the endpoint will be configured and user will be validated against the single role mentioned. Only users having the same role configured can invoke the endpoint.

hasAnyRole() — Accepts multiple roles for which the endpoint will be configured and user will be validated against the roles mentioned. Only users having any of the role configured can call the endpoint.

hasAllRoles() — Accepts multiple roles for which the endpoint will be configured and user will be validated against the roles mentioned. Only users having all of the roles configured can call the endpoint.

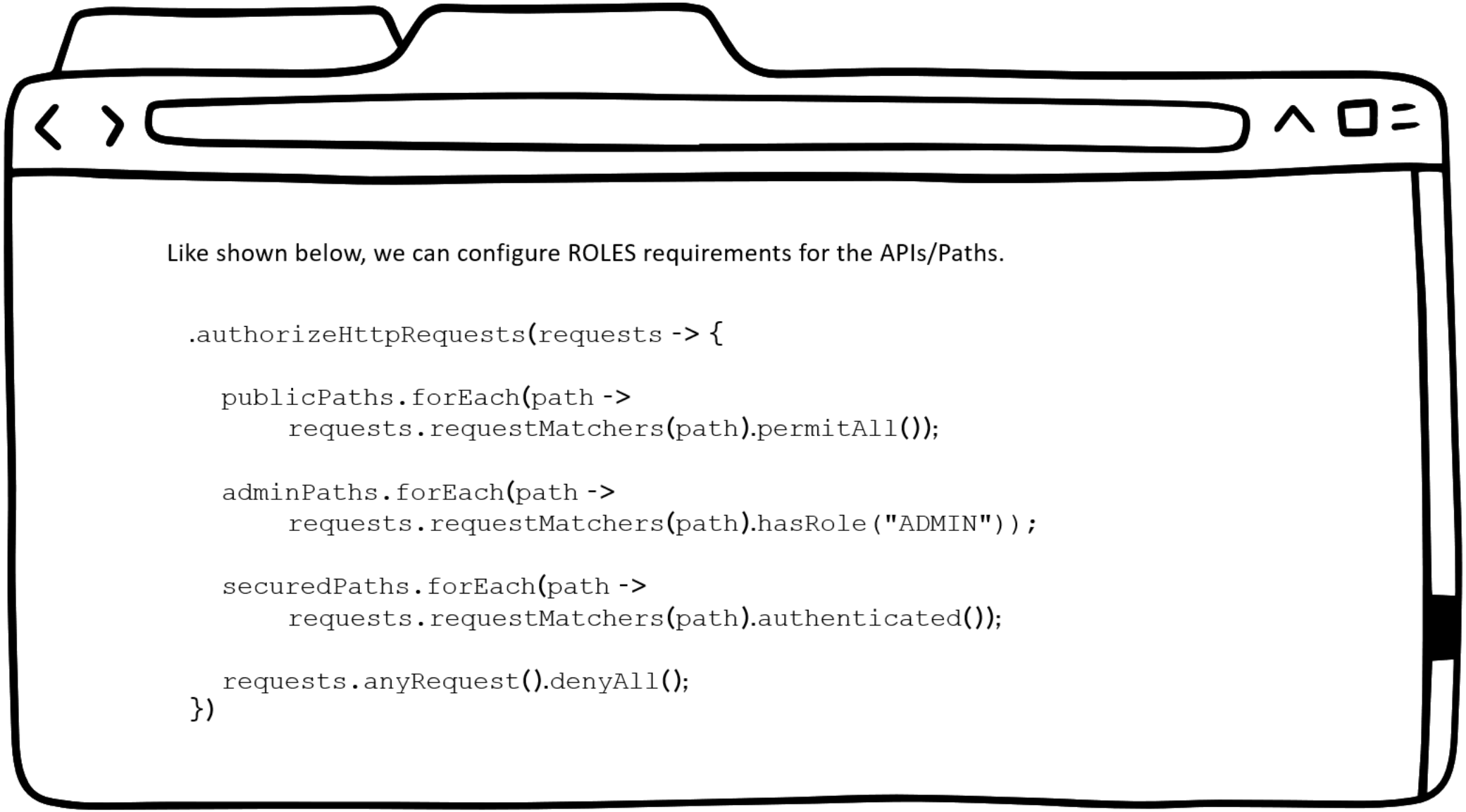
access() — Using Spring Expression Language (SpEL) it provides you unlimited possibilities for configuring roles which are not possible with the above methods. We can use operators like OR, AND inside access() method.

Note :

- ROLE_ prefix only to be used while configuring the role in DB. But when we configure the roles, we do it only by its name.

CONFIGURING ROLES

eazy bytes



Like shown below, we can configure ROLES requirements for the APIs/Paths.

```
.authorizeHttpRequests(requests -> {

   publicPaths.forEach(path ->
        requests.requestMatchers(path).permitAll());

   adminPaths.forEach(path ->
        requests.requestMatchers(path).hasRole("ADMIN"));

   securedPaths.forEach(path ->
        requests.requestMatchers(path).authenticated());

   requests.anyRequest().denyAll();
})
```

Customizing Security Errors in Spring Security

eazy bytes

What is ExceptionTranslationFilter ?

A core Spring Security filter

- Catches security-related exceptions
- Translates them into proper HTTP responses
- Sits before your controller layer

What Exceptions Does It Handle?

Exception	Meaning
AuthenticationException	User is not authenticated
AccessDeniedException	User is authenticated but not authorized

Controllers and **@ControllerAdvice** never see these exceptions. They are handled inside the Security filter chain

Customizing Security Errors in Spring Security

eazy bytes

AuthenticationEntryPoint (401 – Who Are You?)

AuthenticationEntryPoint triggered when User is not authenticated but tries to access a secured resource. Typical Scenarios

- Missing JWT token
- Invalid JWT token
- Expired JWT token

What Happens when anonymous user accessing protected API ?

Spring Security returns **HTTP 401 – Unauthorized** .We can customize this by writing our own **AuthenticationEntryPoint** implementation logic,

```
http.exceptionHandling(exception -> exception
        .authenticationEntryPoint((request, response, authException) -> {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json");
            response.getWriter().write("{\"error\": \"Unauthorized\", \"message\": \"Authentication
                    required\"}");
        })
)
```

Customizing Security Errors in Spring Security

eazy bytes

AccessDeniedHandler (403 – You Can't Do That)

AccessDeniedHandler logic will be triggered when User is authenticated but does not have required role/authority
Typical Scenarios

- USER role accessing ADMIN API
- Missing required authority
- Role mismatch (hasRole("ADMIN") fails)

What Happens when less previliged user accessing protected API ?

Spring Security returns **HTTP 403 – Forbidden** .We can customize this by writing our own **AccessDeniedHandler** implementation logic,

```
http.exceptionHandling(exception -> exception
 .accessDeniedHandler((request, response, accessDeniedException) -> {
    response.setStatus(HttpServletResponse.SC_FORBIDDEN);
    response.setContentType("application/json");
    response.getWriter().write("{\"error\": \"Access Denied\", \"message\": \"You don't have permission
                to access this resource\"}");
})
```

Sorting data with Spring Data JPA

eazy bytes

By default, database results are like a pile of laundry — you might get socks first, then a shirt, then a hat. There is no guaranteed order unless you explicitly ask for one.

To organize this data, Spring Data JPA provides a built-in “organizer” called the Sort object. It also offers default sorting and pagination support through the **PagingAndSortingRepository** interface, allowing results to be returned in a predictable and meaningful order.

Static Sorting using method names

Static sorting means:

- Sort order is fixed at compile time
- Cannot be changed dynamically by the user
- Sorting is defined inside the repository method name or query

Best when sorting rule is always the same

1) Spring Data JPA allows sorting directly in method names.

```
List<Customer> findByStatusOrderByNameAsc(String status);
```

What this means: Filter by status, Sort results by name in Ascending order. No Sort object required.

Sorting data with Spring Data JPA

eazy bytes

2) Always returns latest customers first, Sorting rule is hard-coded
Very common for:Recent orders, Latest transactions, New users list

```
List<Customer> findByStatusOrderByCreatedAtDesc(String status);
```

3) Static Sorting with @Query

```
@Query("SELECT c FROM Customer c WHERE c.status = :status ORDER BY c.name ASC")
List<Customer> findActiveCustomersSorted(@Param("status") String status);
```

Sorting defined inside JPQL. Fixed and predictable
Useful when: Query logic is complex, Method name becomes too long

Use static sorting when:

- Business rule never changes
- Same sort order is used everywhere
- Simplicity is more important than flexibility

Sorting data with Spring Data JPA

eazy bytes

Dynamic Sorting using Sort

Sometimes you want the user to choose how to sort (e.g., clicking a column header). This is where the Sort object shines. You tell:

- Which field to sort by
- Ascending or Descending
- JPA automatically creates the ORDER BY query

1) Basic Sorting Example

```
Sort sort = Sort.by("name");
List<Customer> customers = customerRepository.findAll(sort);
```

What happens?

- Data is sorted by name
- Default order = Ascending (A → Z)

2) Ascending vs Descending

```
Sort sortAsc = Sort.by("createdAt").ascending();
Sort sortDesc = Sort.by("createdAt").descending();
```

Sorting data with Spring Data JPA

eazy bytes

3) Sorting by Multiple Fields

```
Sort sort = Sort.by(
    Sort.Order.asc("role"),
    Sort.Order.desc("createdAt")
);
```

Meaning: First sort by role. If role is same, sort by created date

4) Sorting with Custom Query Methods

```
List<Customer> findByStatus(String status, Sort sort);
```

Usage:

```
Sort sort = Sort.by("name");
repository.findByStatus("ACTIVE", sort);
```

Filtering + Sorting together = powerful combination

Sorting data with Spring Data JPA

eazy bytes

Example code

```
@GetMapping("/sort/admin")
public ResponseEntity<List<ContactResponseDto>> fetchNewContactMsgsWithSort(
        @RequestParam(defaultValue = "createdAt") String sortBy,
        @RequestParam(defaultValue = "asc") String sortDir) {
    List<ContactResponseDto> contactResponseDtos = contactService
            .fetchNewContactMsgsWithSort(sortBy, sortDir);
    return ResponseEntity.status(HttpStatus.OK).body(contactResponseDtos);
}

public List<ContactResponseDto> fetchNewContactMsgsWithSort(String sortBy, String sortDir) {
    // Create Sort object based on sortBy and sortDir parameters
    Sort sort = sortDir.equalsIgnoreCase("desc")
            ? Sort.by(sortBy).descending()
            : Sort.by(sortBy).ascending();
    List<Contact> contacts = contactRepository.findContactsByStatus(
            ApplicationConstants.NEW_MESSAGE, sort);
    List<ContactResponseDto> responseDtos = contacts.stream()
            .map(this::transformToDto)
            .collect(Collectors.toList());
    return responseDtos;
}

public interface ContactRepository extends JpaRepository<Contact, Long> {
    List<Contact> findContactsByStatus(String status, Sort sort);
}
```

Pagination with Spring Data JPA

eazy bytes

Imagine an application with 50,000 records. Loading everything at once is:

- Slow
- Memory-heavy
- Bad for user experience

Users usually want data page by page, not all at once. Pagination breaks large data into smaller, manageable pages.

Think of pagination like a book:

- You don’t read all pages at once
- You read one page at a time
- You move to the next page when needed

Pagination works the same way with database records.

Spring Data JPA provides:

Pageable → tells which page & how many records
PageRequest → implementation of Pageable
Page<T> → response containing data + metadata

No need to write LIMIT and OFFSET manually.

Pagination with Spring Data JPA

eazy bytes

Basic Pagination Example

```
PageRequest pageRequest = PageRequest.of(0, 10);
Page<Customer> page = customerRepository.findAll(pageRequest);
```

Meaning: Page number = 0 (first page), Page size = 10 records per page
Page index starts from 0 (very important!)

Understanding Page<T>

Page<T> is more than just a list. It provides methods like:

- `getContent()` → actual data
- `getTotalElements()` → total records
- `getTotalPages()` → total pages
- `hasNext()` / `hasPrevious()`

Very useful for building UI pagination controls.

Pagination with Spring Data JPA

eazy bytes

Pagination + Sorting (Real-World Use)

```
PageRequest pageRequest =
    PageRequest.of(0, 10, Sort.by("createdAt").descending());
```

Fetch first 10 records. Sorted by latest first
Pagination + Sorting = best practice

Pagination with Custom Finder Methods

```
Page<Customer> findByStatus(String status, Pageable pageable);
```

Usage:

```
PageRequest pageRequest = PageRequest.of(1, 5);
repository.findByStatus("ACTIVE", pageRequest);
```

Pagination with Spring Data JPA

eazy bytes

Example code

```
@GetMapping("/page/admin")
public ResponseEntity<Page<ContactResponseDto>> fetchNewContactMsgsWithPaginationAndSort(
    @RequestParam(defaultValue = "0") int pageNumber,
    @RequestParam(defaultValue = "10") int pageSize,
    @RequestParam(defaultValue = "createdAt") String sortBy,
    @RequestParam(defaultValue = "asc") String sortDir) {
    Page<ContactResponseDto> contactResponseDtoPage = contactService
        .fetchNewContactMsgsWithPaginationAndSort(pageNumber, pageSize, sortBy, sortDir);
   return ResponseEntity.status(HttpStatus.OK).body(contactResponseDtoPage);
}
public Page<ContactResponseDto> fetchNewContactMsgsWithPaginationAndSort(
    int pageNumber, int pageSize, String sortBy, String sortDir) {
        Sort sort = sortDir.equalsIgnoreCase("desc")
                ? Sort.by(sortBy).descending()
                : Sort.by(sortBy).ascending();
    Pageable pageable = PageRequest.of(pageNumber, pageSize, sort);
    Page<Contact> contactPage = contactRepository.findContactsByStatus(
        ApplicationConstants.NEW_MESSAGE, pageable);
    Page<ContactResponseDto> responseDtoPage = contactPage.map(this::transformToDto);
    return responseDtoPage;
}

public interface ContactRepository extends JpaRepository<Contact, Long> {
    Page<Contact> findContactsByStatus(String status, Pageable pageable);
}
```

Fetching current logged in user for Auditing

eazy bytes

Example code

```
public class ApplicationUtility {

    public static String getLoggedInUser() {
        Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();

        if (authentication == null || !authentication.isAuthenticated() ||
                authentication.getPrincipal().equals("anonymousUser")) {
            return ApplicationConstants.SYSTEM;
        }
        Object principal = authentication.getPrincipal();
        String username;
        if (principal instanceof JobPortalUser jobPortalUser) {
            username = jobPortalUser.getEmail();
        } else {
            username = principal.toString(); // fallback
        }
        return username;
    }
}
```

@Transactional in Spring Data JPA

eazy bytes

Imagine you're transferring money from your savings to your checking account at a bank. You don't want half the process to happen—like deducting from savings but forgetting to add to checking—leaving you broke and confused. That's where transactions come in: they're like an all-or-nothing promise. In databases, a transaction ensures a group of operations (like reads and writes) either all succeed or all fail together, keeping your data consistent and reliable. This follows the ACID principles: Atomicity (all or nothing), Consistency (rules are followed), Isolation (transactions don't mess with each other), and Durability (changes stick after commit).

In Spring (and especially Spring Data JPA, which handles database interactions via Java Persistence API), the **@Transactional** annotation is your magic wand for managing these transactions declaratively. No need to write boilerplate code like below — Spring handles it behind the scenes using proxies and aspects

```
public class Example {

    public void executeTransaction() {
        // Get a database connection
        Connection conn = dataSource.getConnection();

        try (conn) {
            // Start the transaction
            connection.setAutoCommit(false);

            // Execute SQL statements here

            // Commit the transaction
            connection.commit();
        } catch (Exception e) {
            // Roll back the transaction in case of an error
            conn.rollback();
        }
    }
}
```

@Transactional in Spring Data JPA

eazy bytes

Enabling @Transactional

To use **@Transactional**, you need to tell Spring to turn on transaction management:

In a plain Spring app, add **@EnableTransactionManagement** to your configuration class.
In Spring Boot (which is common with Spring Data JPA), it's auto-enabled if you have spring-data-jpa or spring-tx in your dependencies. Boom, ready to go!

You'll also need a **PlatformTransactionManager** bean, like **JpaTransactionManager** for JPA setups. Spring Boot sets this up automatically.

Without this, **@Transactional** won't do squat.

Where and How to Slap on @Transactional

You can put **@Transactional** on:

Classes: Applies to all public methods in that class (great for services).
Methods: Overrides class-level settings for fine control.
Interfaces: But it's better on concrete classes or methods for clarity.

Pro tip: Stick it on service-layer methods, not directly on repositories unless you're overriding defaults.

@Transactional in Spring Data JPA

eazy bytes

Spring Data JPA builds on Spring's transaction system but adds repo-specific defaults.

Default Behavior in Repositories:

CrudRepository methods (like save, findAll) are transactional out-of-the-box via SimpleJpaRepository.

- Reads: @Transactional(readOnly = true).
- Writes: Plain @Transactional.

Custom Queries: No default transaction — add @Transactional yourself. For modifying queries (@Modifying), you need it without readOnly.

Overriding: Redeclare methods in your repo interface with @Transactional for custom props (e.g., timeout).

Service Layer is King: For multi-repo ops (e.g., update User and Order together), put @Transactional on the service method. This wraps everything in one tx, overriding repo defaults if needed.

JPA Nuances: readOnly is an optimization hint (e.g., Hibernate skips flushes). But it's not enforced—writes might still slip through. Use it for perf in read-heavy spots.

@Transactional in Spring Data JPA

eazy bytes

Why @Transactional(readOnly = true) Boosts Performance in Spring Data JPA

Setting readOnly = true tells Hibernate: "Hey, I'm just reading—no updates!" So it skips a bunch f background works. Here's how it supercharges performance, step by step:

1. Skips Dirty Checking Entirely

Normally, Hibernate takes a "snapshot" (hydrated state) of every loaded entity to compare later for changes. With readOnly = true, entities are loaded in read-only mode: No snapshots stored, no dirty checks during flush/commit.
Big win: Faster CPU usage, especially if you're loading hundreds or thousands of entities (e.g., large reports or lists). No iterating over entities to compare states.

2. Reduces Memory Usage

Those snapshots? They can eat a surprising amount of heap space in big queries.
Read-only mode discards the loaded-state snapshots completely.
Result: Lower memory footprint per transaction—great for high-concurrency apps

3. Optimizes Flushing

Sets Hibernate's FlushMode to MANUAL. No automatic flushing before queries or at commit (since nothing to flush!).

@Transactional in Spring Data JPA

eazy bytes

4. Database-Level Hints

Spring passes the read-only flag to the JDBC connection (if your driver/DB supports it). Databases like PostgreSQL, MySQL, or Oracle can:

- Use lighter locking (e.g., no row locks, or shared locks only).
- Avoid unnecessary transaction overhead.
- Sometimes reject accidental writes (safety net!).

Not all DBs optimize equally, but many do—fewer resources used.

Example

Imagine fetching 10,000 entities:

Without readOnly: Hibernate stores 10,000 snapshots → heavy memory + slow flush checks.

With readOnly: No snapshots, no checks → quicker execution, less GC pressure.

Best Practice: Put @Transactional(readOnly = true) at the service class level (default for all methods), then override with plain @Transactional for write methods. Spring Data JPA already does this for repo read methods!

When It Shines Most: Read-heavy services, reporting queries, or when loading large object graphs.

Not Always Needed: For tiny queries, the gain is small—but it's zero-cost to add, and future-proofs your code.

@Modifying in Spring Data JPA

eazy bytes

Spring Data JPA's derived methods (like save() or deleteBy...()) are great for single entities or small batches, but they often load entities into memory first, modify them, and save—super inefficient for bulk operations!

Enter @Modifying: It's the sidekick to @Query that lets you run direct modifying queries (UPDATE, DELETE, INSERT, even DDL like altering tables) straight on the database. No loading entities, no extra memory hogging—just pure, fast SQL/JPQL power. It's like skipping the middleman and yelling your changes directly to the database.

Why Do We Need @Modifying?

By default, Spring Data JPA assumes your @Query methods are for reading data (SELECTs).
But for writes (UPDATE/DELETE), Spring needs a heads-up: "Hey, this query changes stuff!" Without @Modifying, it'd try to treat it like a read query and boom—exception or weird behavior.

@Modifying tells Spring:

Execute this using query.executeUpdate() (returns how many rows affected) instead of fetching results. It's only for custom @Query methods. Derived deletes (like deleteByActiveFalse()) or custom impls don't need it—they're already "modifying."

The Must-Have Buddy: @Transactional

Modifying queries always need a transaction—otherwise, changes won't commit (or might error).
Best practice: Add @Transactional on the method (or service calling it). Repo reads often inherit readOnly=true, so override for writes:

@Modifying in Spring Data JPA

eazy bytes

Example:

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    @Modifying
    @Query("UPDATE User u SET u.active = false WHERE u.lastLogin < :cutoffDate")
    int deactivateOldUsers(@Param("cutoffDate") LocalDateTime cutoffDate);

}
```

Superpowers: clearAutomatically and flushAutomatically

flushAutomatically = true (default: false):
Forces a flush (push pending changes to DB) before running your modifying query. Why? If you've modified entities in memory but not flushed, your bulk query might miss or conflict with them.
Analogy: Clean the table before spilling new paint.

clearAutomatically = true (default: false):
Clears the persistence context after the query. Why? Bulk queries bypass the cache—your in-memory entities might now be stale (out of sync with DB). Next read could return old data!
Analogy: Wipe the slate clean so future reads hit the fresh DB state.

Transaction Propagation

eazy bytes

Propagation answers one question:

“What should happen if a transactional method is called when another transaction already exists?”

1) REQUIRED (Default & Most Used)

```
@Transactional
// or
@Transactional(propagation = Propagation.REQUIRED)
placeOrder() {
   saveOrder();
   saveOrderItems();
}
```

Meaning - “If a transaction already exists, join it. If not, start a new one.”

One transaction

- All methods succeed → commit
- Any failure → rollback everything

Why it’s default
This is what developers expect 90% of the time

Transaction Propagation

eazy bytes

2) SUPPORTS (Optional Transaction)

```
@Transactional(propagation = Propagation.SUPPORTS)
getUserProfile()
```

Meaning - “If a transaction exists, use it. If not, run without a transaction.”
Works with or without a transaction. Common for read-only operations

Not the same as “no transaction at all”. Resources may still be shared if synchronization is enabled

3) MANDATORY (Strict Rule Enforcer)

```
@Transactional(propagation = Propagation.MANDATORY)
debitWallet()
```

Meaning - “A transaction must already exist. If not → throw exception.”

Debit must be part of checkout. Calling it alone is illegal

Transaction Propagation

eazy bytes

4) REQUIRES_NEW (Independent Transaction)

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
logAudit();
```

Meaning - “Pause the current transaction. Start a brand-new transaction.”

Even if main flow fails: Audit log → should be committed.
Perfect for Audit logs, Login attempts, Failure history and Notifications

5) NOT_SUPPORTED (No Transaction Zone)

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
generateLargeReport()
```

Meaning - “Run without any transaction. If one exists, suspend it.”

No locking, Better performance, No rollback support
But Use carefully, No transaction = no safety net

Transaction Propagation

eazy bytes

6) NEVER (Transaction Forbidden)

```
@Transactional(propagation = Propagation.NEVER)
externalApiCall()
```

Meaning - “If a transaction exists → throw exception”

Must not run inside DB transaction. Prevents accidental long locks
Rare but powerful. Mostly used for safety constraints

7) NESTED (Savepoints – Advanced)

```
@Transactional(propagation = Propagation.NESTED)
processItems() {
   processItem1();
   processItem2(); // fails
}
```

Meaning - “Create a sub-transaction inside the main one.”
Analogy - Checkpoint in a video game. If you fail → restart from checkpoint, not beginning.

Item 2 rolls back, Item 1 stays, Outer transaction continues

Transaction Propagation

eazy bytes

Summary table

Propagation	Existing Tx	New Tx	Typical usecase
REQUIRED	Join	If none	Default business logic
SUPPORTS	Join	No	Read-only
MANDATORY	Must exist	No	Strict rules
REQUIRES_NEW	Suspend	Yes	Audit / logs
NOT_SUPPORTED	Suspend	No	Reports
NEVER	Error	No	Safety
NESTED	Savepoint	Conditional	Partial rollback

Transaction Isolation

eazy bytes

In a busy app, hundreds of users are changing the database at the same time. If they interfere with each other, data gets messy. Isolation levels define the "walls" between these users. The Trade-off:

- Low Isolation: Faster performance, but high risk of "weird" data.
- High Isolation: Perfect data accuracy, but slower performance (the database has to wait and lock things).

The Three "Data Nightmares"

Before we learn the levels, we need to know what we are trying to prevent:

Dirty Read: Reading data that another person changed but hasn't "Saved" (Committed) yet. If they undo their change, your data is now fake.

Non-Repeatable Read: You read a row, someone else updates it, you read it again, and it’s different! It’s like a book changing its text while you’re reading it.

Phantom Read: You count 10 items, someone else adds a new one, you count again, and suddenly there are 11. A "phantom" appeared!

Transaction Isolation

eazy bytes

1) Level 1 - READ_UNCOMMITTED -> "The Wild West"

```
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
```

What it does: Allows you to see changes the moment they are typed, even if they aren't saved.

Vulnerability: You will see Dirty Reads.

When to use: Rarely. Only when speed is everything and accuracy doesn't matter
(e.g., a "Likes" counter on a post where being off by one isn't a disaster).

2) Level 2 - READ_COMMITTED -> "The Industry Standard"

```
@Transactional(isolation = Isolation.READ_COMMITTED)
```

What it does: You can only see data that has been officially "Committed" (Saved).

The Benefit: No more Dirty Reads!
The Catch: You might still see Non-Repeatable Reads if someone saves a change while you are in the middle of your work.

When to use: This is the default for most databases (like PostgreSQL and SQL Server).

Transaction Isolation

eazy bytes

3) Level 3 - REPEATABLE_READ -> "The Snapshot"

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
```

What it does: Once you read a row, the database "freezes" it for you. Even if someone else updates it, you keep seeing the version you started with.

The Benefit: No Dirty Reads. No Non-Repeatable Reads.
The Catch: You might still see Phantom Reads (new rows appearing).

When to use: Financial reports where you need the numbers to stay consistent while you calculate them.
(Default for MySQL).

The "Snapshot" Concept

When your transaction starts a REPEATABLE_READ session and performs its first read, the database essentially takes a mental picture of the data as it exists at that exact millisecond.

Your View: For the rest of your transaction, you are looking at that "picture."

The Reality: Other users can still change the "live" data. They can update rows and delete them.

The Result: If you query the same row ten times in the same transaction, you will see the exact same values every single time, even if another user changed and "saved" that row in the meantime.

Transaction Isolation

eazy bytes

How Snapshot works: Versioning

The database doesn't actually "freeze" the original row; it creates versions. Imagine a row representing a Bank Balance: ID: 1 | Balance: $100.

Transaction A starts (Repeatable Read) and reads the balance. It sees $100.
Transaction B starts, updates the balance to $200, and Commits (saves).

The database now technically has two versions of this row in its storage:

Old Version: $100 (kept for Transaction A)
Current Version: $200 (for everyone else)

Transaction A reads again. The database checks Transaction A's "start time" and says,
"Oh, you belong to the $100 era. Here you go." Transaction A sees $100 again.

The Difference Between Levels

To really "get" this, compare it to the level below it:

In READ_COMMITTED: The database gives you the latest "Saved" version at the moment you ask. If Transaction B saves a change mid-way through your work, your second read will show the new value. (The "Update" sneaks in).

In REPEATABLE_READ: The database gives you the version that was "Saved" at the moment your transaction began.
(The "Update" is ignored by you).

While REPEATABLE_READ freezes the existing rows you've looked at, it doesn’t freeze the entire table.This resulted into a Weakness called Phantoms

Transaction Isolation

eazy bytes

4) Level 4 - SERIALIZABLE -> "The Fort Knox"

```
@Transactional(isolation = Isolation.SERIALIZABLE)
```

What it does: Transactions happen one by one. It’s like a single-file line at a grocery store.

The Benefit: Perfect data. Zero nightmares.
The Catch: It is very slow because transactions have to wait for each other.

When to use: Only for extremely critical tasks where 100% accuracy is required, like a complex banking transfer involving multiple accounts.

To achieve this, the database often places "Range Locks" on the data. If you query "all users between age 20 and 30," the database locks that entire range so no one can even add a new user until you are done.

Imagine a complex banking operation that needs to:

Check the total balance across five different accounts. Ensure the sum is exactly $1,000,000.
Distribute a 1% bonus to all five accounts. Verify the sum is now exactly $1,010,000.

If another transaction adds a new account or moves money between these accounts while you are calculating, your math will be wrong. Using @Transactional(isolation = Isolation.SERIALIZABLE) ensures that for those few milliseconds, the world stops so your math stays perfect.

Transaction Isolation

eazy bytes

DEFAULT – Let the Database Decide

```
@Transactional(isolation = Isolation.DEFAULT)
```

Use whatever isolation level the database is configured with. For example,

- MySQL → REPEATABLE_READ
- PostgreSQL → READ_COMMITTED
- Oracle → READ_COMMITTED

Spring is not magic here — it respects the DB settings.

Summary table

Isolation Level	Dirty Reads	Non-Repeatable	Phantoms	Speed
READ_UNCOMMITTED	Allowed	Allowed	Allowed	Fastest
READ_COMMITTED	Blocked	Allowed	Allowed	Fast
REPEATABLE_READ	Blocked	Blocked	Allowed	Moderate
SERIALIZABLE	Blocked	Blocked	Blocked	Slowest

The "Gotchas" (Common Pitfalls) of @Transactional

eazy bytes

1. Self-Invocation: If you call a @Transactional method from another method inside the same class, the "Proxy" is bypassed, and the transaction won't start.

What Happens: Calling a @Transactional method from within the same class skips the proxy entirely—no transaction starts!
Why?: Spring wraps your bean in a proxy to intercept calls and manage tx. Internal calls (this.method()) go straight to the target object, bypassing the proxy.

```
@Service
public class UserService {

    @Transactional
    public void createUser(User user) {
        saveUser(user);  // Transactional!
        this.setupProfile(user);  // NO transaction—self-call!
    }

    @Transactional
    private void setupProfile(User user) { ... }  // Ignored anyway (private)
}
```

2. Non-Public Methods (Private/Protected)

What Happens: @Transactional ignored on private/protected/final/static methods.
Why?: Proxies can't override them properly (JDK proxies need interface/public; CGLIB needs non-final).
Fix: Keep transactional methods public. Spring 5+ improved protected support with CGLIB, but stick to public.

The "Gotchas" (Common Pitfalls) of @Transactional

eazy bytes

3. Rollback Rules Misunderstood

Default: Rollback only on unchecked exceptions (RuntimeException/Error). Checked exceptions (e.g., IOException) commit!
What Happens: You throw a checked exception expecting rollback—data commits anyway.
Example: Throwing SQLException (checked) → no rollback.
Fix: Use rollbackFor = MyCheckedException.class or throw unchecked wrappers.

```
@Transactional(rollbackFor = Exception.class)  // Rollback on everything
public void riskyOp() { ... }
```

Opposite: Use noRollbackFor to ignore specific ones.

4. Catching Exceptions Inside the Method

What Happens: If you catch and swallow an exception, Spring never sees it—no rollback!
Fix: Let exceptions propagate, or rethrow after logging.Java@Transactional

```
public void process() {
    try { ... } catch (Exception e) { log.error(e); }  // Bad: No rollback!
    // Better: log and throw new MyBusinessException(e);
}
```

Caching in Spring Boot

eazy bytes

What is Caching?

Caching means storing frequently used data in memory. So next time, the application can reuse the data instead of recalculating or re-fetching it from DB. This improves performance of the application to a great extent.

In Spring, caching works behind the scenes using annotations. The caller never knows whether data came from the database / service, or the cache.

Why Do We Need Caching?

Improve Performance

- Avoid repeated database or API calls
- Faster response times

Reduce Load on Backend

- Fewer DB queries
- Less stress on microservices

Better User Experience

- Pages load faster
- APIs respond quicker

Caching in Spring Boot

eazy bytes

Enable Caching in Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

@EnableCaching
@SpringBootApplication
public class JobportalApplication { ... }
```

Common Caching Annotations

Annotation	Purpose
@Cacheable	Reads from cache if present, else fetch from DB and cache them for future call
@CachePut	Executes method & updates the cache records
@CacheEvict	Removes entry from cache or reset cache

Spring Caching with @Cacheable, @CachePut, @CacheEvict

eazy bytes

Example1: Cache complex logic result

First call → method executes
Next call with same input → cache is used

```
@Cacheable("piDecimals")
public int computePiDecimal(int precision) {
    // costly calculation
}
```

Example2: Cache static DB results

Perfect for reference tables like roles, countries, categories, etc.
We often need to fetch role data like "ROLE_JOB_SEEKER" from the DB:

```
roleRepository.findRoleByName("ROLE_JOB_SEEKER");
```

These roles are static. Instead of hitting the DB every time, we can cache the result.

```
@Cacheable("roles")
Optional<Role> findRoleByName(String name);
```

Spring Caching with @Cacheable, @CachePut, @CacheEvict

eazy bytes

Example code of using caching related annotations

```
public interface RoleRepository extends JpaRepository<Role, Long> {

    //  Caches the role by name on first fetch
    @Cacheable("roles")
    Optional<Role> findRoleByName(String name);

    //  Updates cache when role is saved/updated
    @CachePut(value = "roles", key = "#role.name")
    Role save(Role role);

    //  Evicts role from cache when deleted by name
    @CacheEvict(value = "roles", key = "#name")
    void deleteRoleByName(String name);

    //  Optional: Clear all role cache entries
    @CacheEvict(value = "roles", allEntries = true)
    void deleteAll();  // Only if appropriate
}
```

Tips for Effective Caching

- Use @Cacheable on read-only or static lookup methods.
- Use @CachePut when updating records and want to sync cache.
- Use @CacheEvict when deleting or modifying the cached data.
- Avoid using @Cacheable on write/update operations

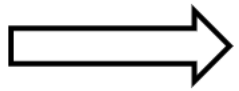
Spring Caching with @Cacheable, @CachePut, @CacheEvict

eazy bytes

How Does Spring Decide the Cache Key?

When you use caching annotations like @Cacheable, Spring must decide: “Under what key should I store and retrieve this data?” Spring follows the following rules for key generation.

Case 1: Method has parameters



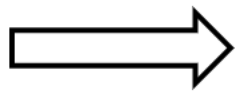
Cache Key = method parameters. Key = id

```
@Cacheable("users")
public User getUserById(Long id) {
    ...
}
```

- getUserById(1) → key = 1
- getUserById(2) → key = 2

Each unique parameter value creates a separate cache entry

Case 2: Method has multiple parameters



Cache Key = combination of all parameters

```
@Cacheable("users")
public User getUser(String email, String role) {
    ...
}
```

Internally, Spring creates a compound key: (email, role)

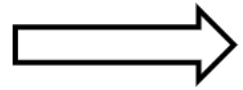
Example: ("a@x.com", "ADMIN"), ("a@x.com", "USER")

Same email + different role → different cache entry

Spring Caching with @Cacheable, @CachePut, @CacheEvict

eazy bytes

Case 3: Method has NO parameters



Spring uses a SimpleKey.EMPTY

```
@Cacheable("roles")
public List<Role> getAllRoles() {
    ...
}
```

One cache entry. Method executes only once
Subsequent calls always use cache

How Spring Creates the Key Internally

Spring uses -> SimpleKeyGenerator

Rules:

- 0 params → SimpleKey.EMPTY
- 1 param → the param itself
- Multiple params → SimpleKey(param1, param2, ...)

You normally don’t see this, but it’s good to know.

Spring Caching with @Cacheable, @CachePut, @CacheEvict

eazy bytes

Custom Cache Key Using key Attribute

You can control the cache key manually.

```
@Cacheable(value = "users", key = "#id")
public User getUserById(Long id) {
    ...
}
```

Using SpEL (Spring Expression Language)

```
@Cacheable(value = "users", key = "#user.email")
public User getUser(User user) {
    ...
}
```

Common expressions:

```
#id
#email
#root.methodName
#root.args[0]
```

Why Custom Keys Matter

Bad key choice:

```
@Cacheable("users")
public User getUser(User user) {
}
```

User object may differ in memory. Cache misses occur unexpectedly

Better approach:

```
@Cacheable(value = "users", key = "#user.id")
```

Common Mistakes

- Caching entities with mutable fields
- Using full objects as cache keys
- Forgetting that same method + different params = different keys

Spring Caching with TTL(Time-To-Live) Configuration

eazy bytes

Spring Caching with Auto-Expiry

To control expiry (TTL), you'll need a real cache provider like Caffeine or Redis, because the default in-memory cache (ConcurrentMapCacheManager) doesn't support TTL.

Use Caffeine Cache (lightweight, in-memory, fast)

What is Caffeine?

- A high-performance, in-memory caching library by the makers of Guava
- Designed as a modern replacement for ConcurrentHashMap or EhCache
- Ideal for caching read-heavy, frequently accessed data

Add below maven dependency

```
<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
</dependency>
```

Spring Caching with TTL(Time-To-Live) Configuration

eazy bytes

Example code of TTL related configs using Caffeine

```
@Configuration
public class CaffeineCacheConfig {

    @Bean
    public CacheManager caffeineCacheManager() {

        CaffeineCache companiesCache = new CaffeineCache(”companies", Caffeine.newBuilder()
                        . expireAfterWrite(10, TimeUnit.MINUTES).maximumSize(500)
                        .build());
        CaffeineCache jobsCache = new CaffeineCache("jobs", Caffeine.newBuilder()
                        . expireAfterWrite(10, TimeUnit.MINUTES).maximumSize(5000)
                        .build());

        CaffeineCache rolesCache = new CaffeineCache("roles",Caffeine.newBuilder()
                        .expireAfterWrite(1, TimeUnit.DAYS).maximumSize(100)
                        .build());

        SimpleCacheManager manager = new SimpleCacheManager();
        manager.setCaches(Arrays.asList(companiesCache, jobsCache, rolesCache));
        return manager;

}
```

Spring Caching with TTL(Time-To-Live) Configuration

eazy bytes

What happens with CacheManager configurations ?

Cache name	TTL Durations	Max Entries
companies	10 Minutes	500
jobs	10 Minutes	5000
roles	1 Day	100

- Entries expire after the TTL duration from their last write
- Keeps memory usage in control
- Automatically refreshes stale data on the next call

It is also possible to define the cache configurations using properties as shown below,

```
spring.cache.type=caffeine
spring.cache.cache-names=jobs,roles
spring.cache.caffeine.spec=maximumSize=5000,expireAfterAccess=600s
```

@ManyToMany Relationship in JPA

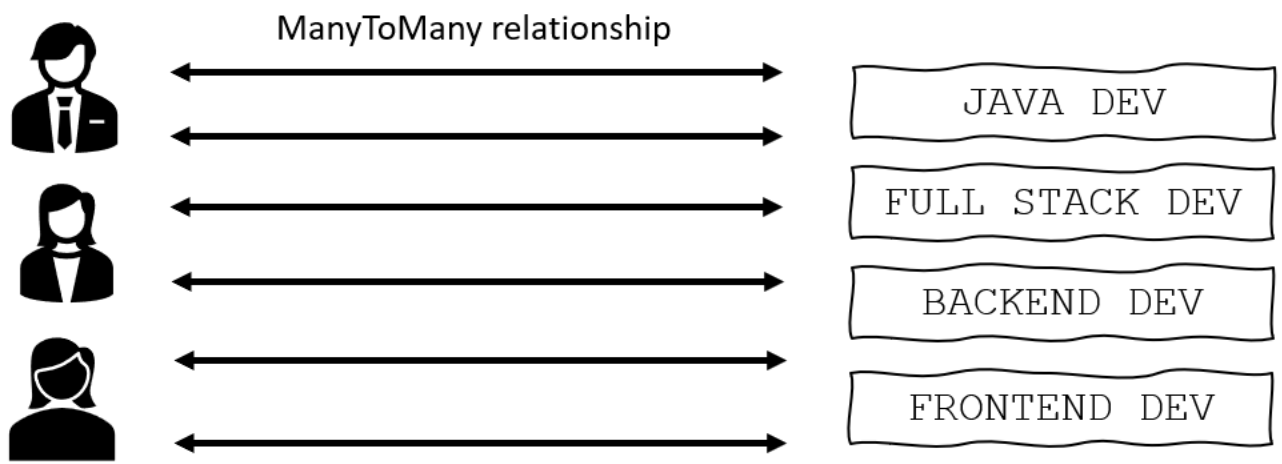
eazy bytes

What is @ManyToMany?

Used when both sides of the relationship can have multiple references to each other.

Example:

- Imagine a Music Streaming App. A single Song can be part of many different Playlists, and a single Playlist obviously contains many different Songs. This "both ways" relationship is a ManyToMany mapping.
- A user can mark multiple jobs as favorites, and a job can be favorited by multiple users.



@ManyToMany Relationship in JPA

eazy bytes

Why do we need a third table in a Many-to-Many relationship?

When two entities share a many-to-many relationship — such as user and job — it's best to use a third (join) table to represent the association.

For example, if a user has multiple favourite jobs, using a separate join table prevents duplication of data in the users or jobs tables. Without it, we'd need to store repetitive or nested data.

users Table

ID	NAME	EMAIL
12	Admin	admin@gmail.com
13	Madan	madan@gmail.com
14	Sanjana	sanjana@gmail.com

saved_jobs Table

user_id	job_id
12	1
12	2
13	1
14	4

jobs Table

ID	NAME
1	JAVA DEV
2	FULL STACK DEV
3	BACKEND DEV
4	FRONTEND DEV

@ManyToMany Relationship in JPA

eazy bytes

Why Not Use @OneToMany?

@OneToMany means a Job is marked favourite to only one User → not reusable
@ManyToMany allows shared jobs — cleaner and more flexible design

Database Design

users table – stores user info
jobs table – stores job details
saved_jobs (Join Table) – connects both with foreign keys

Key Things to Remember

Use a Set, not a List: Using a Set is generally better for performance in ManyToMany to avoid duplicate entries.

The Bridge Table is Automatic: You don't need to create a Java entity class for the saved_jobs table;
JPA handles it behind the scenes!

Don't forget the "Inverse": Always use mappedBy on one side to ensure you have a "bidirectional" relationship that stays in sync.

@ManyToMany Relationship in JPA

eazy bytes

In JPA, we use two main annotations to implement ManyToMany mapping:

@ManyToMany: This tells JPA that "many of me" relate to "many of you."
@JoinTable: This defines the "Bridge Table" name and how the IDs connect.

JobPortalUser.java (Owning Side) - User is the owning side as it controls insert/delete in saved_jobs

```
@ManyToMany
@JoinTable(name = "saved_jobs",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "job_id")
)
private Set<Job> savedJobs = new HashSet<>();
```

Job.java (Inverse Side)

```
@ManyToMany(mappedBy = "savedJobs")
private Set<JobPortalUser> savedByUsers = new HashSet<>();
```

Use Set instead of list to prevent duplicate entries as saved_jobs has composite PK.

@ManyToMany Relationship in JPA

eazy bytes

Saving a job in Service layer

```
user.getSavedJobs().add(job);
userRepository.save(user);
```

Hibernate will execute:

```
INSERT INTO saved_jobs (user_id, job_id) VALUES (?, ?)
```

Removing a job in Service layer

```
user.getSavedJobs().remove(job);
userRepository.save(user);
```

Hibernate will execute:

```
DELETE FROM saved_jobs WHERE user_id = ? AND job_id = ?
```

Fetching saved jobs for a user

```
User user = userRepository.findJobPortalUserByEmail(email).orElseThrow();
Set<Job> savedJobs = user.getSavedJobs();
```

Default fetch type is LAZY (good)

@ManyToMany Relationship in JPA

eazy bytes

Important best practice

Avoid CascadeType.ALL

```
@ManyToMany(cascade = CascadeType.ALL)
```

Why? Deleting a user could delete jobs (dangerous)

Use @ManyToMany only when the join table has no extra columns and represents a pure relationship

When using @ManyToMany is not idle

eazy bytes

@ManyToMany cannot handle extra attributes because the join table is not an entity in JPA — it's treated as an invisible link table, not something you can model or manage.

How @ManyToMany works (internally)

When you write:

```
@ManyToMany
@JoinTable(name = "saved_jobs",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "job_id")
)
private Set<Job> savedJobs = new HashSet<>();
```

JPA thinks - "This table exists only to connect User and Job." So JPA assumes: The table has only two columns user_id, job_id. No business meaning, No lifecycle of its own

The join table is not mapped as a Java class

When using @ManyToMany is not idle

eazy bytes

Why extra attributes break this model

For the Job applications scenario, your real table looks like this:

job_applications

Column name
id
user_id
job_id
applied_at
status
cover_letter
notes
created_at
created_by
updated_at
updated_by

Now ask yourself:

Where does status go? Which entity owns it?
When should it be updated?

There is no place in @ManyToMany to map status, notes & so on. Because there is no Java object representing that row.

Aspect	@ManyToMany	Join Entity
Join table as a entity class	No	Yes
Extra columns	Not supported	Fully supported
Business logic	Impossible	Easy
Auditing	No	Yes
Lifecycle control	No	Yes

When using @ManyToMany is not idle

eazy bytes

JobApplication Entity

```
@Getter @Setter
@Entity
@Table(name = "job_applications")
public class JobApplication extends BaseEntity {

    @NotNull
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JoinColumn(name = "user_id", nullable = false)
    private JobPortalUser user;

    @NotNull
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @OnDelete(action = OnDeleteAction.CASCADE)
    @JoinColumn(name = "job_id", nullable = false)
    private Job job;

    // Other columns related  fields
}
```

When using @ManyToMany is not idle

eazy bytes

Job Entity

```
@Getter @Setter
@Entity
@Table(name = "jobs")
public class Job extends BaseEntity {

    @OneToMany(mappedBy = "job")
    private Set<JobApplication> jobApplications = new LinkedHashSet<>();

}
```

JobPortalUser Entity

```
@Getter @Setter
@Entity
@Table(name = ”users")
public class JobPortalUser extends BaseEntity {

    @OneToMany(mappedBy = "user")
    private Set<JobApplication> jobApplications = new LinkedHashSet<>();
}
```

When using @ManyToMany is not idle

eazy bytes

Applying a job in Service layer

```
JobApplication application = new JobApplication();
application.setUser(user);
application.setJob(job);
application.setStatus("PENDING");
jobApplicationRepository.save(application);
```

Withdraw application in Service layer

```
jobApplicationRepository.deleteByUserIdAndJobId(user.getId(), jobId);
```

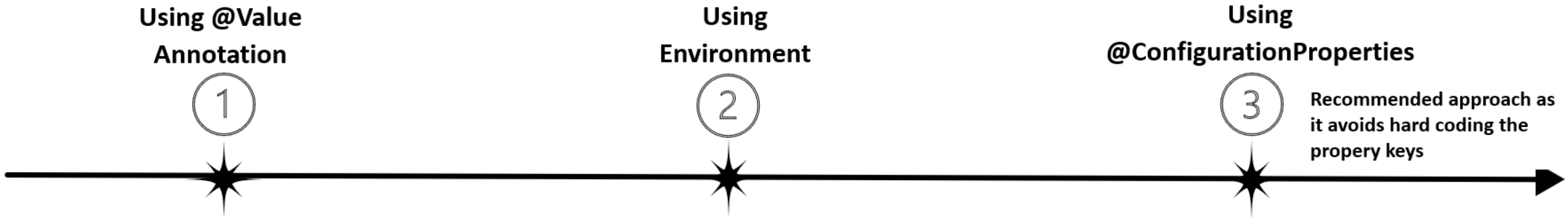
Fetching applied jobs for a user

```
JobPortalUser user = userRepository.findJobPortalUserByEmail(userEmail)
        .orElseThrow(() -> new RuntimeException("User not found with email: " + userEmail));
return user.getJobApplications().stream().map(this::mapToJobApplicationDto)
        .collect(Collectors.toList());
```

HOW TO READ PROPERTIES IN SPRINGBOOT APPs

eazy bytes

In Spring Boot, there are multiple approaches to reading properties. Below are the most commonly used approaches,



Using @Value Annotation

You can use the @Value annotation to directly inject property values into your beans. This approach is suitable for injecting individual properties into specific fields.
For example:

```
@Value("${property.name}")
private String propertyValue;
```

Using Environment

The Environment interface provides methods to access properties from the application's environment. You can autowire the Environment bean and use its methods to retrieve property values. This approach is more flexible and allows accessing properties programmatically. For example:

```
@Autowired
private Environment env;

public void getProperty() {

  String propertyValue =
      env.getProperty("property.name");
}
```

Using @ConfigurationProperties

The @ConfigurationProperties annotation enables binding of entire groups of properties to a bean. You define a configuration class with annotated fields matching the properties, and Spring Boot automatically maps the properties to the corresponding fields.

```
@ConfigurationProperties("prefix")
public class MyConfig {
   private String property;

   // getters and setters
}
```

@EnableConfigurationProperties needs to mention on top of the SpringBoot main class, to enable the above feature

READING PROPERTIES USING @VALUE

eazy bytes

```
@Configuration
public class CaffeineCacheConfig {

    @Value("${cache.jobs.ttl-minutes:5}")
    private int jobsCacheTtlMinutes;

    @Value("${cache.jobs.max-size:2000}")
    private int jobsCacheMaxSize;


    @Bean
    public CacheManager caffeineCacheManager() {

        CaffeineCache jobsCache = new
                  CaffeineCache("jobs",Caffeine.newBuilder()
       .expireAfterWrite(jobsCacheTtlMinutes, TimeUnit.MINUTES)
                        .maximumSize(jobsCacheMaxSize).build());

        SimpleCacheManager manager = new SimpleCacheManager();
        manager.setCaches(Arrays.asList(jobsCache));
        return manager;
    }

}
```

Properties inside application.properties

```
cache.jobs.ttl-minutes=10
cache.jobs.max-size=5000
```

READING PROPERTIES USING Environment

eazy bytes

```
@Component
@RequiredArgsConstructor
public class JwtUtil {

    private final Environment env;

    public String generateJwtToken(Authentication authentication){
        String jwtToken;
        String secret = env.getProperty(ApplicationConstants.JWT_SECRET_KEY,
                ApplicationConstants.JWT_SECRET_DEFAULT_VALUE);
        SecretKey secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        var fetchedUser = (JobPortalUser) authentication.getPrincipal();
        jwtToken = Jwts.builder().issuer("Job Portal").subject("JWT Token")
                .claim("name", fetchedUser.getName())
                .claim("email", fetchedUser.getEmail())
                .claim("mobileNumber", fetchedUser.getMobileNumber())
                .claim("roles", authentication.getAuthorities().stream().map(
                        GrantedAuthority::getAuthority).collect(Collectors.joining(",")))
                .issuedAt(new java.util.Date())
                .expiration(new java.util.Date((new java.util.Date()).getTime() + 24 * 60 * 60 * 1000))
                .signWith(secretKey).compact();
        return jwtToken;
    }
}
```

READING PROPERTIES USING @ConfigurationProperties

eazy bytes

Properties inside application.properties with a common prefix

```
app.cors.allowed-origins=http://localhost:5173
app.cors.allowed-methods=*
app.cors.allowed-headers=*
app.cors.allow-credentials=true
app.cors.max-age=3600
```

A POJO class representing the properties,

```
@ConfigurationProperties("app.cors")
@Getter @Setter
public class CorsProperties {

    private List<String> allowedOrigins;

    private List<String> allowedMethods;

    private List<String> allowedHeaders;

    private Boolean allowCredentials;

    private Long maxAge;

}
```

Spring Boot main class

```
@SpringBootApplication
@EnableConfigurationProperties(value = {CorsProperties.class})
public class JobportalApplication {

    public static void main(String[] args) {
        SpringApplication.run(JobportalApplication.class, args);
    }

}
```

READING PROPERTIES USING @ConfigurationProperties

eazy bytes

Injecting CorsProperties and using it inside the code,

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class JobPortalSecurityConfig {


    private final CorsProperties corsProperties;

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(corsProperties.getAllowedOrigins());
        config.setAllowedMethods(corsProperties.getAllowedMethods());
        config.setAllowedHeaders(corsProperties.getAllowedHeaders());
        config.setAllowCredentials(corsProperties.getAllowCredentials());
        config.setMaxAge(corsProperties.getMaxAge());

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

Reading Properties with @PropertySource

eazy bytes

What is @PropertySource?

@PropertySource is an annotation in Spring that tells the application to load properties from an external file.
This helps you keep configurations separate from your code.

Why use @PropertySource?

To load custom properties files other than application.properties
To organize configuration logically

How to use it?

You add @PropertySource in any Spring managed bean class:

```
@Component
@PropertySource("classpath:jwt.properties")
public class JwtUtil {

}
```

This will load the file jwt.properties from src/main/resources

Reading Properties with @PropertySource

eazy bytes

Accessing properties with @Value

```
@Component
@PropertySource("classpath:jwt.properties")
public class JwtUtil {

    @Value("${jwt.issuer:Job Portal}")
    private String jwtIssuer;

    @Value("${jwt.subject:JWT Token}")
    private String jwtSubject;

    @Value("${jwt.expiration.hours:1}")
    private int jwtExpirationHours;

}
```

You can load multiple files as well using the below syntax:

```
@PropertySource({"classpath:db.properties",
        "classpath:app.properties"})
```

Reading Properties with @PropertySource

eazy bytes

ignoreResourceNotFound in @PropertySource

By default, Spring throws an error if the specified properties file does not exist.
Use ignoreResourceNotFound = true to avoid this error.
This makes your application more flexible if the file is optional.

```
@Configuration
@PropertySource(
    value = "classpath:optional-config.properties",
    ignoreResourceNotFound = true
)
public class AppConfig {
}
```

When to use?

- When the file is optional
- For external properties files which may not exist in all environments
- To avoid application startup failures

EXTERNALIZING PROPERTIES

eazy bytes

Spring Boot allows you to externalize configuration/properties, so the same code works in different environments.

By default, Spring Boot looks for application.properties or application.yml in the classpath.

Spring Boot follows a specific order of precedence for configurations.
Later sources override earlier ones:

- Properties in application.properties or application.yml
- Operating system environment variables
- Java system properties (System.getProperties())
- JNDI attributes (java:comp/env)
- ServletContext init parameters
- ServletConfig init parameters
- Command-line arguments

This order makes it easy to override values for different environments without changing code.

Externalize configurations using Environment variables

eazy bytes

Environment variables are popular for external configuration because they work across all operating systems. Java can read them using System.getenv().

To map an environment variable to a Spring property:

- **Change letters to uppercase**
- **Replace dots (.) and dashes (-) with underscores (_)**

Example:

`SPRING_JPA_SHOW_SQL` **→ maps to Spring property** `spring.jpa.show-sql`

This automatic conversion is called relaxed binding in Spring Boot.

Windows

```
env:SPRING_JPA_SHOW_SQL=false; java -jar jobportal.jar
```

Linux based OS

```
SPRING_JPA_SHOW_SQL=false java -jar jobportal.jar
```

Externalize configurations using JVM System properties

eazy bytes

JVM system properties can also override Spring properties, but have slightly lower priority than command-line arguments.

You can pass JVM properties using -Dproperty.name=value when starting the app.
This helps you change settings without rebuilding the JAR.

```
java -Dspring.jpa.show-sql=false -jar jobportal.jar
```

If both a JVM property and a command-line argument are provided, Spring Boot uses the command-line argument value, as it has the highest priority.

Externalize configurations using command-line arguments

eazy bytes

Spring Boot converts command-line arguments into key/value pairs and adds them to the Environment object.

In production, these arguments have the highest priority.

You can override configuration by passing command-line arguments when running your JAR.

```
java -jar jobportal.jar --spring.jpa.show-sql=false
```

The command-line argument follows the same naming convention as the corresponding Spring property, with the familiar -- prefix for CLI arguments.

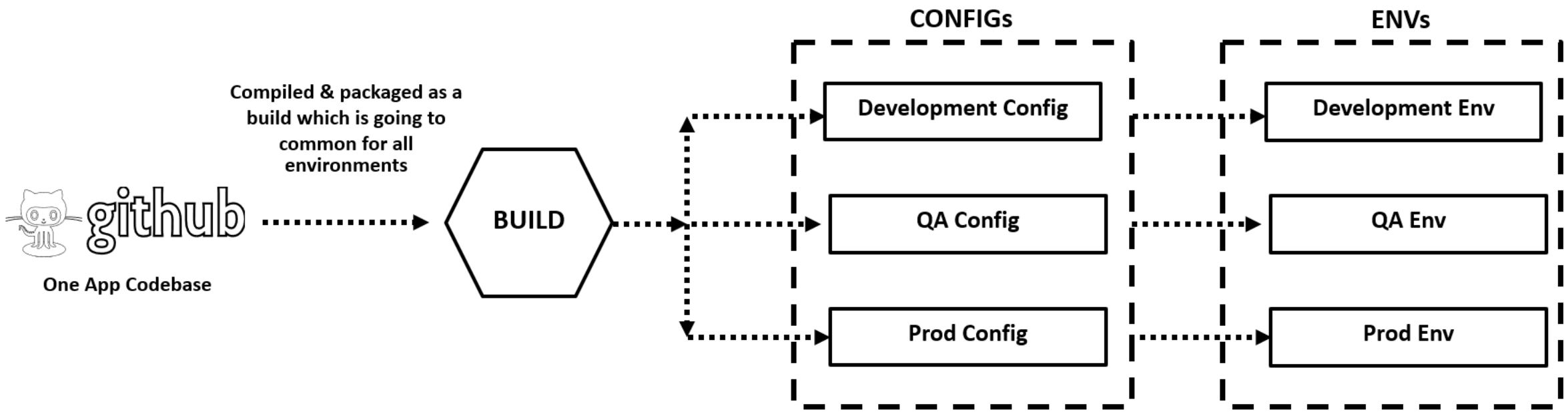
Environment-Specific Configuration

eazy bytes

Imagine our backend app has following needs across multiple environments:

- Development: Define jobs cache with 10 minutes TTL, enable SQL logging, JWT token expiration to 24 hours
- Testing: Define jobs cache with 15 minutes TTL, enable SQL logging, JWT token expiration to 24 hours
- Production: Define jobs cache with 5 minutes TTL, disable SQL logging, JWT token expiration to 1 hour

Changing the application.properties file manually before the deployment of the app into a environment is a cumbersome and error prone process. We need a better and automated approach



Spring Boot Profiles

eazy bytes

What Are Spring Boot Profiles?

Spring Boot profiles allow us to create multiple configurations for different environments in a single application.

Example environments:

- Development (dev)
- Testing (test)
- Staging (staging)
- Production (prod)

You can define different properties, beans, or behaviors for each environment.

Why Do We Need Profiles?

Imagine an application running:

- On a developer's laptop → uses local database
- On QA server → uses test database
- On production → uses secure production database

The same code but different configurations
Profiles avoid manual code changes between environments

Spring Boot Profiles

eazy bytes

Analogy - Clothing for Seasons

Think of Spring profiles like seasonal clothing:

Summer profile → Light clothes
Winter profile → Warm clothes
Rainy profile → Raincoat and boots

Your body (application) is same,
clothing (profile configuration) changes depending on weather (environment).

Similarly:
Your application is same,
Profiles change configuration for the environment it runs in.

What Can You Change with Profiles?

You can customize almost anything!

- application.properties files
- Database connection
- API URLs
- Logging levels
- Bean creation (conditional beans)

Spring Boot Profiles

eazy bytes

How to Define Profiles?

Define profile-specific properties in files like `application-<profile>.yml or application-<profile>.properties`

For examples,

`application-prod.properties` for prod env/profile
`application-qa.properties` for qa env/profile

The default profile is always active. Spring Boot loads all properties in application.properties into the default profile.

We can activate a specific profile using spring.profiles.active property either by directly mentioning inside the corresponding file or by providing externally using CLI, Env variables etc.

Activate profiles via:

Command-line: --spring.profiles.active=prod
Environment variable: SPRING_PROFILES_ACTIVE=prod
Application properties: spring.profiles.active=prod

Spring Boot Profiles

eazy bytes

Executing conditional logic based on the active profile

```
@Value("${jwt.expiration.hours:1}")
private int jwtNonProdExpirationHours;

@Value("${jwt.prod.expiration.hours:1}")
private int jwtProdExpirationHours;

List<String> profiles = Arrays.asList(env.getActiveProfiles());

int jwtExpirationHours;
if(profiles.contains("prod")) {
    jwtExpirationHours = jwtProdExpirationHours;
} else {
    jwtExpirationHours = jwtNonProdExpirationHours;
}
```

Conditional Bean Creation in Spring Boot

eazy bytes

What is Conditional Bean Creation?

Sometimes you want to create a bean only if a certain condition is true. Spring Boot provides powerful annotations to create beans conditionally:

This is useful for:

- Switching configurations
- Enabling features for certain environments
- Avoiding unnecessary beans in production

@ConditionalOnProperty

Create a bean **only if a property exists or has a specific value.

```
@Bean
@ConditionalOnProperty(name = "feature.enabled",
        havingValue = "true")
public MyService myService() {
    return new MyService();
}
```

If you set feature.enabled=true in application.properties, the bean will be created.

Conditional Bean Creation in Spring Boot

eazy bytes

@Profile

Create a bean only for a specific profile.

```
@Profile("dev")
@Bean
public DataSource devDataSource() {
    return new DevDataSource();
}
```

This bean is active only when spring.profiles.active=dev

@ConditionalOnMissingBean

Create a bean only if the bean does not already exist.

```
@Bean
@ConditionalOnMissingBean
public MyService myService() {
    return new MyService();
}
```

Useful for providing default beans that users can override.

Conditional Bean Creation in Spring Boot

eazy bytes

@ConditionalOnClass

Create a bean only if a specific class exists in the classpath.

```
@Bean
@ConditionalOnClass(name = "com.example.SomeLibrary")
public MyLibraryBean myLibraryBean() {
    return new MyLibraryBean();
}
```

Great for auto-configurations.

Benefits of Conditional Beans

- Clean & flexible configuration
- Activate features only when needed
- Reduce memory and startup time
- Avoid duplicate beans

The above concepts applicable for stereotype annotations as well.

Spring Boot Actuator

eazy bytes

Spring Boot Actuator is a tool that helps monitor and manage a Spring Boot application. It provides ready-made REST APIs to check the health, metrics, and status of your app.

With Actuator, you can:

- Check if the app is running properly.
- Get system metrics (memory, CPU, threads).
- Monitor HTTP requests, database connections.
- See which Spring Beans are loaded.

To enable Actuator, add this dependency in pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Spring Boot Actuator

eazy bytes

By default, only the health check (/actuator/health) endpoint is enabled.

To enable more endpoints, update application.properties:

```
management.endpoints.web.exposure.include=*
```

You can add custom information in application.properties:

```
management.info.env.enabled=true
info.app.name=jobportal
info.app.description=Job Portal REST API
info.app.version=1.0.0
```

Now, if you visit /actuator/info, you will see:

```
{
  "app": {
    "name": "jobportal",
    "description": "Job Portal REST API",
    "version": "1.0.0"
  }
}
```

Spring Boot Actuator

eazy bytes

To check Application Health -> Access the path http://localhost:8080/actuator/health to see below response,

```
{
  "status": "UP"
}
```

To view Metrics -> Access the path
http://localhost:8080/actuator/metrics to see response like below,

```
{
  "names": [
    "jvm.memory.used",
    "system.cpu.usage",
    "http.server.requests",
    "logback.events"
  ]
}
```

To get specific metric details like CPU usage, visit: http://localhost:8080/actuator/metrics/system.cpu.usage

Common Actuator Endpoints

eazy bytes

Endpoint	What It Does
/actuator/health	Shows the overall health status of the application and its dependencies
/actuator/info	Displays custom application information like build or version details
/actuator/metrics	Provides runtime metrics such as memory, CPU, and HTTP request statistics
/actuator/env	Exposes application environment properties and their current values
/actuator/beans	Lists all Spring beans loaded in the application context
/actuator/mappings	Shows all HTTP endpoints and their request mappings
/actuator/loggers	Allows viewing and changing logging levels at runtime
/actuator/threaddump	Displays a snapshot of all running threads and their states
/actuator/conditions	Shows why Spring Boot created or skipped beans based on auto-configuration conditions
/actuator/configprops	Displays all configuration properties bound to @ConfigurationProperties classes
/actuator/sbom	Exposes the Software Bill of Materials, listing all dependencies used in the application
/actuator/scheduledtasks	Lists all scheduled jobs and cron tasks configured in the application
/actuator/caches	Shows all caches and cache managers currently used by the application

Spring Boot Actuator

eazy bytes

Is Actuator Safe? Yes! But you should secure it in production.

By default:

Sensitive data is protected.
Only health and info are public.
You can restrict access using Spring Security.

By default, all web endpoints are available beneath the path /actuator. The base path can be updated using the below property,

```
management.endpoints.web.base-path=/manage
```

With the above property, the URL info endpoint would become `/manage/info`

Common Actuator usecases

eazy bytes

Evict All Caches

To clear all available caches, make a DELETE request to /actuator/caches as shown in the following curl-based example:

```
$ curl 'http://localhost:8080/actuator/caches' -i -X DELETE
```

Evict a Cache by Name

To evict a particular cache, make a DELETE request to /actuator/caches/{name} as shown in the following curl-based example:

```
$ curl 'http://localhost:8080/actuator/caches/countries'
 -i -X DELETE -H 'Content-Type: application/x-www-form-urlencoded'
```

Common Actuator usecases

eazy bytes

Setting a Log Level

To set the level of a logger, make a POST request to /actuator/loggers/{logger.name} with a JSON body that specifies the configured level for the logger, as shown in the following curl-based example:

```
$ curl 'http://localhost:8080/actuator/loggers/com.example' -i -X POST \
    -H 'Content-Type: application/json' \
    -d '{"configuredLevel":"debug"}’
```

The preceding example sets the configuredLevel of the com.example logger to DEBUG.

Clearing a Log Level

To clear the level of a logger, make a POST request to /actuator/loggers/{logger.name} with a JSON body containing an empty object, as shown in the following curl-based example:

```
$ curl 'http://localhost:8080/actuator/loggers/com.example' -i -X POST \
    -H 'Content-Type: application/json' -d '{}’
```

The preceding example clears the configured level of the com.example logger.

Common Actuator usecases

eazy bytes

Shutting Down the Application

To shut down the application, make a POST request to /actuator/shutdown, as shown in the following curl-based example:

```
$ curl 'http://localhost:8080/actuator/shutdown' -i -X POST
```

WHAT IS OBSERVABILITY ?

eazy bytes

Observability is the ability to understand the internal state of a system by observing its outputs. In today's cloud-native world, observability is essential. **Metrics** tell you how your application behaves, traces show how requests move across classes, and logs reveal what the application is communicating at every step.

The three pillars of observability are:

Metrics: Metrics are quantitative measurements of the health of a system. They can be used to track things like CPU usage, memory usage, and response times.

Logs: Logs are a record of events that occur in a system. They can be used to track things like errors, exceptions, and other unexpected events.

Traces: Traces are a record of the path that a request takes through a system. They can be used to track the performance of a request and to identify bottlenecks.

By collecting and analyzing data from these three sources, you can gain a comprehensive understanding of the internal state of your backend application. This understanding can be used to identify and troubleshoot problems, improve performance, and ensure the overall health of your system.

WHAT IS OBSERVABILITY ?

Spring Boot treats observability as a first-class feature, not an afterthought. With Spring Boot 4.0:

- Observability setup is simple
- Uses industry standards
- Avoids vendor lock-in

`spring-boot-starter-opentelemetry` (One Starter to Rule Them All)

New official starter in Spring Boot 4.0. Available directly from start.spring.io
Automatically wires Metrics, Traces, OTLP export
This is the recommended and preferred way to enable observability in Spring Boot

The OpenTelemetry starter brings together:

- OpenTelemetry API
- Micrometer → OpenTelemetry bridge
- OTLP exporters for metrics & traces
- Spring Boot auto-configuration
- No manual wiring
- No vendor-specific code

WHAT IS OpenTelemetry ?

eazy bytes

OpenTelemetry is an observability framework and toolkit designed to facilitate the Generation, Export, Collection of telemetry data such as traces, metrics, and logs.

Open source, as well as vendor- and tool-agnostic, meaning that it can be used with a broad variety of observability backends, including open source tools like Grafana, Jaeger and Prometheus, as well as commercial offerings. OpenTelemetry is not an observability backend itself.

A major goal of **OpenTelemetry** is to enable easy instrumentation of your applications and systems, regardless of the programming language, infrastructure, and runtime environments used.

The backend (storage) and the frontend (visualization) of telemetry data are intentionally left to other tools.

The name **OpenTelemetry** is a combination of Open + Telemetry

The **word** Open highlights that the project is open-source and vendor-neutral.

Telemetry comes from the Greek roots tele (remote) and metron (measure). In software, it refers to the data collected from your applications to monitor their health from a distance.

Implementing Observability in Spring Boot

eazy bytes

Think of observability as a pipeline:

Your Spring Boot App → Telemetry Data → Transport → Backend → Dashboards

Spring Boot (Your application)

Generates business logic, handles requests, talks to DB, etc. Does NOT natively know how to expose metrics/traces/logs in a standardized way. Needs observability libraries

Micrometer (Spring's Observability Facade)

Micrometer is Spring's abstraction layer for observability. Think of it as SLF4J for metrics & tracing.

- Collects metrics (CPU, memory, request count, latency), Instruments traces (via Micrometer Tracing)
- Integrates deeply with Spring Boot Actuator

Micrometer does not store data. Micrometer only collects Telemetry data and forwards

Implementing Observability in Spring Boot

eazy bytes

OpenTelemetry (OTel)

Standard observability framework. Defines:

- How telemetry data looks
- How it should be collected
- How it should be exported

OpenTelemetry provides APIs, SDKs, Vendor-neutral standards
Micrometer can bridge to OpenTelemetry

OTLP (OpenTelemetry Protocol)

OTLP is the wire protocol. It defines how telemetry data is sent
Works over:

- HTTP
- gRPC

OTLP is to observability what HTTP is to REST.

Because of OTLP:

- Spring Boot doesn’t care where data goes
- Backends don’t care who produced the data

Implementing Observability in Spring Boot

eazy bytes

LGTM Stack (The Backend)

LGTM is just a popular observability backend stack

Letter	Tool	Signal
L	Loki	Logs (log aggregation system)
G	Grafana	UI (for visualization and dashboards)
T	Tempo	Traces (distributed tracing backend)
M	Mimir	Metrics (long-term storage for Prometheus metrics)

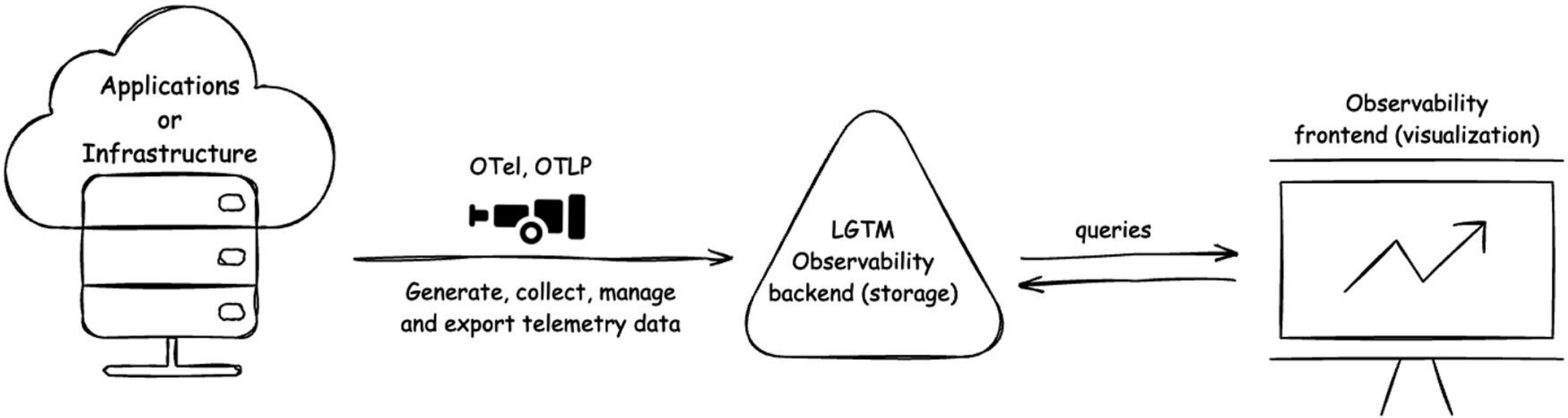
These tools store, query, and visualize telemetry data. They do not collect data from your app directly

Implementing Observability in Spring Boot

eazy bytes

Think of observability as a pipeline:

Your Spring Boot App → Telemetry Data → Transport → Backend → Dashboards



Implementing Observability in Spring Boot

eazy bytes

Step 1 – Adding below dependencies in pom.xml

```
<dependency>
        <groupId>io.opentelemetry.instrumentation</groupId>
        <artifactId>opentelemetry-logback-appender-1.0</artifactId>
        <version>2.24.0-alpha</version>
</dependency>
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
```

Enable OpenTelemetry support in Spring Boot
Export Micrometer metrics & traces via OTLP
Send application logs to OpenTelemetry

Logs normally stay in Console, Log files.
OpenTelemetry Logback Appender converts logs into telemetry data. Sends logs via OTLP

Implementing Observability in Spring Boot

eazy bytes

Step 2 – Adding below config inside Docker Compose file for LGTM Stack

```
services:
  grafana-lgtm:
    image: 'grafana/otel-lgtm:latest'
    ports:
      - '3000:3000'
      - '4317:4317'
      - '4318:4318'
```

Port	Purpose
3000	Grafana UI
4317	OTLP (gRPC)
4318	OTLP (HTTP)

Provides Grafana (UI), Tempo (Traces), Loki (Logs) and Mimir (Metrics). Includes an OTLP receiver.

Step 3 – Adding below properties inside application.properties file,

```
management.tracing.sampling.probability=1.0
management.otlp.metrics.export.url=http://localhost:4318/v1/metrics
management.opentelemetry.tracing.export.otlp.endpoint=http://localhost:4318/v1/traces
management.opentelemetry.logging.export.otlp.endpoint=http://localhost:4318/v1/logs
management.otlp.metrics.export.step=10s
```

Provides Grafana (UI), Tempo (Traces), Loki (Logs) and Mimir (Metrics). Includes an OTLP receiver.

Implementing Observability in Spring Boot

eazy bytes

Step 4 – OpenTelemetry Appender Initializer

```
@Component
public class OpenTelemetryAppenderInitializer implements InitializingBean {

    private final OpenTelemetry openTelemetry;

    OpenTelemetryAppenderInitializer(OpenTelemetry openTelemetry) {
        this.openTelemetry = openTelemetry;
    }

    @Override
    public void afterPropertiesSet() {
        OpenTelemetryAppender.install(this.openTelemetry);
    }

}
```

Logback does not know about OpenTelemetry unless we explicitly connect them. Above code connects Logback with OpenTelemetry. Makes OTEL appender active at runtime

Implementing Observability in Spring Boot

eazy bytes

Step 5 – Logback Configuration

```
<appender name="OTEL"
class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender" />
```

The above appender sends logs to OpenTelemetry by using OTLP protocol. Makes logs visible in Grafana Loki

Console Pattern with Trace Context

```
traceId=%X{traceId} spanId=%X{spanId}
```

Step 6 – Accessing Grafana

```
http://localhost:3000
```

Inside Grafana, you can check

- Metrics dashboards
- Distributed traces
- Correlated logs

Making Trace IDs Visible to Clients

eazy bytes

```
@RestControllerAdvice
@RequiredArgsConstructor
public class GlobalExceptionHandler {

    private final Tracer tracer;

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleException(Exception exception, WebRequest webRequest) {
        TraceContext context = tracer.currentTraceContext().context();
        String traceId = "";
        if (context != null) {
            traceId = context.traceId();
        }
        ErrorResponseDto errorResponseDto = new ErrorResponseDto(
                webRequest.getDescription(false), HttpStatus.INTERNAL_SERVER_ERROR,
                exception.getMessage(), LocalDateTime.now(),traceId);
        return new ResponseEntity<>(errorResponseDto, HttpStatus.INTERNAL_SERVER_ERROR);
    }

}
```

eazy bytes

Consuming REST APIs using

RestClient

eazy bytes

RestClient

The evolution of REST clients in Spring

RestTemplate → Deprecated (maintenance mode)
WebClient → Reactive, but overkill for simple sync calls
RestClient → Modern, simple, synchronous, fluent API

RestClient = RestTemplate + WebClient style

RestClient is a synchronous, blocking REST client.

Adding RestClient Dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-restclient</artifactId>
</dependency>
```

Key Characteristics

- Fluent & readable API
- Built on top of WebClient internals
- Ideal for microservice-to-microservice calls
- Designed as a replacement for RestTemplate

Spring’s future direction = RestClient

RestClient Configuration

eazy bytes

```
public RestClientTodoService(RestClient.Builder builder) {
    this.restClient = builder
        .baseUrl("https://jsonplaceholder.typicode.com")
        .defaultHeader("Accept", "application/json")
        .build();
}
```

Centralized configuration
Reusable across methods
Clean constructor injection (Spring-friendly)

GET: Fetching a List of Resources

```
public List<TodoDto> findAll() {
        return restClient.get()
                .uri(TODOS_API)
                .retrieve()
                .onStatus(HttpStatusCode::isError, (req, res) ->
                        new IllegalStateException("Failed to fetch todos"))
                .body(new ParameterizedTypeReference<>() {
                });
    }
```

RestClient Configuration

eazy bytes

GET by ID (Path Variables)

```
public TodoDto findById(Long id) {
        return restClient.get()
                .uri(TODOS_API + "/{id}", id)
                .retrieve()
                .onStatus(HttpStatusCode::is4xxClientError, (req, res) ->
                        new IllegalArgumentException("Todo not found with id: " + id))
                .body(TodoDto.class);
    }
```

Creating a Resource

```
public TodoDto create(TodoDto todo) {
    return restClient.post()
        .uri("/todos")
        .body(todo)
        .retrieve()
        .body(TodoDto.class);
}
```

RestClient Configuration

eazy bytes

PUT: Updating a Resource

```
public TodoDto update(Long id, TodoDto todo) {
        return restClient.put()
                .uri(TODOS_API + "/{id}", id)
                .body(todo)
                .retrieve()
                .body(TodoDto.class);
    }
```

DELETE: Removing a Resource

```
public void delete(Long id) {
        restClient.delete()
                .uri(TODOS_API+ "/{id}", id)
                .retrieve()
                .toBodilessEntity();
    }
```

eazy bytes

HTTP Service Client

What Problem Does This Solve ?

Traditional Approach
- Write service classes
- Build URLs manually
- Repeat RestClient logic

Declarative HTTP Client Approach
- Define what API looks like
- Spring handles how to call it

You focus on business logic, not HTTP plumbing.

How HTTP Service Client works ?

Step 1: @HttpExchange: The Entry Point

```
@HttpExchange
public interface TodoService
```

Marks interface as an HTTP client
Spring generates a runtime proxy
Methods become HTTP calls

Step 2: HTTP Method Mapping with Exchange Annotations

```
@GetExchange
@PostExchange
@PutExchange
@DeleteExchange
```

Maps Java methods → HTTP verbs
No need for RestClient code
Clear & readable API contract

HTTP Service Client

eazy bytes

// Path Variables

```
@GetExchange("/{id}")
TodoDto findById(@PathVariable Long id);
```

// Request Body

```
@PostExchange
TodoDto create(@RequestBody TodoDto todo);
```

Spring automatically handles Path variable substitution, JSON serialization/deserialization, Content-Type handling which results into Zero manual mapping code

@ImportHttpServices Explained

```
@ImportHttpServices(group = "todos", types = {TodoService.class})
@ImportHttpServices(group = "posts", types = {PostService.class})
```

What this does

- Registers HTTP clients at startup
- Groups clients logically
- Allows group-specific configuration
- Think of group = logical client category

Why Do We Need Groups?

Without Groups
- Same base URL
- Same headers
- Same interceptors

With Groups
- Different base URLs
- Different headers
- Different auth strategies
- Perfect for real-world systems

HTTP Service Client

eazy bytes

Centralized Client Configuration

```
@Configuration
@ImportHttpServices(group = "todos", types = {TodoService.class})
@ImportHttpServices(group = "posts", types = {PostService.class})
public class HttpServiceClientConfig {

    @Bean
    public RestClientHttpServiceGroupConfigurer groupConfigurer() {
            return groups -> {
                groups.filterByName("todos").forEachClient(
                        (group, restClientBuilder) -> {
                            restClientBuilder.baseUrl("https://jsonplaceholder.typicode.com/todos")
                                    .requestInterceptor((request, body, execution) -> {
                                        //request.getHeaders().setBearerAuth("Bearer Token");
                                        // request.getHeaders().add("Header Name", "Header Value");
                                        return execution.execute(request, body);
                                    }).build();
                        }
                );
                  .... Other group configurations
             };
    }

}
```

Deploying Spring Boot Applications

eazy bytes

Spring Boot's flexible packaging options provide a great deal of choice when it comes to deploying your application.

You can deploy Spring Boot applications to a variety of cloud platforms, to virtual/real machines

The SpringBoot Apps can be deployed to

- ✓ Public Clouds like AWS, Azure, GCP
- ✓ Kubernetes
- ✓ Heroku
- ✓ OpenShift etc.

majority of the Organizations, let's

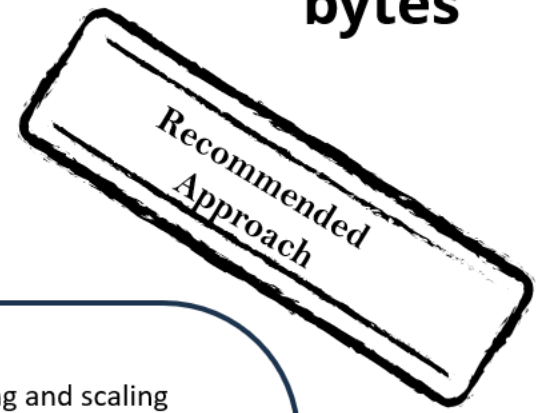
exploSince **AWS** is a famous cloud provider used by re on how to deploy a SpringBoot application using AWS

The common approaches to deploy a Spring Boot inside AWS is using either **EC2** or

Elastic Beanstalk

AWS EC2 vs AWS Elastic Beanstalk

eazy bytes



Amazon Elastic Compute Cloud (EC2) is a virtual cloud infrastructure service offered by AWS that provides users on-demand computing resources through which users can create powerful servers in the cloud. Additionally, EC2 enables users to get a virtual machine up and running in just a few clicks.

With EC2, we can deploy our Spring Boot application into AWS Cloud. But it is a traditional approach and has many drawbacks. When used EC2, the Developer or Organization, **need to take care of installing required software, libraries like Java, Tomcat and handle Auto scaling, load balancing etc. manually.**

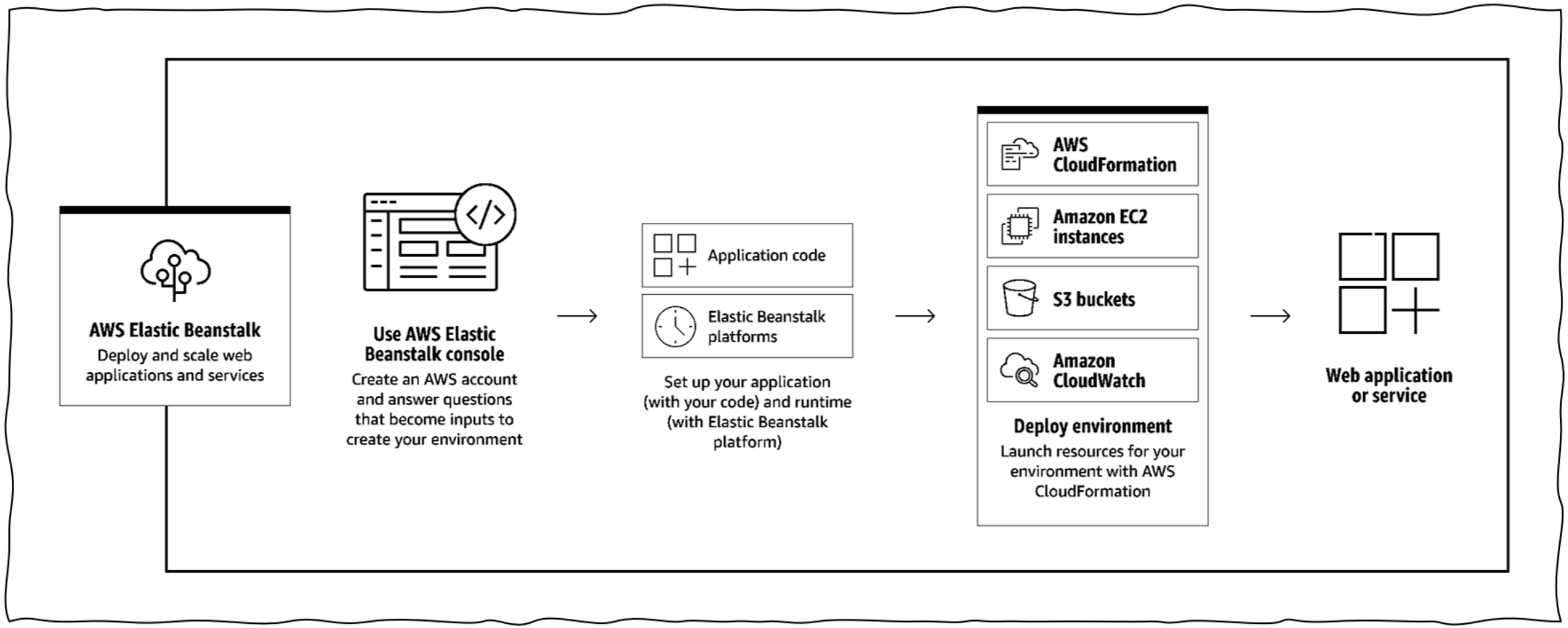
Elastic Beanstalk is a service for deploying and scaling web applications and services. Upload your code and Elastic Beanstalk automatically handles the deployment—from capacity provisioning, load balancing, and auto scaling to application health monitoring.

With Beanstalk, Developers can **focus on writing code instead of provisioning and managing infrastructure**.

Elastic Beanstalk supports applications developed in **Go, Java, .NET, Node.js, PHP, Python, and Ruby**. When you deploy your application, Elastic Beanstalk builds the selected supported platform version and provisions one or more AWS resources, such as Amazon EC2 instances, to run your application.

AWS Elastic Beanstalk - How it works

eazy bytes



SOURCE: https://aws.amazon.com/elasticbeanstalk/